Authentication / Security

- **How have you implemented token-based authentication?**

  In my projects, I have implemented token-based authentication using JWT (JSON Web Tokens) in a .NET-based Web API environment. I configured authentication middleware in ASP.NET Core using the built-in JWT Bearer authentication scheme. After validating user credentials (typically via database or Azure Active Directory), the system generates a signed JWT containing essential claims such as user ID, roles, and permissions. The token is signed using a secure secret key or certificate and returned to the client. On subsequent requests, the client sends the token in the Authorization header as a Bearer token. The middleware validates the token's signature, expiration time, issuer, and audience before granting access to protected endpoints. I have also implemented role-based and policy-based authorization using claims embedded within the token. Additionally, I ensured secure token handling by setting appropriate expiration times, enabling HTTPS, storing tokens securely on the client side, and implementing refresh token mechanisms where required to maintain session continuity without compromising security.
- **What is access token and refresh token?**

  An access token is a short-lived security credential issued after successful authentication that allows a user to access protected resources such as APIs. It contains claims like user identity, roles, and permissions, and is sent in the Authorization header as a Bearer token with each request. Because access tokens are typically short-lived (e.g., 15–60 minutes), they reduce security risks if compromised. A refresh token, on the other hand, is a long-lived token used to obtain a new access token without requiring the user to log in again. It is stored securely (usually in HttpOnly cookies or secure storage) and is sent only to the authentication server, not to resource APIs. When the access token expires, the client uses the refresh token to request a new access token, maintaining session continuity while preserving security.

- **How did you implement JWT token?**
  I implemented JWT-based authentication in ASP.NET Core by creating a secure token-issuance service and wiring up the built-in JWT Bearer middleware. After validating credentials against the user store, my service generated a signed JWT containing essential claims (user id, username, roles, and any custom permissions) and returned it to the client. The app's `Program.cs` configures the `JwtBearer` handler with issuer/audience validation, lifetime checks, and a signing key stored in a secure secret store (Key Vault or environment variable). Clients include the token in the `Authorization: Bearer <token>` header; middleware validates the token and populates `HttpContext.User`. I enforced role- and policy-based

authorization using claims and implemented refresh tokens to renew short-lived access tokens securely.

- **How do you authenticate users for particular resources?**
  I authenticate users for particular resources by implementing JWT-based authentication combined with role-based and policy-based authorization in ASP.NET Core. After a user logs in, the system issues a JWT containing claims such as user ID, roles, and permissions. When the user accesses a protected resource, the token is validated by the JWT middleware, which verifies the signature, issuer, audience, and expiration. Once validated, the claims are attached to the HttpContext. I then use role-based authorization using attributes like `[Authorize(Roles = "Admin")]` or policy-based authorization configured in `AddAuthorization()` to restrict access to specific resources. In more complex scenarios, I implement custom authorization handlers to check database-driven permissions before granting access. This ensures that users can only access resources they are explicitly authorized for.

3. Web API / HTTP / REST

- **Can we use a GET method instead of POST?**
  In general, a GET method should not be used instead of POST because each HTTP method has a specific purpose according to REST principles. GET is designed to retrieve data without modifying server state and should be idempotent and safe, meaning multiple identical requests produce the same result without side effects. POST, on the other hand, is used to create or process data and may change the server's state. Additionally, GET requests send parameters in the URL, which can be cached, logged, or bookmarked, whereas POST sends data in the request body. Therefore, the choice between GET and POST should be based on the nature of the operation—use GET for fetching data and POST when submitting or modifying data.

When should we use PUT vs POST vs PATCH?
PUT, POST, and PATCH are used for different types of data modification operations in REST APIs. POST is primarily used to create a new resource or trigger a processing operation, and it is not idempotent, meaning multiple identical requests may create multiple resources. PUT is used to completely update or replace an existing resource and is idempotent, meaning sending the same request multiple times produces the same result. PATCH is used for partial updates, where only specific fields of a resource need to be modified without replacing the entire object. Therefore, POST is used for creation, PUT for full replacement, and PATCH for partial modification.

🔎 Clear Comparison

Method	Purpose	Idempotent	Example Use Case
POST	Create new resource	❌ No	Create new user
PUT	Replace entire resource	✅ Yes	Update full user profile
PATCH	Partial update	Usually Yes	Update only email

✅ When to Use POST

- Creating a new resource
- When server generates the ID
- When action is not idempotent

Example:

```
POST /api/users
```

✅ When to Use PUT

- Updating entire resource
- Client knows resource ID
- Replace full object

Example:

```
PUT /api/users/10
```

Body contains complete user object.

✅ When to Use PATCH

- Updating specific fields only
- Avoid sending full object
- Improve performance

Example:

```
PATCH /api/users/10
```

Body:

```
{
  "email": "new@email.com"
}
```

POST is used to create resources, PUT is used to completely replace an existing resource, and PATCH is used to partially update specific fields of a resource.

What is content negotiation?

Content negotiation is a mechanism in HTTP that allows the client and server to agree on the format of the response data. The client specifies the desired media type using headers such as `Accept`, and the server returns the response in a format it supports, such as JSON, XML, or plain text. If the requested format is not supported, the server may return a `406 Not Acceptable` response. Content negotiation ensures flexibility in APIs by allowing the same endpoint to serve different representations of the same resource based on client requirements.

Client sends request with header:

```
Accept: application/json
```

or

```
Accept: application/xml
```

Server checks supported formatters and responds accordingly.

Request:

```
GET /api/users/10
Accept: application/xml
```

Response:

```
<User>
  <Id>10</Id>
  <Name>John</Name>
</User>
```

If:

```
Accept: application/json
```

Response:

```
{
  "id": 10,
  "name": "John"
}
```

Content negotiation allows the client to request a specific response format using the Accept header, and the server returns the resource in that format if supported.

Where do we come across content negotiation in Web API?
We come across content negotiation in Web API whenever a client requests a resource and specifies the desired response format using the `Accept` header. In ASP.NET Core Web API, content negotiation happens automatically when an action method returns an object (for example, `return Ok(user);`). The framework checks the `Accept` header sent by the client and selects an appropriate output formatter, such as JSON or XML, to serialize the response. It also applies during request handling when the `Content-Type` header determines how the incoming request body should be deserialized. Therefore, content negotiation is commonly encountered in RESTful APIs when supporting multiple media formats like JSON and XML.

1. **Returning API Responses**
2. `return Ok(user);`

   ASP.NET Core decides whether to return JSON or XML based on the `Accept` header.

3. **Supporting Multiple Formats**
   When your API supports:
   - `application/json`
   - `application/xml`
4. **Incoming Requests**
   When a client sends:
5. `Content-Type: application/json`

   The framework selects the correct input formatter to parse the body.

We encounter content negotiation in Web API when the client specifies a response format using the Accept header, and the framework automatically selects the appropriate formatter to serialize or deserialize data.

- **How do we define content negotiation in headers?**

  Content negotiation is defined in HTTP headers primarily using the `Accept` and `Content-Type` headers. The client specifies the desired response format using the `Accept` header (such as `application/json` or `application/xml`), and the server responds in a supported format. The `Content-Type` header indicates the format of the data being sent in the request body, informing the server how to parse it. Based on these headers, the server selects the appropriate input and output formatters to process and return the data.

4.Design Patterns

- **What design patterns have you used?**
  In my projects, I have primarily used design patterns that promote maintainability, scalability, and loose coupling. Some of the key patterns I have implemented include Repository Pattern for abstracting data access logic, Dependency Injection for managing object creation and reducing tight coupling, Factory Pattern for controlled object instantiation, Singleton Pattern for shared services like caching or logging, and Strategy Pattern for implementing interchangeable business rules. In ASP.NET Core applications, I have also followed MVC architecture and layered architecture patterns to separate concerns effectively.

What are SOLID principles?
SOLID is a set of five object-oriented design principles that help create maintainable and scalable software.

1. **S – Single Responsibility Principle (SRP)**
   A class should have only one reason to change.
2. **O – Open/Closed Principle (OCP)**
   Software entities should be open for extension but closed for modification.
3. **L – Liskov Substitution Principle (LSP)**
   Derived classes should be replaceable with base classes without breaking functionality.
4. **I – Interface Segregation Principle (ISP)**
   Clients should not be forced to depend on interfaces they do not use.
5. **D – Dependency Inversion Principle (DIP)**
   High-level modules should not depend on low-level modules; both should depend on abstractions.

Which design patterns have you followed?
In my projects, I have followed several design patterns to ensure clean architecture, maintainability, and scalability. Primarily, I have implemented the Repository Pattern to abstract data access logic and separate it from business logic, making the application easier to maintain and test. I have used Dependency Injection to achieve loose coupling between components and improve unit testing capabilities. I have applied the Unit of Work Pattern to manage transactions across multiple repositories efficiently. In scenarios requiring flexible business logic selection, I have implemented the Strategy Pattern. I have also used the Factory Pattern when object creation involved complex logic. Additionally, in ASP.NET Core applications, I naturally follow the MVC pattern for separation of concerns and layered architecture (Controller → Service → Repository) to structure enterprise-level applications effectively.

1Repository Pattern

- Separates data access logic
- Makes switching databases easier
- Improves testability

2Dependency Injection Pattern

- Reduces tight coupling
- Improves maintainability
- Enables easy mocking in unit tests

3 Unit of Work Pattern

- Manages transactions
- Ensures all operations succeed or fail together

4Strategy Pattern

- Allows dynamic selection of algorithms
- Useful for business rule variations

5Factory Pattern

- Centralizes object creation logic
- Useful when object creation is complex

6MVC Pattern

- Separates UI, business logic, and data access
- Default architectural pattern in ASP.NET Core

In which scenarios will we use Dependency Injection?
Dependency Injection is used when we want to reduce tight coupling between classes and improve testability, scalability, and maintainability. It is especially useful when a class depends on external services such as database repositories, logging services, email services, or APIs. Instead of creating objects inside a class using the `new` keyword, we inject the dependencies through the constructor, allowing us to switch implementations easily and mock them during unit testing.

Common scenarios:

- Database repositories
- Logging services
- External API clients
- Email/SMS services
- Caching services

Can you give an example?

❌ Without DI (Tightly Coupled)

```
public class OrderService
{
    private readonly EmailService _emailService = new EmailService();

    public void PlaceOrder()
    {
        _emailService.SendEmail();
    }
}
```

Problem: Hard to test and tightly coupled.

✅ With DI (Loosely Coupled)

```
public interface IEmailService
{
    void SendEmail();
}

public class EmailService : IEmailService
{
    public void SendEmail()
    {
        Console.WriteLine("Email Sent");
    }
}

public class OrderService
{
    private readonly IEmailService _emailService;

    public OrderService(IEmailService emailService)
```

```
    {
        _emailService = emailService;
    }

    public void PlaceOrder()
    {
        _emailService.SendEmail();
    }
}
```

Can you give me the syntax?

Registering in Program.cs

```
builder.Services.AddScoped<IEmailService, EmailService>();
builder.Services.AddScoped<OrderService>();
```

Usage via Constructor Injection

```
public class OrderController : ControllerBase
{
    private readonly OrderService _orderService;

    public OrderController(OrderService orderService)
    {
        _orderService = orderService;
    }
}
```

How do we follow constructor design pattern?

Constructor-based design follows the principle of injecting required dependencies through the constructor instead of creating them inside the class. This ensures that the class cannot exist without its required dependencies, making the design more robust and testable. It also aligns with the Dependency Inversion Principle of SOLID.

Example:

```
public class PaymentService
{
    private readonly ILogger _logger;

    public PaymentService(ILogger logger)
    {
        _logger = logger;
    }
}
```

This ensures the dependency is mandatory and initialized when the object is created.

How do we use dependency injection in constructor?

Step-by-Step Explanation:

1. Create interface.
2. Implement the interface.
3. Register in DI container.
4. Inject through constructor.

Example:

```
// Step 1
public interface ILoggerService
{
    void Log(string message);
}

// Step 2
public class LoggerService : ILoggerService
{
    public void Log(string message)
    {
        Console.WriteLine(message);
    }
}

// Step 3 (Program.cs)
builder.Services.AddScoped<ILoggerService, LoggerService>();

// Step 4
public class ProductService
{
    private readonly ILoggerService _logger;

    public ProductService(ILoggerService logger)
    {
        _logger = logger;
    }
}
```

5. ASP.NET Core

What are the benefits of ASP.NET Core?

ASP.NET Core is a modern, high-performance, cross-platform framework used to build web applications, APIs, and microservices. One of its main benefits is cross-platform support, allowing applications to run on Windows, Linux, and macOS. It is lightweight and modular, meaning only required packages are included, which improves

performance. ASP.NET Core has built-in Dependency Injection, making applications loosely coupled and easier to test. It provides high performance compared to the older .NET Framework due to its optimized request pipeline and Kestrel web server. It also supports middleware architecture, enabling flexible request handling. Additionally, it offers strong security features such as built-in authentication, authorization, data protection, and integration with Identity and JWT. Its support for cloud-native development, containerization (Docker), and seamless integration with Azure makes it highly suitable for modern distributed systems.

🔎 Key Benefits Explained

1Cross-Platform

Runs on:

- Windows
- Linux
- macOS

2High Performance

- Kestrel web server
- Optimized middleware pipeline
- One of the fastest web frameworks

3 Built-in Dependency Injection

- Promotes loose coupling
- Improves testability

4Modular & Lightweight

- Only required packages are added
- Improves startup performance

5Middleware Architecture

- Flexible request pipeline
- Custom middleware support

6Strong Security

- Authentication & Authorization
- JWT support
- OAuth, OpenID Connect
- Data Protection APIs

7 Cloud & Microservices Ready

- Docker support
- Azure integration
- Environment-based configuration

Why do we use middleware?
Middleware in ASP.NET Core is used to handle HTTP requests and responses as they move through the application's request processing pipeline. Each middleware component can inspect, modify, or short-circuit the request before passing it to the next component. We use middleware to implement cross-cutting concerns such as authentication, authorization, logging, exception handling, routing, CORS, and response compression. It provides a flexible and modular way to control how requests are processed, improving maintainability and separation of concerns.

Middleware is mainly used for:

1 Request Processing Pipeline

- Every HTTP request passes through middleware.
- Each component decides whether to:
    - Handle the request
    - Modify it
    - Pass it to the next middleware

2Cross-Cutting Concerns

Examples:

- Authentication (`UseAuthentication`)
- Authorization (`UseAuthorization`)
- Logging
- Global exception handling
- CORS
- Static file serving
- Routing

3Short-Circuiting Requests

A middleware can stop further execution.

Example:
If authentication fails → return 401 without hitting controller.

🔎 Example in Program.cs

```
app.UseExceptionHandler("/error");
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();
```

Each `Use` is a middleware added to the pipeline.

🔎 Custom Middleware Example

```
public class LoggingMiddleware
{
    private readonly RequestDelegate _next;

    public LoggingMiddleware(RequestDelegate next)
    {
        _next = next;
    }

    public async Task Invoke(HttpContext context)
    {
        Console.WriteLine("Request Incoming");
        await _next(context);
        Console.WriteLine("Response Outgoing");
    }
}
```

Register:

```
app.UseMiddleware<LoggingMiddleware>();
```

- **What is the benefit of using a startup page?**
  The benefit of using a startup page (Startup class in older ASP.NET Core versions or Program.cs in .NET 6+) is that it centralizes application configuration and middleware setup in one place. It is responsible for configuring services, dependency injection, middleware pipeline, routing, authentication, authorization, and other application-level settings. This separation keeps the application organized, maintainable, and scalable by clearly defining how the application starts and how HTTP requests are processed. It also improves readability and makes environment-based configuration easier to manage

- **What is the use of Startup class?**
  The Startup class in ASP.NET Core is used to configure the application's services and define the HTTP request processing pipeline. It contains two important methods: `ConfigureServices`, where we register services with the dependency injection container (such as database context, repositories, authentication, and custom services), and `Configure`, where we define the middleware components that handle incoming HTTP requests. The Startup class helps centralize application configuration, maintain separation of concerns, and make the application more organized and scalable.

What is the use of scoped service?
A scoped service in ASP.NET Core is used when we want a service instance to be created once per HTTP request and shared throughout that request. This means that during a single request, the same instance of the service is used wherever it is injected, but a new instance is created for each new request. Scoped services are commonly used for database contexts like `DbContext`, repositories, and business services that need to maintain request-level consistency but should not be shared across multiple users or requests.

Why We Use Scoped Services

1One Instance Per Request

- Created once when request starts
- Disposed automatically when request ends

2Common Use Cases

- Entity Framework `DbContext`
- Repository classes
- Business services that depend on DbContext

3Prevents Data Leakage

Since each request gets a new instance, data from one user request is not shared with another.

Registration Syntax

```
builder.Services.AddScoped<IUserService, UserService>();
```

🔎 Difference from Other Lifetimes

Lifetime	Instance Created
Singleton	Once for entire application
Scoped	Once per request
Transient	New instance every time requested

What is Singleton service?
A Singleton service in ASP.NET Core is a service that is created only once during the application's lifetime and shared across all requests and users. The same instance is reused every time it is injected. Singleton services are useful for stateless services, configuration services, caching mechanisms, logging services, or services that are expensive to create and do not depend on request-specific data. Since the same instance is shared across the application, singleton services must be thread-safe.

Key Characteristics of Singleton

- Created once when application starts
- Shared across all HTTP requests
- Lives until application stops
- Must be thread-safe

Registration Syntax

```
builder.Services.AddSingleton<ICacheService, CacheService>();
```

Common Use Cases

- Caching services
- Logging services
- Configuration services
- In-memory data stores

What is Transient service?

A Transient service in ASP.NET Core is a service that is created every time it is requested from the dependency injection container. This means a new instance is created each time the service is injected, even within the same HTTP request. Transient services are typically used for lightweight, stateless services that do not need to maintain shared state. Since a new object is created every time, it ensures complete isolation but may have a slight performance overhead if the service is heavy.

Key Characteristics of Transient

- New instance created every time it is injected
- No shared state
- Suitable for lightweight services
- Short-lived

Registration Syntax

```
builder.Services.AddTransient<IEmailService, EmailService>();
```

Example Scenario

If a controller depends on a Transient service and calls it multiple times within the same request, each injection will create a new instance.

When to Use Transient

- Email services
- Helper/utility services
- Stateless business logic services
- Formatting or calculation services

How do we access dependency injection before controller execution?
We can access Dependency Injection before controller execution by using Middleware, Filters, or by injecting services into the request pipeline components. Since middleware executes before the controller is reached, it is the most common way to access services early in the request lifecycle. In ASP.NET Core, services registered in the DI container can be injected directly into custom middleware constructors or into the `Invoke`/`InvokeAsync` method parameters. Similarly, action filters or custom authorization filters can also use dependency injection to access services before the controller action runs.

Using Middleware (Most Common Approach)

Middleware runs before controllers.

Example:

```
public class LoggingMiddleware
{
    private readonly RequestDelegate _next;
    private readonly ILoggerService _logger;

    public LoggingMiddleware(RequestDelegate next, ILoggerService
logger)
    {
        _next = next;
        _logger = logger;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        _logger.Log("Request started");
        await _next(context);
    }
}
```

Register:

```
app.UseMiddleware<LoggingMiddleware>();
```

This executes before the controller.

2Using Filters (Before Action Execution)

Filters run before controller actions.

```
public class CustomActionFilter : IActionFilter
```

```
{
    private readonly ILoggerService _logger;

    public CustomActionFilter(ILoggerService logger)
    {
        _logger = logger;
    }

    public void OnActionExecuting(ActionExecutingContext context)
    {
        _logger.Log("Before Controller Execution");
    }

    public void OnActionExecuted(ActionExecutedContext context) { }
}
```

Register:

```
builder.Services.AddScoped<CustomActionFilter>();
```

How do we use services in filters?
We use services in filters in ASP.NET Core by leveraging Dependency Injection. Since filters are part of the request pipeline, they can receive dependencies through constructor injection, just like controllers. To use services inside a filter, we first register the service in the DI container, then inject it into the filter's constructor, and finally register the filter properly (either globally, at controller level, or action level). When the request executes, the framework resolves the filter along with its dependencies automatically.

✅ Step-by-Step Explanation

1 Create a Service

```
public interface ILoggerService
{
    void Log(string message);
}

public class LoggerService : ILoggerService
{
    public void Log(string message)
    {
        Console.WriteLine(message);
    }
}
```

2 Create a Filter and Inject Service

```
using Microsoft.AspNetCore.Mvc.Filters;

public class CustomActionFilter : IActionFilter
{
    private readonly ILoggerService _logger;

    public CustomActionFilter(ILoggerService logger)
    {
        _logger = logger;
    }

    public void OnActionExecuting(ActionExecutingContext context)
    {
        _logger.Log("Before Controller Execution");
    }

    public void OnActionExecuted(ActionExecutedContext context)
    {
        _logger.Log("After Controller Execution");
    }
}
```

3 Register Services in Program.cs

```
builder.Services.AddScoped<ILoggerService, LoggerService>();
builder.Services.AddScoped<CustomActionFilter>();
```

4Apply Filter

◈ Option A – At Controller Level

```
[ServiceFilter(typeof(CustomActionFilter))]
public class HomeController : Controller
{
}
```

Option B – At Action Level

```
[ServiceFilter(typeof(CustomActionFilter))]
public IActionResult Index()
{
    return View();
}
```

Option C – Globally

```
builder.Services.AddControllers(options =>
{
    options.Filters.Add<CustomActionFilter>();
});
```

Alternative Approach: TypeFilter

If you don't want to register the filter explicitly:

```
[TypeFilter(typeof(CustomActionFilter))]
```

`TypeFilter` resolves dependencies automatically.

We use services in filters by injecting them through constructor injection. The filter is registered in the dependency injection container and applied using ServiceFilter, TypeFilter, or globally in the controller configuration.

How will we handle session management?
Session management in ASP.NET Core is handled by storing user-specific data across multiple HTTP requests. Since HTTP is stateless, session helps maintain user data such as user preferences, temporary form data, or authentication-related information during a browsing session. In ASP.NET Core, session is enabled by configuring distributed memory cache or a distributed store like Redis, adding session middleware, and then accessing session data using the `HttpContext.Session` object. For scalable applications, distributed session storage such as Redis is preferred over in-memory session to support load-balanced environments.

How Session Management Works in ASP.NET Core

1Enable Session in Program.cs

```
builder.Services.AddDistributedMemoryCache();
builder.Services.AddSession();
```

Add middleware:

```
app.UseSession();
```

⚠ Must be added before `MapControllers()`.

2Set Session Value

```
HttpContext.Session.SetString("UserName", "Ashok");
```

3Get Session Value

```
var userName = HttpContext.Session.GetString("UserName")
```

Production-Level Session Management

In real-world applications:

- Use **Distributed Cache (Redis)** instead of in-memory
- Configure session timeout
- Store minimal data in session
- Avoid storing sensitive information
- Use secure cookies (`Secure`, `HttpOnly`)

Example with Redis:

```
builder.Services.AddStackExchangeRedisCache(options =>
{
    options.Configuration = "localhost:6379";
});
```

Important Interview Points

- HTTP is stateless → session maintains state.
- In-memory session is not suitable for load-balanced environments.
- Session is stored on server, session ID stored in browser cookie.
- For APIs, JWT-based stateless authentication is preferred over session.

Session management is used to maintain user-specific data across multiple HTTP requests. In ASP.NET Core, it is configured using session middleware and a distributed cache like Redis for scalable applications.

How do we use AddSession and UseSession?

`AddSession()` is used to register session services in the dependency injection container, while `UseSession()` is used to enable session middleware in the HTTP request pipeline. In ASP.NET Core, both are required to properly configure and use session management. `AddSession()` prepares the application to use sessions, and `UseSession()` ensures that session handling is applied to incoming HTTP requests.

Step-by-Step Implementation

1Register Session Services (AddSession)

Inside `Program.cs`:

```
builder.Services.AddDistributedMemoryCache();  // Required for session
storage
builder.Services.AddSession(options =>
```

```
{
    options.IdleTimeout = TimeSpan.FromMinutes(20);
    options.Cookie.HttpOnly = true;
    options.Cookie.IsEssential = true;
});
```

👉 `AddDistributedMemoryCache()` provides storage for session data.
👉 `AddSession()` configures session settings like timeout and cookie behavior.

---

2 Enable Session Middleware (UseSession)

```
app.UseSession();
```

⚠ Important:
`UseSession()` must be placed **before** `MapControllers()` and after routing if used.

Example:

```
app.UseRouting();
app.UseSession();
app.UseAuthorization();
app.MapControllers();
```

---

3 Using Session in Controller

Set Value

```
HttpContext.Session.SetString("UserName", "Ashok");
```

Get Value

```
var name = HttpContext.Session.GetString("UserName");
```

How do we serialize and deserialize session data?

In ASP.NET Core, session only supports storing byte arrays, so to store complex objects in session, we must serialize them into a string or byte format (commonly JSON) before saving. When retrieving the data, we deserialize it back into the original object type. Typically, we use `System.Text.Json` or `Newtonsoft.Json` for serialization and deserialization.

🔎 Why Serialization is Needed

Session supports:

- `SetString()`
- `SetInt32()`
- `Set()` (byte[])

It does NOT directly support storing complex objects like classes.

So we:

1. Convert object → JSON string (Serialize)
2. Store JSON string in session
3. Retrieve JSON string
4. Convert back → Object (Deserialize)

✅ Example

1 Sample Model

```
public class User
{
    public int Id { get; set; }
    public string Name { get; set; }
}
```

---

2 Serialize and Store in Session

```
var user = new User { Id = 1, Name = "Ashok" };

var jsonData = JsonSerializer.Serialize(user);

HttpContext.Session.SetString("UserData", jsonData);
```

---

3 Retrieve and Deserialize

```
var jsonData = HttpContext.Session.GetString("UserData");

if (jsonData != null)
{
    var user = JsonSerializer.Deserialize<User>(jsonData);
}
```

🔎 Alternative Using Extension Methods (Best Practice)

To avoid repeating code, we create extension methods:

```
public static class SessionExtensions
{
    public static void SetObject<T>(this ISession session, string key,
T value)
    {
        session.SetString(key, JsonSerializer.Serialize(value));
    }

    public static T GetObject<T>(this ISession session, string key)
    {
        var value = session.GetString(key);
        return value == null ? default :
JsonSerializer.Deserialize<T>(value);
    }
}
```

Usage:

```
HttpContext.Session.SetObject("User", user);

var user = HttpContext.Session.GetObject<User>("User");
```

Since session only stores byte arrays or strings, we serialize complex objects into JSON before storing them and deserialize them back into objects when retrieving. This is typically done using System.Text.Json or Newtonsoft.Json.

6. LINQ / Collections

What is IEnumerable?

`IEnumerable` is an interface in .NET that represents a collection of objects that can be iterated one item at a time. It supports forward-only iteration using `foreach`. LINQ queries executed on `IEnumerable` are processed in memory, meaning the data is first fetched and then filtering or transformations are applied. It is suitable for working with in-memory collections such as lists or arrays.

🔎 Key Points

- Works with in-memory collections
- Executes query on client side
- Deferred execution supported
- Namespace: `System.Collections`

Example

```
IEnumerable<int> numbers = new List<int> { 1, 2, 3, 4, 5 };
```

```
var result = numbers.Where(x => x > 2);

foreach (var num in result)
{
    Console.WriteLine(num);
}
```

Filtering happens in memory.

What is IQueryable?

`IQueryable` is an interface that extends `IEnumerable` and is used for querying data from external data sources like databases. Unlike `IEnumerable`, LINQ queries on `IQueryable` are translated into SQL queries and executed on the server side. It improves performance when working with large datasets because filtering and projections happen at the database level rather than in memory.

🔎 Key Points

- Used for database queries (Entity Framework)
- Executes query on server side
- Translates LINQ to SQL
- Namespace: `System.Linq`

Example (Entity Framework)

```
IQueryable<User> users = _context.Users;

var result = users.Where(u => u.Age > 25);

var list = result.ToList();
```

👉 SQL generated:

```
SELECT * FROM Users WHERE Age > 25
```

Filtering happens in database.

What is the difference between IEnumerable and IQueryable?

The main difference between `IEnumerable` and `IQueryable` lies in where the query is executed. `IEnumerable` executes queries in memory after fetching data, while `IQueryable` executes queries at the data source level, such as in a database. `IQueryable` is more efficient for large datasets because it reduces data transfer by applying filters before retrieving data.

🔎 Comparison Table

Feature	IEnumerable	IQueryable
Execution Location	In Memory	Database Server
Performance	Slower for large data	Better for large data
Query Translation	No SQL translation	Translates to SQL
Namespace	System.Collections	System.Linq
Use Case	In-memory data	Database queries

When should we use IEnumerable?
We should use `IEnumerable` when working with in-memory collections such as lists, arrays, or data that has already been loaded from the database. It is suitable when the dataset is small and performance impact is minimal.

Example Scenarios:

- Filtering list data
- Processing in-memory objects
- Small datasets

When should we use IQueryable?
We should use `IQueryable` when querying data from a database or remote source, especially when working with large datasets. It ensures that filtering, sorting, and pagination are performed at the database level, improving performance and reducing memory usage.

Example Scenarios:

- Entity Framework queries
- Pagination (`Skip` / `Take`)
- Large dataset filtering

7. Angular

- **What Angular lifecycle hooks have you worked on?**
  In Angular applications, I have worked with several lifecycle hooks to manage

component initialization, change detection, and DOM interaction. The most commonly used hooks in my projects are `ngOnInit`, `ngOnChanges`, `ngOnDestroy`, `ngAfterViewInit`, and `ngAfterViewChecked`. I use `ngOnInit` for initialization logic such as API calls and data binding setup, `ngOnChanges` to respond to input property changes, `ngAfterViewInit` when interacting with child components or DOM elements via `@ViewChild`, and `ngOnDestroy` to clean up subscriptions to prevent memory leaks. In Angular 16+ and 17, I also leverage `takeUntilDestroyed` for automatic subscription cleanup.

Have you worked on ngOnInit?

Yes, I have extensively used `ngOnInit`. It is executed once after Angular initializes the component's input properties. I typically use it to perform component initialization tasks such as calling APIs, setting up default values, subscribing to services, and initializing forms. Since the constructor should only be used for dependency injection, business logic and initialization are best handled inside `ngOnInit`.

Example:

```
ngOnInit(): void {
  this.loadRegimenDetails();
  this.initializeForm();
}
```

Important Interview Point:

- Constructor → for DI only
- ngOnInit → for initialization logic

Have you worked on ngOnChanges?

Yes, I have used `ngOnChanges` when working with `@Input()` properties. This hook is triggered whenever the value of an input-bound property changes. It is particularly useful when a parent component dynamically passes data to a child component and we need to react to those changes.

Example:

```
@Input() therapyId!: number;

ngOnChanges(changes: SimpleChanges): void {
  if (changes['therapyId']) {
    this.loadTherapyData(changes['therapyId'].currentValue);
  }
}
```

Important Note:

`ngOnChanges` runs:

- Before `ngOnInit`
- Every time input changes

Have you worked on View lifecycle hooks?
Yes, I have worked with view lifecycle hooks such as `ngAfterViewInit` and `ngAfterViewChecked`. I use `ngAfterViewInit` when I need to access child components or DOM elements using `@ViewChild` or `@ViewChildren`. Since the view is fully initialized at this stage, it is safe to perform DOM-related operations. I avoid heavy logic in `ngAfterViewChecked` because it runs frequently during change detection cycles.

Example:

```
@ViewChild('inputField') inputElement!: ElementRef;

ngAfterViewInit(): void {
  this.inputElement.nativeElement.focus();
}

I commonly use ngOnInit for initialization, ngOnChanges for handling
dynamic input changes, ngAfterViewInit for DOM interactions, and
ngOnDestroy to clean up subscriptions. I follow best practices by
keeping constructors clean and managing subscriptions properly to
prevent memory leaks.
```

8. Data Structures

- **Have you worked on Hash Tables?**
  Yes, I have worked with hash tables in .NET, mainly using `Dictionary<TKey, TValue>` and `Hashtable`. I have used them for fast lookups, caching data, grouping records, and ensuring constant-time retrieval of values based on keys. In most modern applications, I prefer `Dictionary<TKey, TValue>` because it is type-safe and more efficient than the older `Hashtable`.

What is a Hash Table?

A hash table is a data structure that stores key-value pairs and uses a hash function to compute an index where the value is stored. This allows for fast insertion, deletion, and lookup operations, typically with O(1) average time complexity. Internally, it converts the key into a hash code and maps it to a specific bucket in memory.

Simple Explanation

Key → Hash Function → Index → Store Value

Example:

```
Dictionary<int, string> users = new Dictionary<int, string>();
users.Add(1, "Ashok");

var name = users[1];
```

Where do we use Hash Table?

We use hash tables when we need fast data retrieval based on unique keys. Common scenarios include caching data, removing duplicates, counting occurrences, implementing lookup tables, and indexing records for quick search operations.

Common Real-World Uses

- Caching user sessions
- Storing configuration values
- Frequency counting
- Implementing dictionaries/maps
- Checking existence quickly

- **Does Hash Table preserve sorting order?**
  No, a hash table does not preserve insertion or sorted order because data is stored based on the hash value of the key, not in sequential order. If sorted order is required, we should use `SortedDictionary` or `SortedList` instead.

9.Database / SQL Server

How can we perform performance tuning?

Performance tuning is the process of identifying and resolving performance bottlenecks in database queries or applications. It involves analyzing slow queries, checking execution plans, optimizing indexes, reducing unnecessary data retrieval, minimizing

joins, and ensuring efficient query design. The goal is to reduce query execution time, CPU usage, memory consumption, and IO operations.

What are the standard steps for performance tuning?
The standard steps include:

1. Identify slow queries using profiling tools (SQL Profiler / Query Store).
2. Analyze execution plan.
3. Check missing or unused indexes.
4. Reduce unnecessary columns (avoid SELECT *).
5. Optimize joins and filtering conditions.
6. Check statistics and update them.
7. Monitor IO, CPU, and memory usage.
8. Test improvements and compare performance.

How do you analyze execution plan?
To analyze an execution plan, I enable the actual execution plan in SQL Server Management Studio and examine operators like table scans, index scans, key lookups, joins, and sort operations. I focus on high-cost operators, missing index suggestions, estimated vs actual row counts, and logical reads. If I see table scans on large tables, it usually indicates missing indexes.

What to Look For:

- Table Scan (Bad for large tables)
- Index Scan vs Index Seek
- Key Lookup
- High cost operators
- Warnings (Missing Index, Hash Match spills)

How do you reduce query cost?
Query cost can be reduced by:

- Adding proper indexes
- Avoiding SELECT *
- Filtering early (WHERE clause)
- Reducing joins
- Avoiding functions on indexed columns
- Using EXISTS instead of IN for large datasets
- Proper indexing for JOIN columns
- Avoiding unnecessary sorting

- **How do indexes improve performance?**
  Indexes improve performance by allowing SQL Server to quickly locate rows without scanning the entire table. Instead of performing a full table scan, the database engine uses index seek operations to directly retrieve matching rows, significantly reducing IO operations and execution time.

What is the use of clustered index?
A clustered index determines the physical order of data in a table. Since data is physically sorted based on the clustered key, it improves performance for range queries and sorting operations. A table can have only one clustered index.

Best Used For:

- Primary Key
- Range queries (BETWEEN, >, <)
- Frequently sorted columns

What is the use of non-clustered index?
A non-clustered index stores a separate structure containing key values and a pointer to the actual table data. It improves performance for search queries and lookups without changing the physical order of the table. A table can have multiple non-clustered indexes.

Best Used For:

- WHERE clause columns
- JOIN columns
- Frequently searched fields

When should we use temp table?
Temp tables are used when handling large intermediate result sets, complex joins, or when data needs indexing for multiple operations within a session. They support indexes and statistics, making them suitable for large datasets.

Use Temp Table When:

- Large data processing
- Multiple joins
- Need indexing
- Reusing intermediate data

When should we use table variables?
Table variables are used for small datasets and lightweight temporary storage. They consume less logging but do not maintain statistics, so they are not suitable for large datasets.

Use Table Variable When:

- Small data sets
- Limited scope usage
- Simple operations

When should we use CTE?
CTE (Common Table Expression) is used to simplify complex queries, improve readability, and handle recursive queries. It is useful when you need temporary result sets without storing them physically like temp tables.

Use CTE When:

- Recursive queries
- Improving readability
- Breaking complex queries

How do you optimize joins?
optimize joins:

- Ensure indexes on join columns
- Join on indexed fields
- Use appropriate join type (INNER vs LEFT)
- Filter early in WHERE clause
- Avoid unnecessary joins
- Ensure matching data types

What are best practices for SQL optimization?
best practices are:

- Avoid SELECT *
- Use proper indexing strategy
- Keep transactions short
- Update statistics regularly

- Avoid functions in WHERE clause
- Use EXISTS instead of IN for large datasets
- Normalize properly but avoid over-normalization
- Monitor execution plans
- Use pagination (OFFSET / FETCH)
- Avoid unnecessary cursors

10. Microservices / Architecture

◈ How are your APIs consumed by third parties?

Our APIs are consumed by third parties using secure RESTful endpoints over HTTPS. We expose versioned APIs following REST standards, and authentication is handled using JWT or OAuth 2.0 depending on the integration type. Third parties consume the APIs through API keys, bearer tokens, or Azure AD authentication. We provide API documentation using Swagger/OpenAPI to ensure clear contract definitions. For external integrations, we enforce rate limiting, logging, and proper error handling to maintain security and reliability.

Additional Points (Impressive in Interview)

- API versioning (`v1`, `v2`)
- Swagger documentation
- HTTPS enforced
- Rate limiting
- Logging and monitoring (App Insights)

◈ How do you handle API acknowledgments?

API acknowledgments are handled using standard HTTP status codes and structured response objects. For synchronous APIs, we return appropriate status codes like 200 (Success), 201 (Created), 400 (Bad Request), 401 (Unauthorized), 404 (Not Found), and 500 (Internal Server Error), along with a consistent response structure containing success flags and messages. For asynchronous processing, we may return 202 (Accepted) and

process the request in the background using queues or Azure Service Bus, and optionally provide a tracking ID for status polling.

Example Response Structure

```
{
  "success": true,
  "message": "Record created successfully",
  "data": { }
}
```

◈ 3 How do different systems communicate in your architecture?

In our architecture, systems communicate through REST APIs for synchronous communication and message brokers like Azure Service Bus for asynchronous communication. For real-time updates, we may use SignalR. Microservices communicate securely via internal APIs or messaging queues. We also use database-level integration carefully when required, but we prefer decoupled communication through events and queues to improve scalability and fault tolerance.

Communication Types

- REST APIs → synchronous
- Azure Service Bus → asynchronous
- Event-driven architecture → decoupled communication
- Webhooks → third-party notifications

◈ 4What Azure services have you used in microservices?

In microservices architecture, I have worked with Azure App Services for hosting APIs, Azure Service Bus for asynchronous communication, Azure SQL Database for persistence, Azure Redis Cache for caching, Azure Key Vault for secure secret management, Azure Application Insights for monitoring and logging, Azure Storage (Blob) for file storage, and Azure DevOps for CI/CD pipelines. For containerized environments, I have worked with Azure Kubernetes Service (AKS) and Docker.

Common Azure Services in Microservices

- Azure App Service
- Azure Service Bus
- Azure SQL
- Azure Redis Cache

- Azure Key Vault
- Azure Application Insights
- AKS / Docker

◈ 5 How do you integrate third-party APIs?

To integrate third-party APIs, I use HttpClient or RestSharp in .NET to make secure HTTP calls. I handle authentication using OAuth 2.0, API keys, or JWT depending on the provider. I implement proper exception handling, timeout policies, and retry mechanisms using Polly to handle transient failures. I also log request and response data carefully while masking sensitive information. For scalability and reliability, external API calls may be wrapped inside background services or message queues when appropriate.

Example (HttpClient)

```
var response = await
_httpClient.GetAsync("https://api.thirdparty.com/data");

if (response.IsSuccessStatusCode)
{
    var content = await response.Content.ReadAsStringAsync();
}
```

Strong Senior-Level Summary

In our architecture, APIs are securely exposed via REST over HTTPS with proper authentication and versioning. Systems communicate using both synchronous REST calls and asynchronous messaging via Azure Service Bus. Third-party integrations are handled using HttpClient with proper authentication, retry policies, logging, and error handling to ensure reliability and resilience.

If you want, I can now explain:

- Circuit breaker pattern (very common microservices question)
- API Gateway concept
- How to secure APIs in production
- Webhook vs polling difference
- How to design scalable microservices

Can you tell me a difference between a (hash) set and a list?

The main difference between a `HashSet` and a `List` is how they store and retrieve data. A `List` maintains insertion order and allows duplicate elements, whereas a `HashSet` stores only unique elements and does not guarantee order. In terms of performance, a `List` provides O(n) lookup time because it performs a linear search, while a `HashSet` provides O(1) average lookup time due to its hash-based internal implementation.

Can you specifically, in C#, explain as a concept what is a Hashtable and what is a List?

In C#, a `List<T>` is a dynamic array that stores elements sequentially in memory. It allows duplicates and preserves insertion order. It is indexed and provides fast access by index but slower search performance.

A `Hashtable` is a key-value collection that uses a hash function to compute an index for storing values. It allows fast lookup based on keys rather than position. Unlike `List<T>`, it does not store items sequentially.

Modern C# prefers `Dictionary<TKey, TValue>` over `Hashtable` because it is type-safe.

3 With regards to storing an object in a Hashtable versus storing an object in a List — what happens under the hood?

✅ List (Under the Hood)

- Stored as a dynamic array.
- Items placed sequentially in memory.
- If capacity exceeded → new larger array created and elements copied.
- Searching → linear scan (O(n)).

✅ Hashtable (Under the Hood)

- Key's `GetHashCode()` is called.
- Hash converted into bucket index.
- Entry stored in bucket.
- If collision → stored in chain within same bucket.
- Lookup jumps directly to bucket (O(1) average).

If you are storing a key-value in a Hashtable, how does it happen and what is the advantage of putting it that way?

✅ What Happens:

1. Key is provided.
2. `GetHashCode()` called.
3. Hash compressed into valid bucket index.
4. Key-value pair stored in that bucket.
5. If collision → chained.

✅ Advantage:

- Fast retrieval.
- No need to scan entire collection.
- Efficient for large datasets.

Time complexity:

- Average: O(1)
- Worst case: O(n) (if many collisions)

Suppose you have an Employee object and you put it into a Hashtable with EmployeeId as the key — what happens under the hood? (Explain the algorithm.)

Example:

```
table.Add(101, employeeObject);
```

Algorithm:

1. Call `GetHashCode()` on key (101).
2. Compute bucket index:
3. `index = hash % bucketSize`
4. Check if bucket empty:
    - If yes → insert entry.
    - If no → compare keys.
5. If same key → replace.
6. If collision → add to chain.

Retrieval:

```
var emp = table[101];
```

1. Hash again.

2. Go directly to bucket.
3. Compare keys.
4. Return value.

Do you understand application modernization?

Yes, I understand application modernization as the process of upgrading legacy applications to improve scalability, maintainability, performance, and security by adopting modern technologies and architectural patterns. This may involve refactoring monolithic systems into microservices, moving from on-premise infrastructure to cloud environments, adopting containerization, implementing CI/CD pipelines, and improving DevOps practices. The goal is to enhance agility, reduce technical debt, and align the application with current business and compliance requirements.

Have you worked on rewriting legacy applications?

Yes, I have worked on refactoring and partially rewriting legacy modules. In modernization projects, we typically evaluate whether to refactor, re-architect, or rebuild components. I have been involved in migrating tightly coupled legacy logic into layered architecture, improving code quality by applying SOLID principles, replacing outdated frameworks with .NET Core, and implementing RESTful APIs in place of older service mechanisms. The focus was to reduce technical debt and improve maintainability without disrupting business continuity.

◈ Have you worked on migrating legacy platforms to modern platforms?

Yes, I have experience migrating applications from older .NET Framework versions to ASP.NET Core, moving on-premise deployments to Azure App Services, and transitioning from monolithic deployments to modular or service-oriented architectures. This included updating authentication mechanisms (e.g., from basic authentication to JWT/OAuth), improving logging with structured logging frameworks, and implementing CI/CD pipelines in Azure DevOps to support automated deployments.

◈Do you have experience supporting modernization and optimization approaches?

Yes, I have supported modernization efforts through code refactoring, performance tuning, implementing caching strategies (e.g., Redis), optimizing database queries,

introducing dependency injection where missing, modularizing tightly coupled components, and introducing monitoring tools such as Application Insights. I also contributed to improving system scalability by introducing asynchronous processing and message-based communication where appropriate.

◈ Do you understand hybrid cloud architecture?

Yes, hybrid cloud architecture refers to an environment where applications or infrastructure span across on-premise systems and cloud platforms such as Azure. This model allows organizations to retain sensitive workloads or legacy systems on-premise while leveraging cloud scalability and services for newer components. In healthcare, hybrid models are common due to compliance and data residency requirements. Secure connectivity is typically established using VPN or ExpressRoute, and identity management is handled via Azure AD or federated identity systems.

Below are **interview-style answers** for each question. Each answer includes:

- **Clear explanation**
- **Real-world analogy**
- **How you might say it confidently in an interview**

---

1. How did you handle encryption?

Interview Answer

In our application, encryption was used to protect sensitive data both **in transit and at rest**. For data in transit, we used **HTTPS with TLS encryption**, ensuring that all communication between the client and server was encrypted. For sensitive stored data such as tokens or credentials, we relied on **secure storage mechanisms and hashing algorithms like SHA-256 or encryption using AES** where necessary.

Encryption ensures that even if data is intercepted during transmission or accessed without authorization, it cannot be read without the correct decryption key.

Analogy

Encryption is like **sending a letter inside a locked box**. Even if someone intercepts the box during delivery, they cannot read the letter unless they have the key to unlock it.

---

2. Did you use HTTPS?

Interview Answer

Yes, all communication between the client and server was done over **HTTPS**. HTTPS uses **TLS (Transport Layer Security)** to encrypt the communication channel, preventing attacks such as **man-in-the-middle attacks, packet sniffing, or data tampering**.

In our .NET applications hosted on Azure or IIS, HTTPS was enforced by installing a valid SSL certificate and configuring the server to **redirect all HTTP requests to HTTPS automatically**.

Analogy

Using HTTP is like **sending postcards through the mail where anyone can read them**. HTTPS is like **sending sealed envelopes**, ensuring that only the intended recipient can read the contents.

---

3. How did you implement TLS?

Interview Answer

TLS was implemented by configuring **SSL certificates on the web server or Azure App Service**. Once the certificate is installed, the server establishes a **TLS handshake** with the client whenever a secure connection is requested.

During this handshake, the server shares its **public certificate**, and both client and server agree on a symmetric session key that encrypts the communication for that session.

In our architecture, TLS was handled primarily at the **infrastructure level**, such as Azure Application Gateway, Load Balancer, or App Service, which automatically manages the encryption layer.

Analogy

TLS is like **two people agreeing on a secret language before starting a conversation**, so even if someone overhears them, they cannot understand what is being said.

---

4. How did you implement Azure AD authentication?

Interview Answer

Azure AD authentication was implemented using **OAuth 2.0 and OpenID Connect protocols**. The application redirects the user to **Azure Active Directory for login**, where Azure validates the user's credentials.

Once authentication is successful, Azure AD issues a **JWT token** that contains user identity and claims. The application then validates this token and uses the claims to authorize access to resources.

In .NET applications, this is commonly configured using **Microsoft.Identity.Web or middleware like AddAuthentication() with Azure AD settings in appsettings.json**.

Analogy

Azure AD authentication is like **entering a secure office building using a central security desk**. Instead of every room verifying your identity separately, the security desk verifies you once and gives you a badge that grants access to different areas.

---

5. Did you implement message-level security?

Interview Answer

Yes, message-level security can be implemented when the payload itself needs protection beyond the transport layer. While TLS encrypts the communication channel, message-level security encrypts the **actual message content**.

In some APIs or microservices architectures, this can involve **encrypting payload data or signing messages using cryptographic signatures** to ensure integrity and authenticity even if the message passes through multiple services.

However, in many modern REST APIs, **transport-level security (TLS) combined with token-based authentication is usually sufficient**.

Analogy

TLS security is like **locking the truck that carries packages**, while message-level security is like **locking each individual package inside the truck**.

---

6. Did you implement JWT token validation?

Interview Answer

Yes, JWT token validation was implemented in the API middleware. Every request to protected endpoints required a **valid JWT token in the Authorization header using the Bearer scheme**.

The API validates several aspects of the token, including:

- Signature validation
- Token expiration
- Issuer verification
- Audience validation

In .NET, this is usually implemented using the **JWT Bearer authentication middleware**, which automatically validates tokens against the configured identity provider.

Analogy

JWT token validation is like **checking a visitor badge before allowing someone into a secure office area**. If the badge is expired or issued by the wrong authority, access is denied.

---

7. How do you verify tokens using public/private keys?

Interview Answer

JWT tokens are typically signed using **asymmetric cryptography**, where the identity provider signs the token using a **private key**, and the application verifies the signature using the **public key**.

When the API receives a JWT token, it retrieves the **public key from the identity provider's metadata endpoint** (such as Azure AD's OpenID configuration). The API then uses this public key to verify that the token was issued by the trusted authority and that it has not been tampered with.

Since only the identity provider possesses the private key, the signature cannot be forged.

Analogy

This is similar to **a bank issuing signed checks**. The bank signs the check with its private stamp, and anyone can verify its authenticity using the bank's publicly known stamp pattern, but no one else can create that stamp.

✅ **Interview Tip**

When answering these questions in interviews, try to follow this structure:

1. **Concept**
2. **How you implemented it**
3. **Why it is important**
4. **Analogy**

This makes the answer sound **clear, practical, and experienced**.

If you'd like, I can also give you the **top 10 security interview questions for .NET developers** (very commonly asked in senior interviews). 🔐

Below are **clear interview-style answers with explanation and analogy** for the **.NET Core Async/Await section**. You can speak these **almost exactly in interviews**.

---

1. Can you explain your understanding of async/await?

Interview Answer

Async and Await in .NET are used to perform **asynchronous operations without blocking the executing thread**. When a method is marked as `async`, it allows the use of `await` inside it. The `await` keyword pauses the execution of the method until the awaited task completes, but it **does not block the thread**. Instead, the thread is returned to the thread pool and can perform other work while waiting for the operation to finish.

This is particularly useful for operations such as **API calls, database queries, or file access**, which take time to complete but do not require CPU processing during the wait.

Analogy

It is like **ordering food at a restaurant**. After placing the order, you don't stand in the kitchen waiting. You sit at your table and do other things while the food is prepared. When the food is ready, you continue.

---

2. What is the difference between I/O-bound and CPU-bound operations?

Interview Answer

I/O-bound operations are tasks that spend most of their time **waiting for external resources**, such as network calls, database queries, file reading, or API calls. These operations benefit greatly from asynchronous programming because the thread can be released while waiting.

CPU-bound operations, on the other hand, require **heavy computation by the CPU**, such as image processing, mathematical calculations, or data processing algorithms. These tasks use the CPU continuously and are better handled using **parallel processing or background threads** rather than async/await.

Analogy

I/O-bound work is like **waiting for a download to finish**, while CPU-bound work is like **compressing a large video file**, where the processor is actively working the entire time.

---

3. Why do we need async/await?

Interview Answer

We need async/await to improve **application scalability and responsiveness**. Without asynchronous programming, threads remain blocked while waiting for operations like database queries or API calls to complete.

Async/await allows the application to **free the thread during waiting periods**, enabling it to serve other requests. This is especially important in **web applications where thousands of users might be making requests simultaneously**.

It helps prevent thread starvation and improves system throughput.

Analogy

Imagine a **single cashier in a store** who must wait while each customer's credit card is processed. If the cashier could step aside and help another customer while waiting for the card approval, the store would serve more customers efficiently.

---

4. If I call an API without using await, will the thread still wait?

Interview Answer

No, if you call an asynchronous API without using `await`, the method will immediately return a `Task` and the execution will continue to the next line of code. The thread will **not wait for the operation to complete** unless you explicitly block it using methods like `.Result` or `.Wait()`.

However, not awaiting a task can lead to issues such as **unhandled exceptions or incomplete operations**, so it should generally be avoided unless you intentionally want the operation to run in the background.

Analogy

It is like **sending a courier package and immediately continuing your work without waiting for delivery confirmation**.

---

5. Why do we need await if the thread is already waiting?

Interview Answer

Actually, when we use `await`, the thread **does not wait**. Instead, the thread is **released back to the thread pool** while the asynchronous operation continues.

Once the operation completes, the runtime schedules the continuation of the method. This allows the application to use threads more efficiently.

Without `await`, the code would either continue prematurely or require blocking calls like `.Wait()`, which defeats the purpose of asynchronous programming.

Analogy

Await is like **leaving your phone number at a restaurant instead of standing in line**. When your table is ready, they call you back.

---

6. What happens internally when await is encountered?

Interview Answer

When the compiler encounters `await`, it transforms the async method into a **state machine**. The method execution is paused at the await point, and the remaining code is stored as a continuation.

The current thread is released, and once the awaited task completes, the continuation resumes execution from where it left off.

This transformation is handled automatically by the **C# compiler**, which generates the required state machine behind the scenes.

Analogy

Think of it like **saving the progress of a video game**. When you pause the game, your current state is stored, and when you resume, the game continues exactly where it stopped.

---

7. Does async create a new thread?

Interview Answer

No, `async` does not automatically create a new thread. Async/await is primarily about **non-blocking execution**, not multi-threading.

The asynchronous operation may use existing threads from the **thread pool**, but the async keyword itself simply allows the method to run asynchronously without blocking the current thread.

If we need explicit parallel processing, we would use constructs like **Task.Run or Parallel programming**.

Analogy

Async is like **pausing a task and letting someone else use the workspace temporarily**, rather than creating a completely new workspace.

---

8. What is the state machine in async?

Interview Answer

When an async method is compiled, the C# compiler converts it into a **state machine structure**. This state machine tracks the execution state of the method.

Each `await` represents a possible pause point. The state machine keeps track of which step the method is currently on and resumes execution when the awaited task completes.

This mechanism allows asynchronous methods to **pause and resume execution seamlessly without blocking threads**.

Analogy

It is similar to a **multi-step workflow where the system remembers which step was completed and continues from the next step once the required condition is met**.

---

9. What is context switching?

Interview Answer

Context switching refers to the process where the CPU switches from executing one thread to another. It involves saving the current thread's state and loading another thread's state so that execution can continue.

In asynchronous programming, minimizing unnecessary context switching improves performance because switching threads frequently can introduce overhead.

Async/await helps reduce blocking and improves thread utilization.

Analogy

It is like a **chef working on multiple dishes in a kitchen**, switching between them as needed while remembering exactly where each dish left off.

---

10. What happens if two operations are independent — how do you handle them asynchronously?

Interview Answer

If two operations are independent, they can be executed **concurrently using Task.WhenAll()**. Instead of awaiting each operation sequentially, both tasks are started first, and then we await their completion together.

This allows both operations to run in parallel, reducing overall execution time.

This approach improves efficiency when the operations do not depend on each other's results.

Analogy

It is like **ordering two dishes at a restaurant at the same time instead of waiting for one dish to finish cooking before ordering the next**.

✅ **Interview Tip**

Interviewers love this **follow-up concept**:

Sequential Execution

```
await Task1();
await Task2();
```

Total time = **Task1 + Task2**

Parallel Execution

```
var t1 = Task1();
var t2 = Task2();
await Task.WhenAll(t1, t2);
```

Total time = **max(Task1, Task2)**

If you'd like, I can also show you the **5 hardest async/await interview questions that senior .NET developers often fail** (very common in Microsoft-style interviews).

Below are **interview-style answers with explanation and analogy** for the **Dependency Injection (.NET Core)** section.

---

1. What are interfaces?

Interview Answer

An interface in C# is a **contract that defines what methods or properties a class must implement without defining how they are implemented**. It specifies the behavior that a class should follow while allowing different classes to provide their own implementations.

Interfaces promote **loose coupling** between components. Instead of depending on a concrete class, the system depends on an abstraction (interface). This makes the code easier to maintain, extend, and test.

For example, a logging system may define an `ILogger` interface, and different classes such as `FileLogger`, `DatabaseLogger`, or `ConsoleLogger` can implement that interface.

Analogy

An interface is like a **remote control**. The remote defines the buttons (volume, power, channel), but the TV manufacturer decides how those buttons actually work internally.

---

2. What are the advantages of Dependency Injection?

Interview Answer

Dependency Injection (DI) is a design pattern where the dependencies required by a class are **provided from the outside rather than being created inside the class**.

The major advantages include:

• **Loose coupling** between components
• **Improved testability** (mock dependencies during unit testing)
• **Better maintainability** and cleaner architecture
• **Easier replacement of implementations**
• **Centralized dependency management**

By injecting dependencies rather than instantiating them directly, the application becomes more flexible and easier to extend.

Analogy

Dependency Injection is like **a car factory assembling a car using parts supplied by different vendors instead of building every component inside the factory**. This allows parts to be replaced easily without redesigning the entire car.

---

3. How does DI help in logging scenarios?

Interview Answer

Dependency Injection helps in logging by allowing applications to depend on **logging abstractions rather than specific logging implementations**.

In .NET Core, logging is typically handled using the `ILogger<T>` interface. The DI container injects the appropriate logging implementation automatically.

This approach allows developers to easily switch logging providers such as:

- Console logging
- File logging
- Azure Application Insights
- Serilog or NLog

Without changing the application code.

This improves flexibility and makes logging configurable across environments.

Analogy

It is like **plugging different microphones into the same sound system**. The system doesn't care which microphone you use as long as it follows the required interface.

---

4. How do you register and inject implementations?

Interview Answer

In .NET Core, services are registered in the **dependency injection container**, usually in `Program.cs` or `Startup.cs`.

Services are typically registered using three lifetimes:

- **AddSingleton** – one instance for the entire application
- **AddScoped** – one instance per request
- **AddTransient** – a new instance each time it is requested

Once registered, the framework automatically injects the dependency through **constructor injection** when the class is created.

This makes dependency management centralized and automatic.

Analogy

It is similar to **a hotel reception desk assigning rooms**. When a guest arrives, the system already knows which room type they should get and provides it automatically.

---

5. What is an IoC container?

Interview Answer

An IoC (Inversion of Control) container is a **framework component responsible for creating and managing object dependencies automatically**.

Instead of classes creating their own dependencies, the IoC container provides them. This inverts the control of object creation from the class to the container.

In .NET Core, the built-in IoC container manages services registered in the application and automatically injects them where needed.

It also handles object lifetimes and dependency resolution.

Analogy

An IoC container is like **a restaurant kitchen manager**. Instead of chefs fetching their own ingredients, the manager organizes and supplies everything needed so chefs can focus on cooking.

✅ Quick Interview Summary

Concept	Key Idea
Interface	Defines contract, not implementation
Dependency Injection	Dependencies provided externally
DI Benefits	Loose coupling, testability, maintainability
Logging with DI	Inject `ILogger` implementations
IoC Container	Framework that manages dependencies

If you'd like, I can also explain the **3 most commonly asked advanced DI interview questions** (very common in senior .NET interviews):

• Difference between **Singleton, Scoped, and Transient**
• **Constructor vs Method vs Property Injection**
• **Service lifetime issues in ASP.NET Core**.

Below are **interview-style answers with explanation + analogy** for the **Design Patterns section**. These are written so you can **speak them naturally in interviews**.

---

1. What design patterns have you used?

Interview Answer

In my projects, especially while working with **.NET and EHR systems**, I have used several design patterns to improve maintainability, scalability, and separation of concerns. Some common patterns I have used include:

- **Repository Pattern** – to abstract database access and separate data logic from business logic.
- **Dependency Injection Pattern** – to manage dependencies and achieve loose coupling between services.
- **Factory Pattern** – to create objects dynamically without exposing the creation logic.
- **Singleton Pattern** – for services that require a single shared instance across the application, such as configuration services.
- **Strategy Pattern** – to implement different business rules or algorithms dynamically.

Using these patterns helps make the code **cleaner, easier to test, and easier to extend**.

Analogy

Design patterns are like **architectural blueprints**. Instead of designing a building from scratch every time, architects use proven structures that are known to work efficiently.

---

2. Explain Repository Pattern.

Interview Answer

The Repository Pattern is used to **separate data access logic from the business logic** of the application. It acts as a middle layer between the application and the database.

Instead of business services directly interacting with Entity Framework or SQL queries, they communicate with a repository interface. The repository then handles all database operations such as **CRUD operations, queries, and data mapping**.

This abstraction allows the application to remain independent of the underlying data access technology.

For example, if we switch from **SQL Server to another database**, we only need to modify the repository implementation rather than changing the entire application.

Analogy

A repository is like **a library system**. Instead of readers going directly into the storage room to find books, they ask the librarian. The librarian retrieves the books and handles all the storage details.

---

3. Why use abstraction in repository?

Interview Answer

Abstraction in the repository pattern allows the application to depend on **interfaces rather than concrete implementations**. This helps achieve **loose coupling** between the business logic and data access layer.

By using abstraction:

- The business layer doesn't need to know how data is stored or retrieved.
- It becomes easier to **replace the data source** (for example switching from SQL to an API).
- Unit testing becomes easier because we can **mock repository interfaces**.

This makes the system more flexible, maintainable, and testable.

Analogy

Abstraction is like **using a payment terminal in a store**. The cashier only interacts with the terminal interface, while the internal payment processing system handles the complex operations behind the scenes.

---

4. How do you prevent exposing implementation details?

Interview Answer

Implementation details can be hidden by following **encapsulation and abstraction principles**.

In practice, this is achieved by:

- Exposing only **interfaces** to the application layers
- Keeping implementation classes **internal or private**
- Using **service layers or repositories** to handle business and data operations
- Avoiding direct access to database or internal logic from controllers or UI layers

This approach ensures that higher layers of the application only interact with **well-defined contracts**, preventing tight coupling with internal implementations.

Analogy

It is like **driving a car**. The driver interacts with the steering wheel, pedals, and dashboard but does not see or interact with the internal engine mechanics.

✅ Quick Interview Tip

Interviewers often follow this section with a deeper question like:

"What is the difference between Repository Pattern and Service Layer Pattern?"

A strong short answer is:

- **Repository → Handles data access**
- **Service Layer → Handles business logic**

If you'd like, I can also show you the **7 design patterns that senior .NET developers are most often asked in interviews (with real examples)**.

Below are **advanced interview-style answers for Singleton Pattern**, with **clear explanations and analogies** so you can explain confidently in interviews.

---

1. How would you implement Singleton from scratch?

Interview Answer

The Singleton pattern ensures that a class has **only one instance throughout the application's lifecycle** and provides a global access point to that instance.

To implement a Singleton from scratch, we typically:

1. Make the **constructor private** so no external class can create an instance.
2. Create a **static variable** to hold the single instance.
3. Provide a **public static method or property** that returns the instance.
4. Ensure **thread safety** if the application is multi-threaded.

The instance is created only once and reused everywhere in the application.

Analogy

Singleton is like **a country having only one president at a time**. Everyone refers to the same president rather than creating multiple presidents.

---

2. What are the important design considerations?

Interview Answer

When implementing Singleton, several design considerations are important:

- **Thread safety** to prevent multiple instances in concurrent environments
- **Lazy initialization** so the instance is created only when needed
- **Performance optimization** to avoid unnecessary locking
- **Preventing reflection or serialization attacks** that may create additional instances
- **Ensuring global accessibility while maintaining encapsulation**

These considerations ensure that the Singleton remains reliable and efficient.

Analogy

It is like designing a **bank vault system** where only one vault exists and strict rules ensure no duplicate vaults are accidentally created.

---

3. Why should the constructor be private?

Interview Answer

The constructor is made private to **prevent external classes from creating new instances** of the Singleton class.

If the constructor were public, any class could create multiple instances using the `new` keyword, which would break the Singleton pattern.

A private constructor ensures that the **only way to access the instance is through the controlled access method provided by the class**.

Analogy

It is like **a private club with a locked entrance**. The only way to enter is through an authorized gate controlled by the club.

---

4. Why make the class sealed?

Interview Answer

A Singleton class is often marked as **sealed** to prevent inheritance.

If another class could inherit from the Singleton class, it could create additional instances or override behavior, potentially breaking the Singleton guarantee of a single instance.

Sealing the class ensures that the Singleton design remains intact.

Analogy

It is like **protecting a master key**. If copies of the key could be made freely, the security system would fail.

---

5. How do you handle multithreading?

Interview Answer

In a multi-threaded environment, multiple threads may attempt to create an instance of the Singleton simultaneously. To handle this, we use **thread-safe techniques** such as:

- **Locking mechanisms**
- **Double-check locking**
- **Static initialization**
- **Lazy initialization in .NET**

These methods ensure that only one thread creates the instance while others reuse it.

Analogy

Imagine **multiple people trying to enter a meeting room simultaneously**, but a security guard ensures only one person sets up the meeting.

---

6. Why use locking?

Interview Answer

Locking is used to ensure that **only one thread can execute the instance creation code at a time**.

Without locking, two threads might simultaneously detect that the instance is null and both create new instances, violating the Singleton principle.

The `lock` statement ensures mutual exclusion when creating the object.

Analogy

Locking is like **a single-person entry gate**. Only one person can pass through at a time.

---

7. What is a mutex?

Interview Answer

A mutex (mutual exclusion object) is a synchronization mechanism that ensures **only one thread can access a shared resource at a time**, even across different processes.

While `lock` works within a single application, a mutex can be used for **cross-process synchronization**.

Mutexes are heavier than locks and are usually used when synchronization must occur across multiple applications.

Analogy

A mutex is like **a key to a shared restroom**. Only the person holding the key can enter until they release it.

---

8. What happens when multiple threads access at the same time?

Interview Answer

If multiple threads access the Singleton creation code simultaneously without proper synchronization, they may all detect that the instance is null and create multiple instances.

This results in **race conditions**, which violate the Singleton guarantee.

Proper synchronization ensures that **only one instance is created regardless of how many threads attempt access simultaneously**.

Analogy

It is like **multiple people trying to book the same hotel room at the same time**. Without proper control, the system might allocate the same room to multiple people.

---

9. Every thread has to go through lock — how can we improve it?

Interview Answer

If every thread has to acquire a lock, it can cause **performance overhead**.

To improve performance, we use **Double-Check Locking**. In this approach:

1. First check if the instance already exists without locking.
2. Only if it is null do we enter the lock.
3. Inside the lock, check again before creating the instance.

This reduces unnecessary locking once the instance has already been created.

Analogy

It is like **checking if the door is already open before using the key**. If it is open, you just walk in without unlocking it.

---

10. Can there be race conditions in your implementation?

Interview Answer

Yes, race conditions can occur if the Singleton implementation is not properly synchronized.

For example, if two threads simultaneously check whether the instance is null, they might both create separate instances before either assigns the value.

To avoid race conditions, we must use **proper synchronization techniques such as locking, Lazy initialization, or static initialization**.

Analogy

It is like **two employees simultaneously printing the same report**, both thinking it has not yet been printed.

---

11. What happens if two threads check null simultaneously?

Interview Answer

If two threads check the instance variable at the same time and both see that it is null, they may both proceed to create new instances.

This leads to **multiple instances being created**, which breaks the Singleton design.

To prevent this, synchronization mechanisms such as **locking or Lazy initialization** must be used.

Analogy

It is like **two chefs seeing an empty pot and both starting to cook the same dish**, resulting in duplicate meals.

---

12. How do you avoid double instantiation?

Interview Answer

Double instantiation can be avoided using techniques such as:

- **Locking**
- **Double-check locking**
- **Static initialization**
- **Lazy initialization**

In modern .NET applications, the recommended approach is to use **Lazy**, which ensures thread-safe lazy initialization without explicit locking.

This guarantees that the instance is created **only once even in highly concurrent environments**.

Analogy

It is like **a factory machine that automatically prevents producing duplicate serial numbers**.

✅ **Interview Tip (Important)**

If the interviewer asks **"What is the best Singleton implementation in .NET?"**

A strong answer is:

The recommended approach in modern .NET is using **Lazy** because it provides **built-in thread safety, lazy initialization, and better performance without explicit locks.**

If you'd like, I can also show you **the 4 most common mistakes developers make when implementing Singleton in interviews** (these are frequently used as trick questions).

Below are **clear interview-style answers with explanation and analogy** for both sections.

---

7 IEnumerable / Custom Collections

1. Have you implemented your own IEnumerable class?

Interview Answer

Yes, it is possible to implement a custom collection by implementing the `IEnumerable` interface. When we implement `IEnumerable`, we allow our custom class to be **iterated using a `foreach` loop**. This requires implementing the `GetEnumerator()` method which returns an `IEnumerator`.

By doing this, we can define **how the elements of our custom collection should be traversed**.

This is useful when we create custom data structures or when we want to control the order or logic of iteration.

Analogy

It is like creating **a playlist in a music player**. The playlist defines how songs are stored, and the iterator defines how they are played one after another.

---

2. Have you derived a class from IEnumerable?

Interview Answer

Yes, when creating custom collections, a class can implement the `IEnumerable` interface. This allows the class to be treated as a **collection that can be iterated through using `foreach` or LINQ operations**.

When deriving from `IEnumerable`, we must implement the `GetEnumerator()` method which returns an enumerator that iterates through the collection.

This makes the custom collection compatible with **.NET collection handling and LINQ queries**.

Analogy

It is like designing a **book shelf system** where readers can go through each book sequentially without knowing how the books are internally arranged.

---

3. What does IEnumerator do?

Interview Answer

`IEnumerator` is responsible for **iterating through a collection one element at a time**.

It provides methods and properties such as:

- **MoveNext()** → moves to the next element
- **Current** → retrieves the current element
- **Reset()** → resets the enumerator to the beginning

The enumerator controls the **actual iteration logic**, while `IEnumerable` simply provides access to the enumerator.

Analogy

`IEnumerator` is like **a cursor in a document**. It points to the current item and moves step by step through the content.

---

4. Have you extended IEnumerable manually?

Interview Answer

Yes, `IEnumerable` can be extended manually by implementing the interface in a custom class and defining the iteration logic inside the `GetEnumerator()` method.

In many modern C# implementations, this can also be simplified using the **`yield return` keyword**, which automatically generates the enumerator logic.

This makes the code simpler while still providing full iteration capability.

Analogy

It is like **designing a guided museum tour** where you decide the order in which visitors see each exhibit.

---

8 Entity Framework

---

1. Are you using Entity Framework?

Interview Answer

Yes, Entity Framework is commonly used as an **Object Relational Mapper (ORM)** in .NET applications. It allows developers to interact with the database using **C# objects instead of writing raw SQL queries**.

Entity Framework handles tasks such as:

- Mapping database tables to C# classes
- Managing relationships between entities
- Generating SQL queries automatically
- Tracking changes and persisting them to the database

This improves productivity and reduces boilerplate database code.

Analogy

Entity Framework is like **a translator between the application and the database**, converting object-oriented code into SQL commands.

---

2. Are you using Code First or Database First?

Interview Answer

Both approaches exist in Entity Framework, but in many enterprise projects, **Database First is commonly used**, especially when working with existing databases.

In Database First, the database schema already exists, and Entity Framework generates the entity models and context classes based on the database structure.

This approach is useful when working with **large legacy systems or enterprise databases where the schema is already defined**.

Analogy

Database First is like **constructing a building based on an existing architectural blueprint**.

---

3. Why Database First approach?

Interview Answer

The Database First approach is often used when the database already exists or when the database structure is managed by a dedicated database team.

Some reasons for using Database First include:

- Working with **legacy databases**
- Ensuring **database schema consistency**
- Allowing DBAs to manage schema changes
- Generating entity models automatically from the database

This approach helps maintain alignment between the application and the existing database design.

Analogy

It is like **building furniture based on pre-existing measurements of a room** rather than designing the room around the furniture.

---

4. How do you map models in EF?

Interview Answer

In Entity Framework, models are mapped to database tables using either **conventions, data annotations, or Fluent API configuration**.

- **Conventions** automatically map class names to table names and properties to columns.
- **Data Annotations** use attributes in the model classes to define mappings.
- **Fluent API** provides advanced configuration through the `OnModelCreating` method in the DbContext.

Fluent API is often preferred for complex mappings because it keeps the entity classes clean and centralizes configuration.

Analogy

Mapping models in EF is like **creating a blueprint that tells how objects in the application correspond to tables and columns in the database**.

✅ Quick Interview Tip

Interviewers often ask this follow-up:

"What is the difference between IEnumerable, IQueryable, and List?"

A simple answer:

Type	Execution
IEnumerable	Executes query in memory
IQueryable	Executes query in database
List	Concrete collection stored in memory

If you'd like, I can also show you **the 10 most common Entity Framework interview questions senior developers struggle with** (very commonly asked in .NET interviews).

Below are **interview-style answers with explanation and analogy** for the **SQL Server / Transactions** and **C# Version Knowledge** sections.

---

9 SQL Server / Transactions

1. Have you used transactions in stored procedures?

Interview Answer

Yes, transactions are commonly used in stored procedures to ensure **data consistency and atomicity**. A transaction groups multiple database operations into a single unit of work so that **either all operations succeed or all fail together**.

For example, when processing financial records or updating multiple related tables, transactions ensure that partial updates do not occur if an error happens during execution.

If any step fails, the transaction is rolled back to maintain data integrity.

Analogy

A transaction is like **transferring money between two bank accounts**. The system must debit one account and credit another. If either step fails, the entire operation is cancelled to avoid inconsistencies.

---

2. Have you used isolation levels?

Interview Answer

Yes, isolation levels are used to control **how transactions interact with each other when accessing shared data**. They help manage issues like dirty reads, non-repeatable reads, and phantom reads.

By selecting the appropriate isolation level, we can balance **data consistency and system performance** depending on the application's requirements.

---

3. Which isolation levels have you used?

Interview Answer

The most commonly used isolation levels include:

• **Read Committed** – prevents dirty reads and is the default isolation level in SQL Server
• **Repeatable Read** – ensures rows read during a transaction cannot change until the transaction completes
• **Serializable** – provides the highest level of isolation by preventing new rows from appearing during a transaction
• **Read Uncommitted** – allows dirty reads but improves performance in some scenarios

In most enterprise applications, **Read Committed** is widely used because it balances consistency and performance.

Analogy

Isolation levels are like **traffic rules at an intersection**. Some rules allow more cars to move faster, while stricter rules ensure safety but slow down traffic.

---

4. Have you encountered deadlocks?

Interview Answer

Yes, deadlocks can occur in high-concurrency environments when two or more transactions **block each other while waiting for resources held by the other transaction**.

For example, one transaction may lock Table A and request Table B, while another transaction locks Table B and requests Table A. Neither transaction can proceed, causing a deadlock.

SQL Server automatically detects deadlocks and terminates one transaction (called the **deadlock victim**) to allow the other to continue.

Analogy

A deadlock is like **two cars entering a narrow bridge from opposite sides and blocking each other**, preventing either car from moving forward.

---

5. What causes deadlocks?

Interview Answer

Deadlocks are typically caused by:

• Accessing tables in **different orders** across transactions
• **Long-running transactions** holding locks for extended periods
• **High concurrency operations** updating the same resources
• Poorly optimized queries that lock many rows

Proper query design and consistent resource access order can significantly reduce deadlock risks.

---

6. How did you handle deadlocks?

Interview Answer

Deadlocks can be handled using several approaches:

• Ensuring **consistent table access order** across queries
• Keeping transactions **short and efficient**
• Creating appropriate **indexes** to reduce locking
• Implementing **retry logic in the application layer** if a deadlock occurs
• Using **row-level locking strategies**

In many systems, when a deadlock occurs, the application retries the transaction automatically.

Analogy

Handling deadlocks is like **setting traffic signals and rules so vehicles approach intersections in a predictable order**.

---

7. How did you monitor or notify deadlock situations?

Interview Answer

Deadlocks can be monitored using several SQL Server tools:

• **SQL Server Profiler**
• **Extended Events**
• **Deadlock graphs** in SQL Server Management Studio
• **System health extended event session**

Many teams configure **alerts or logging systems** to notify administrators when deadlocks occur so they can analyze the queries involved.

Analogy

Monitoring deadlocks is like **installing surveillance cameras at traffic intersections to detect and analyze traffic jams**.

---

10 C# Version Knowledge

1. You are using .NET 5 — what C# version are you using?

Interview Answer

.NET 5 typically uses **C# 9** as its default language version. C# 9 introduced several improvements focused on **immutability, cleaner syntax, and improved pattern matching**.

---

2. What are new features in C# 9?

Records

Interview Answer

Records are a new reference type introduced in C# 9 designed for **immutable data models**. They are commonly used for objects that represent data rather than behavior.

Records automatically provide features such as:

• Value-based equality
• Built-in immutability
• Simplified syntax for data models

This reduces boilerplate code when creating data objects.

Analogy

Records are like **official forms where information is recorded once and cannot be modified easily afterward**.

Improvements in C# 9

Some important improvements include:

- **Pattern matching enhancements**
- **Init-only properties**
- **Top-level statements**
- **Improved lambda expressions**

These features make the language more expressive and reduce boilerplate code.

Default Property Initialization

Interview Answer

C# allows properties to be initialized with default values directly during declaration. This ensures that objects start with a **valid state without requiring constructors to initialize every property**.

This improves code readability and reduces repetitive initialization logic.

Analogy

It is like **pre-configuring a device with default settings so it works immediately when turned on**.

Regex Improvements

Interview Answer

Newer C# and .NET versions introduced **performance improvements in regular expressions**, including better compilation and optimized pattern matching.

These improvements allow faster processing when working with large datasets or complex pattern searches.

Analogy

Regex improvements are like **upgrading a search engine so it finds patterns in text faster and more efficiently**.

✅ Interview Tip

For senior interviews, they may ask this follow-up:

"What is the difference between record and class?"

Short strong answer:

Feature	Record	Class
Equality	Value-based	Reference-based
Usage	Data models	Behavior-heavy objects
Immutability	Encouraged	Optional

If you'd like, I can also show you **20 real .NET interview questions that Microsoft, Amazon, and fintech companies commonly ask senior developers**.

11 Nice — lots to cover. Below I've formatted everything **in interview style** so you can read or say it directly. Each item has:

1. **Short, confident answer you can speak in an interview**,
2. **Concise technical details / bullets** you can expand on if asked, and
3. **A quick analogy** to make the idea memorable.

Microservices

1) How many microservices have you worked on?

Interview answer (sample)
"I've designed, developed, and maintained **about X–Y microservices** end-to-end across multiple bounded contexts — for example, authentication, billing, patient-data, regimen-

management and scheduling. I worked on everything from API contracts and database schema design to CI/CD and observability."

Details you can expand with

- Say the actual number you've touched (e.g., "~8 microservices" or "20+ microservices across two products").
- Mention responsibilities: design, migration, performance tuning, deployment (containers/Kubernetes), and monitoring.

Analogy
"Think of each microservice as a small shop in a marketplace — each shop owns its product line and can open/close independently without shutting the entire market."

---

2) What services did you migrate?

Interview answer (sample)
"I migrated user-management and order-processing services from a monolith to independently deployable services. The migration included extracting domain logic, creating service APIs, moving data ownership, and introducing event-driven communication for eventual consistency."

Details

- Mention specific services (e.g., Auth, Regimen Library, Billing).
- Steps: identify seams, create API facade, data migration strategy (strangling pattern), add consumer/producer contracts, tests, deploy to staging, monitor.

Analogy
"It's like moving departments from a single large office to separate offices — you first set up reception (API gateway) and define how mail (events/requests) is routed."

---

3) Why migrate from MVC to .NET Core?

Interview answer (sample)
"We migrated from ASP.NET MVC (Full Framework) to .NET Core to gain **better performance, cross-platform support, lightweight hosting (containers), improved middleware pipeline**, and modern dependency injection and configuration support."

Key points

- Performance gains and lower memory footprint.

- Cross-platform and container friendliness (Linux containers).
- Built-in DI, better HTTP pipeline (middleware).
- Simplified deployment and consistent tooling (SDK CLI).
- Long-term support / modern ecosystem (.NET 6/7/8).

Analogy
"It's like moving from a heavy desktop app to a modular web app that runs faster and can be deployed anywhere."

---

4) What scalability issues were you solving?

Interview answer (sample)
"We were solving horizontal scaling limits, high latency on synchronous calls, database hotspots, and single-process failures. Solutions included stateless services, caching, async/event-driven flows, partitioning data, and autoscaling."

Common issues & fixes

- **CPU-bound endpoints** → move to background processing / scale-out.
- **I/O and blocking** → async IO, non-blocking patterns.
- **Database bottlenecks** → read replicas, sharding, CQRS.
- **Session/state** → make services stateless; use distributed cache (Redis).
- **Thundering herd** → rate limiting, backpressure, circuit breakers.
- **Deployment scale** → container orchestration (K8s), autoscaling policies.

Analogy
"Before: one checkout cashier for all customers. After: multiple cashiers (instances), a queuing system, and a fast lookup for recurring customers (cache)."

---

2 Cloud Migration Strategy (Very Important)

1) You are required to migrate an application to cloud. What are the things you would consider?

Interview answer (sample)
"When migrating to cloud, I consider: application architecture, data gravity and storage, security/compliance, operational model (DevOps/CI-CD), cost, reliability & SLAs, networking, and observability. I'd run a discovery, categorize components, pick migration patterns, and pilot the move before broad rollout."

Checklist

- **Inventory & discovery**: components, dependencies, data stores, integrations.
- **Business drivers**: cost, scalability, DR, time-to-market.
- **Compliance/security**: encryption, identity, logging, region restrictions.
- **Migration pattern**: rehost (lift & shift), replatform, refactor, replace, or retire.
- **Data migration plan**: sync, cutover, rollback strategy.
- **Networking & connectivity**: VNet, peering, private endpoints.
- **Automation & CI/CD**: pipelines, infra-as-code (ARM/Bicep/Terraform).
- **Observability**: metrics, logs, tracing.
- **Cost modeling** and tagging for chargeback.
- **Runbook & rollback**: validation tests and fallbacks.

Analogy
"Migrating to cloud is like moving a factory: you audit machines (apps), decide which move as-is, which need retrofit, how to ship raw materials (data), and set up new logistics (network and monitoring)."

---

2) What are the different factors you evaluate before migrating a legacy system to Azure?

Interview answer (sample)
"I evaluate application dependencies, data size & patterns, regulatory needs, current architecture (stateful vs stateless), expected scale and performance, third-party integrations, operational readiness, and vendor lock-in concerns."

Factors

- **Compatibility** (Windows/Linux requirements).
- **Data gravity** (large datasets favor co-located services).
- **Latency requirements** (geo distribution).
- **Statefulness** (session, sticky behavior).
- **Third-party services** (SaaS integrations).
- **Skillset & operational maturity** (DevOps readiness).
- **Cost and licensing** (bring-your-own-license).
- **Security & compliance** (HIPAA, GDPR).
- **Disaster recovery & backup requirements**.

Analogy
"Think of it as checking whether your car parts will fit into the new garage and if the new garage meets your insurance requirements."

---

3) How do you decide between IaaS, PaaS, or other cloud models?

Interview answer (sample)
"Choose IaaS if you need full control of the OS or have legacy constraints. Choose PaaS to minimize operational overhead and accelerate development when the platform fits (managed databases, web apps). Consider serverless for event-driven, bursty workloads and containers for microservices portability."

Decision rules

- **IaaS** (VMs) → lift & shift, OS-level dependencies, custom runtime.
- **PaaS** (App Service, Azure SQL Managed Instance) → less ops, built-in scaling and patching.
- **Containers/Kubernetes** → microservices, portability, complex orchestration.
- **Serverless (Functions)** → short-lived event-driven tasks, scale-to-zero.
- **Managed DBs** → reduces ops, use when you can accept platform constraints.

Trade-offs

- Control vs. operational effort.
- Cost predictability vs. developer velocity.
- Vendor-managed features (backups, HA) vs customization.

Analogy
"IaaS is renting the entire house; PaaS is renting a serviced apartment; serverless is like booking a desk in a co-working space for only the hours you need."

---

4) What would you evaluate before moving a monolithic application?

Interview answer (sample)
"For a monolith I'd evaluate coupling between modules, database schema and ownership, deployment cadence, performance hotspots, test coverage, and business priorities to decide if we should rehost, replatform, or refactor to microservices."

Evaluation steps

- **Domain decomposition**: Identify bounded contexts and seams.
- **Code complexity & test coverage**: is refactor safe?
- **Data ownership**: shared DB vs separate schemas.
- **Operational constraints**: deployment window, downtime tolerance.
- **Performance & resource patterns**: which parts need scaling.
- **Inter-module dependencies**: tight coupling increases refactor cost.
- **Business criticality**: migrate low-risk modules first (strangler pattern).

Analogy
"Moving a monolith is like remodeling an old house — you can renovate room-by-room (strangle), or move the whole neighborhood to a new development (rewrite), depending on time and risk."

---

5) How do you approach modernization from legacy to cloud?

Interview answer (sample)
"My approach: 1) Assess and prioritize (business value vs risk), 2) Start small with a pilot, 3) Use the strangler pattern to extract features into services, 4) Introduce CI/CD and infra-as-code, 5) Move data with sync & cutover strategy, 6) Harden with monitoring, security and autoscaling."

Phases

1. **Assessment & discovery**
2. **Quick wins & lift-and-shift** for non-critical parts
3. **Refactor/Replace** for high-value components
4. **Containerize** where portability helps
5. **Adopt managed services** (DB, identity) where beneficial
6. **Automation, observability, and runbooks**
7. **Operate & optimize** (cost, performance)

Analogy
"Modernization is like renovating a train while it's running — you replace carriages one at a time so service continues."

---

6) How would you handle background services in cloud migration?

Interview answer (sample)
"For background work I'd evaluate whether to use managed queues and background processors: e.g., Azure Service Bus/Storage Queues + Azure Functions, Azure WebJobs, or containerized worker services with Kubernetes. For long-running jobs, use durable functions or container jobs with robust retry and idempotency."

Options & trade-offs

- **Short/background tasks** → Azure Functions + Queue triggers (serverless).
- **Worker services** → Containerized .NET Worker with Kubernetes/AKS or App Service WebJobs for more control.
- **Scheduling** → Azure Scheduler, Durable Functions, K8s cronjobs.
- **Messaging** → Service Bus (pub/sub, FIFO), Event Grid, or Storage Queue.

- **State & retries** → durable patterns, idempotency, poison-message handling.
- **Observability** → correlate messages/requests with distributed tracing.

Analogy
"Background services are the kitchen staff that prepare food behind the scenes; you decide whether to outsource some tasks to a catering service (managed queues/functions) or run your own kitchen (worker containers)."

---

7) What tech stack considerations would you make?

Interview answer (sample)
"I'd choose technologies that match business needs and team skills: standardized .NET version, container runtime, orchestrator (AKS), managed DB (Azure SQL / Cosmos for NoSQL), messaging (Service Bus/Event Grid), identity (Azure AD), observability (Application Insights), and infra-as-code (Bicep/Terraform)."

Checklist

- **Runtime**: target LTS .NET version (compatibility).
- **Containers**: Docker + Kubernetes (AKS) or App Service / Azure Container Apps.
- **Datastore**: Azure SQL, Cosmos DB, Blob storage choices based on workload.
- **Messaging**: Service Bus vs Event Grid vs Storage Queue.
- **Identity**: Azure AD / Managed Identities.
- **CI/CD**: GitHub Actions / Azure DevOps pipelines.
- **IaC**: Terraform or Bicep for reproducible infra.
- **Monitoring**: App Insights, Log Analytics, Prometheus/Grafana for metrics.
- **Security**: Key Vault for secrets, NSGs, WAF, and encryption.
- **Cost & SLA**: pick managed services to reduce ops when budget allows.

Analogy
"It's like choosing the right vehicle and support crew for a journey: truck, van, or courier, plus GPS, maintenance plan, and insurance depending on cargo and route."

---

1. Suppose we have an order entry system that writes to a database. How would you design it?
In an interview, I would say I would design the order entry system in a layered way, just like how a restaurant works. The customer places an order at the counter (this is like the frontend or API layer), the waiter checks the order and makes sure everything is correct (this is the business logic layer), and then the kitchen records and prepares the order (this

is the database layer). I would separate the system into presentation layer, service layer, and data access layer. The service layer would handle validations like checking stock, calculating price, and applying discounts. The data layer would handle saving orders in the database. I would also make the system stateless so it is easy to scale later. If the system is small, I would start with a clean modular monolith, and if it grows large, I can split it into microservices like Order Service, Inventory Service, and Payment Service.

---

2. What scalability considerations would you make?
Scalability means the system should handle more users without slowing down, just like adding more counters in a supermarket when customers increase. I would make the application stateless so we can add multiple servers behind a load balancer. I would also separate read and write operations if traffic becomes heavy, using database replicas for read operations. Caching frequently used data, like product details, would reduce load on the database. If data becomes very large, I might use database sharding to split data across multiple servers. The main goal is to remove bottlenecks and allow the system to grow smoothly.

---

3. How would you ensure high availability?
High availability means the system should not stop working even if something fails. It is like having backup generators in a hospital — if one fails, another takes over. I would deploy the application on multiple servers in different availability zones. A load balancer would send traffic only to healthy servers. For the database, I would use replication and automatic failover so if the main database crashes, a replica becomes the new primary. I would also implement monitoring, health checks, and alerts to quickly detect and fix problems. This way, downtime is minimized.

---

4. What auto-scaling strategies would you use?
Auto-scaling is like hiring more workers during a sale season and reducing them when business is normal. I would use cloud auto-scaling groups based on metrics like CPU usage, memory usage, or number of requests. For example, if CPU usage goes above 70%, new instances are automatically added. When traffic decreases, extra instances are removed to save cost. For background jobs, I would use queue-based scaling, where the number of worker instances increases if the message queue grows. This ensures the system adjusts automatically based on demand.

---

5. How would you design for performance?
Designing for performance means making sure the system responds quickly, like organizing a library so books are easy to find. I would optimize database queries and use

proper indexing. I would use caching (like Redis) to store frequently accessed data. Connection pooling would reduce database connection overhead. Heavy tasks like sending emails or generating reports would be handled asynchronously in the background. I would also perform load testing to identify slow areas and improve them. The idea is to reduce response time and make efficient use of resources so the system remains fast even under heavy load.

• When would you use microservices? • When would you avoid microservices? • What challenges do microservices introduce? • What is circuit breaker pattern? • How do microservices communicate in Azure? • What are the different communication protocols between microservices? • How do you implement asynchronous communication? • How do you implement Azure Service Bus? • What is the difference between Event Hub and Service Bus? • When would you use queues vs topics? • How do you handle distributed transactions?

1. When would you use microservices?
In an interview, I would say I use microservices when the system becomes large and complex, like a big shopping mall where each shop operates independently. If different modules like Orders, Payments, Inventory, and Shipping need to scale separately or are handled by different teams, microservices are a good choice. They are useful when features change frequently, when we need independent deployments, or when certain parts of the system require high scalability. For example, the payment service may need more resources during peak sales, while other services may not. Microservices allow each part to grow independently without affecting others.

---

2. When would you avoid microservices?
I would avoid microservices when the application is small or just starting, like a small local shop that doesn't need multiple departments. Microservices add complexity in deployment, monitoring, and communication. If the team is small and the system is simple, a well-structured monolithic application is better. Starting with microservices too early can slow down development because of extra infrastructure and operational overhead. So, if scalability and independent deployment are not immediate concerns, I would prefer a modular monolith.

---

3. What challenges do microservices introduce?
Microservices bring many benefits, but they also introduce challenges. It is like managing multiple small shops instead of one big store. Communication between services becomes complex. Network failures can happen. Debugging becomes harder because requests travel across multiple services. Data consistency is also a challenge since each service may have its own database. Monitoring, logging, and deployment become more complicated. So, while microservices provide flexibility, they increase operational complexity.

---

4. What is the circuit breaker pattern?
The circuit breaker pattern is like an electrical circuit breaker in a house. If there is a fault, it automatically cuts off power to prevent damage. In microservices, if Service A is calling Service B and Service B keeps failing, the circuit breaker stops calling it temporarily instead of repeatedly trying and wasting resources. After some time, it tries again to see if the service has recovered. This prevents system overload and improves stability.

---

5. How do microservices communicate in Azure?
In Azure, microservices can communicate either synchronously or asynchronously. For synchronous communication, we usually use REST APIs over HTTP. For asynchronous communication, we use messaging services like Azure Service Bus or Event Grid. Azure also provides API Management for routing and securing APIs. For containerized apps, Azure Kubernetes Service (AKS) is often used, where services communicate through internal service discovery.

---

6. What are the different communication protocols between microservices?
Microservices can communicate using HTTP/REST, gRPC, or messaging protocols like AMQP. HTTP/REST is simple and widely used. gRPC is faster and uses Protocol Buffers, making it suitable for high-performance systems. Messaging systems use protocols like AMQP for reliable message delivery. The choice depends on performance needs and system design.

---

7. How do you implement asynchronous communication?
Asynchronous communication is like sending a courier instead of making a phone call. You send a message and don’t wait for an immediate response. I would use message queues like Azure Service Bus. One service publishes a message to the queue, and another service consumes it when ready. This reduces tight coupling and improves system resilience. It is useful for tasks like sending emails, processing payments, or updating inventory.

---

8. How do you implement Azure Service Bus?
To implement Azure Service Bus, I would first create a namespace in Azure. Then I would create a queue or topic inside it. In the application, I would use the Azure.Messaging.ServiceBus SDK. One service sends messages using a ServiceBusSender, and another service listens using a ServiceBusProcessor. Messages

are processed and completed after successful handling. This ensures reliable message delivery.

---

9. What is the difference between Event Hub and Service Bus?
I explain it like this: Service Bus is like a post office where messages are delivered reliably to specific recipients. It supports queues and topics with guaranteed delivery and transaction support. Event Hub is like a news broadcast system. It handles high-volume streaming data, such as telemetry or logs. Service Bus is used for business messaging between services, while Event Hub is used for big data and event streaming scenarios.

---

10. When would you use queues vs topics?
Queues are used when one message should be processed by only one consumer. It is like a support ticket assigned to one employee. Topics are used when one message needs to be delivered to multiple subscribers. It is like sending a company-wide announcement email. If multiple services need the same event, I would use topics with subscriptions.

---

11. How do you handle distributed transactions?
Distributed transactions are challenging in microservices because each service has its own database. Instead of traditional two-phase commit, which is complex and slow, I would use the Saga pattern. Saga works like a travel booking process. If flight booking succeeds but hotel booking fails, the system cancels the flight booking as a compensation action. This ensures eventual consistency without locking resources across services.

• How do you implement security? • Have you implemented Identity Server? • How do you use OpenID Connect? • How do you implement JWT? • How do claims and resources work? • Have you worked with certificates? • How do you use certificates in Azure? • What is the protocol used for secure communication? • Do you know about TLS 1.2? • How do you verify if Azure Storage supports TLS 1.2? • How do you configure TLS? • What is the difference between TLS versions? ⚠ TLS questions were probing for deep Azure security understanding.

1. How do you implement security?
In an interview, I would say security is like protecting a bank. First, you control who can enter (authentication). Then you control what they can access inside (authorization). I implement authentication using OAuth2/OpenID Connect and JWT tokens. For authorization, I use role-based or policy-based access control. I always encrypt communication using HTTPS (TLS), store secrets in Azure Key Vault, hash passwords securely, and follow the principle of least privilege. I also implement logging, monitoring, and input validation to prevent attacks like SQL injection and XSS.

---

2. Have you implemented Identity Server?
Yes, I have worked with Identity Server for centralized authentication. Identity Server acts like a security gatekeeper. Instead of every application managing its own login system, Identity Server provides a single authentication service. Applications redirect users to Identity Server for login, and it issues tokens (like entry passes). This makes the system secure, scalable, and easier to manage because authentication logic is centralized.

---

3. How do you use OpenID Connect?
OpenID Connect is built on top of OAuth2 and is used for authentication. I explain it like this: OAuth2 gives permission, OpenID Connect confirms identity. When a user logs in, the application redirects them to the identity provider. After successful login, the identity provider sends back an ID token and access token. The ID token confirms who the user is, and the access token is used to access APIs securely.

---

4. How do you implement JWT?
JWT (JSON Web Token) is like a digital ID card. After a user logs in successfully, the server generates a signed JWT containing user details (claims). This token is sent with each request in the Authorization header. The API validates the token signature and expiry before processing the request. In .NET, I configure JWT authentication middleware, define issuer, audience, signing key, and validation parameters to secure APIs.

---

5. How do claims and resources work?
Claims are pieces of information about the user, like name, role, or email. Think of claims as details written on your ID card. Resources are what the user wants to access, like APIs or data. During authorization, the system checks if the user's claims allow access to that resource. For example, if a claim says "Role: Admin," the user can access admin APIs. Claims-based authorization makes access control flexible and secure.

---

6. Have you worked with certificates?
Yes, I have worked with certificates for secure communication and authentication. Certificates are like digital passports that prove identity. They contain a public key and are issued by a trusted Certificate Authority (CA). I have used certificates for enabling HTTPS, securing APIs, and configuring secure connections between services.

---

7. How do you use certificates in Azure?
In Azure, certificates can be stored in Azure Key Vault. Applications can retrieve certificates securely from Key Vault instead of storing them in code. Certificates are used for enabling HTTPS on Azure App Services, securing custom domains, and authenticating applications (like service-to-service authentication). In Azure Application Gateway or App Service, we bind the certificate to enable secure SSL/TLS communication.

---

8. What is the protocol used for secure communication?
The protocol used for secure communication over the internet is HTTPS, which uses TLS (Transport Layer Security). TLS encrypts data between client and server so attackers cannot read or modify it. It ensures confidentiality, integrity, and authentication.

---

9. Do you know about TLS 1.2?
Yes. TLS 1.2 is a secure version of the Transport Layer Security protocol. It provides strong encryption and secure handshake mechanisms. It protects against older vulnerabilities found in SSL and earlier TLS versions. Most modern systems enforce TLS 1.2 or higher because older versions like TLS 1.0 and 1.1 are considered insecure.

---

10. How do you verify if Azure Storage supports TLS 1.2?
In Azure, I can verify this by checking the “Minimum TLS version” setting in the Azure Storage Account configuration. Azure allows enforcing a minimum TLS version (like TLS 1.2). I can also use Azure CLI or PowerShell to check this configuration. If TLS 1.2 is set as minimum, any lower version connection will be rejected.

---

11. How do you configure TLS?
TLS can be configured in Azure by setting the minimum TLS version in App Service, Storage Account, or other Azure resources. For web apps, I enable HTTPS only and select minimum TLS 1.2 in the TLS/SSL settings. I also bind a valid SSL certificate to the domain. This ensures encrypted and secure communication.

---

12. What is the difference between TLS versions?
The main difference between TLS versions is security strength and performance. Older versions like TLS 1.0 and 1.1 have known vulnerabilities and weaker encryption algorithms. TLS 1.2 introduced stronger cipher suites and better security. TLS 1.3 is even faster and more secure because it reduces handshake steps and removes insecure

algorithms. In simple terms, each newer version improves security and efficiency compared to the previous one.

Since these TLS questions probe deep Azure security knowledge, I would also mention that enforcing minimum TLS version, using managed certificates, integrating with Azure Key Vault, and regularly auditing security configurations are best practices in enterprise Azure environments.

Architecture-Level Depth Probing • How would you handle integration architecture? • How would you define cloud strategy for large enterprise? • How would you standardize architecture across applications? • How would you design reusable architecture patterns?

1. How would you handle integration architecture?
In an interview, I would say integration architecture is like building proper roads between different cities. If the roads are not well planned, traffic becomes messy. Similarly, in enterprise systems, different applications like CRM, ERP, Billing, and third-party systems must communicate in a structured way. I would avoid point-to-point integration because that becomes difficult to manage, like connecting every house directly with wires. Instead, I would use an API-first approach and an integration layer using API Gateway or middleware like Azure API Management and Service Bus. For real-time communication, I would use REST or gRPC. For asynchronous communication, I would use messaging systems like Azure Service Bus or Event Grid. I would also define clear contracts, versioning strategy, centralized logging, monitoring, and security standards. The goal is loose coupling, scalability, and easy maintainability.

---

2. How would you define cloud strategy for a large enterprise?
Defining a cloud strategy for a large enterprise is like planning a smart city. You cannot just move everything randomly to the cloud. First, I would assess the current landscape — applications, dependencies, compliance requirements, and data sensitivity. Then I would define whether we follow public cloud, hybrid cloud, or multi-cloud strategy. I would categorize applications using the 6R strategy (Rehost, Refactor, Re-architect, Rebuild, Replace, Retire). Governance is very important — I would define policies for security, cost management, tagging, identity management, and networking. I would implement centralized monitoring, DevOps pipelines, disaster recovery strategy, and cost optimization practices. The cloud strategy must align with business goals, scalability needs, compliance requirements, and long-term digital transformation vision.

---

3. How would you standardize architecture across applications?
Standardizing architecture is like creating company-wide building standards. Every building may look different, but safety rules and foundation principles remain the same. I

would define enterprise architecture guidelines that include coding standards, security policies, logging frameworks, CI/CD pipelines, API standards, naming conventions, and infrastructure templates. I would create reusable templates using Infrastructure as Code like ARM or Terraform. Shared components like authentication, logging, exception handling, and monitoring would be standardized. I would also establish architecture review boards to ensure compliance. This reduces inconsistency, improves maintainability, and ensures all teams follow best practices.

---

4. How would you design reusable architecture patterns?
Designing reusable architecture patterns is like creating a blueprint that can be reused to build multiple houses. Instead of solving the same problem again and again, we define proven design patterns. For example, API Gateway pattern, Circuit Breaker pattern, Saga pattern, Event-Driven architecture, or CQRS pattern. I would document these patterns clearly with use cases, diagrams, pros and cons, and implementation guidelines. I would also create reference implementations or starter templates that teams can use. Reusable patterns save time, improve quality, and ensure consistency across projects. The key idea is to convert experience and lessons learned into repeatable, scalable solutions.

Performance Optimization Scenario • You mentioned performance issues — what was the root cause? • How did you solve database performance bottlenecks? • How did you handle replication (primary/secondary)? • What optimizations did you apply at database level? • What optimizations did you apply at C# level? • How did you use async or parallelism to improve performance?

1. You mentioned performance issues — what was the root cause?
In an interview, I would explain it like this: the root cause was similar to a traffic jam on a single narrow road. The system was slow during peak hours because too many requests were hitting the database at the same time. After investigation, we found slow SQL queries, missing indexes, and some N+1 query problems from the ORM layer. There were also synchronous calls blocking threads. So the main root causes were inefficient queries, lack of indexing, and blocking operations at application level.

---

2. How did you solve database performance bottlenecks?
I treated it like fixing a warehouse where items were hard to find. First, I analyzed slow queries using execution plans. Then I added proper indexes on frequently searched columns. I removed unnecessary joins and optimized queries to fetch only required columns instead of SELECT *. I also reduced database round trips by batching operations. In some cases, I introduced caching for frequently read data. These changes reduced database load significantly and improved response time.

---

3. How did you handle replication (primary/secondary)?
Replication is like having a main office and branch offices. The primary database handled write operations, and secondary replicas handled read operations. I configured read-only replicas for reporting and heavy read traffic. In the application, I separated read and write connection strings. This reduced load on the primary database and improved scalability. It also improved availability because if the primary failed, failover mechanisms could promote a secondary.

---

4. What optimizations did you apply at database level?
At the database level, I applied indexing on high-filter and join columns. I reviewed execution plans to remove table scans. I optimized stored procedures and reduced locking by using proper isolation levels. I ensured proper normalization but avoided over-normalization when it affected performance. I also implemented partitioning for very large tables and regular maintenance tasks like index rebuilding and updating statistics. These steps kept the database efficient and stable.

---

5. What optimizations did you apply at C# level?
At the C# level, I optimized LINQ queries to avoid unnecessary in-memory processing. I avoided loading large datasets when not needed and used pagination. I implemented caching using in-memory cache or distributed cache like Redis. I reduced object creation where possible and used dependency injection efficiently. I also ensured proper use of async/await to avoid thread blocking. These changes reduced memory usage and improved API response times.

---

6. How did you use async or parallelism to improve performance?
Using async is like allowing a waiter to serve multiple tables instead of waiting at one table until the customer finishes. I converted blocking database and API calls to async/await so threads were not blocked while waiting for I/O operations. For independent tasks, like calling multiple external services, I used Task.WhenAll to run them in parallel. For CPU-intensive tasks, I used parallel processing carefully. This improved throughput and allowed the system to handle more concurrent users without increasing hardware resources.

1. Are you comfortable writing parallel programming code?
Yes, I am comfortable writing parallel programming code. I understand that parallel programming is like assigning multiple workers to complete tasks faster instead of relying on a single worker. However, I also know that parallelism must be used carefully because it introduces complexity like race conditions and deadlocks. So I use it only when there is a real performance need and tasks are independent.

---

2. What constructs have you used for parallel programming?
I have used constructs like `Task`, `async/await`, `Parallel.ForEach`, `Thread`, `lock`, `Monitor`, `Mutex`, and `Semaphore`. I also use `Task.WhenAll` for running multiple asynchronous operations concurrently. These constructs help in managing concurrency while maintaining control over shared resources.

---

3. Have you used Parallel.ForEach?
Yes, I have used `Parallel.ForEach` when processing large collections where each iteration is independent. It is like dividing a large pile of files among multiple employees so they can process them simultaneously. It improves performance for CPU-bound operations when there are no dependencies between iterations.

---

4. Have you used Tasks?
Yes, I frequently use `Task` and `async/await`. Tasks are lightweight and managed by the thread pool, making them more efficient than manually creating threads. They are very useful for I/O-bound operations like database calls or API calls because they do not block threads while waiting.

---

5. Have you used Threading?
Yes, I have worked with threads directly in earlier implementations. However, I prefer using `Task` because it is higher-level and easier to manage. Direct threading is like manually managing workers, while Tasks are like assigning work to a managed team that handles scheduling automatically.

---

6. What are drawbacks of Threading?
Direct threading can be complex and risky. Threads consume more memory, and managing them manually can lead to race conditions, deadlocks, and resource exhaustion. It is harder to scale and debug. That is why modern .NET applications prefer the Task Parallel Library instead of raw threads.

---

7. When should we NOT use Parallel.ForEach?
We should not use `Parallel.ForEach` when tasks are I/O-bound or when each iteration depends on another. It is also not suitable for small datasets where the overhead of

managing parallel threads may reduce performance instead of improving it. If operations involve shared state heavily, it may create contention issues.

---

8. What happens if dependencies exist between iterations?
If iterations depend on each other, parallel execution can cause incorrect results. It is like multiple people editing the same document without coordination — changes may conflict. In such cases, we should use sequential processing or redesign the logic to remove dependencies.

---

9. Is it safe to use shared variables in parallel loops?
No, it is not safe without proper synchronization. Shared variables can cause race conditions where multiple threads update the same variable at the same time, leading to unpredictable results.

---

10. How do you handle shared state?
To handle shared state, I use synchronization techniques like `lock`, `Monitor`, or thread-safe collections like `ConcurrentDictionary`. Sometimes I redesign the logic to avoid shared state completely, which is the safest approach.

---

11. Have you used locking?
Yes, I have used the `lock` keyword in C#. It ensures that only one thread can enter a critical section at a time. It is simple and commonly used for protecting shared resources.

---

12. Have you used Monitor?
Yes, `Monitor` is the underlying mechanism behind `lock`. It provides more control like `Wait`, `Pulse`, and `TryEnter`. It is useful when we need advanced synchronization beyond basic locking.

---

13. Have you used Mutex?
Yes, Mutex is used for cross-process synchronization. Unlike `lock` or `Monitor`, which work within a single process, Mutex can synchronize access across multiple applications. It is heavier than Monitor and should be used only when needed.

---

14. When should we use Semaphore?
Semaphore is used when we want to limit the number of threads accessing a resource. For example, if only 5 threads should access a database connection pool at a time, we use Semaphore. It controls concurrency rather than allowing only one thread like a lock.

---

15. When should we use Mutex vs Monitor?
Monitor is used for synchronization within the same process and is lightweight. Mutex is used when synchronization is needed across multiple processes. So if only threads inside one application need coordination, Monitor (or lock) is better. If multiple applications must coordinate, use Mutex.

---

16. Do you know server core usage considerations?
Yes. When using parallel programming, we must consider the number of CPU cores. Creating more threads than available cores can cause context switching overhead and reduce performance. For CPU-bound tasks, the degree of parallelism should generally match the number of cores. For I/O-bound tasks, async programming is better than creating many threads. Proper thread pool management ensures optimal CPU utilization without exhausting server resources.

These questions mainly test concurrency maturity — understanding not just how to use parallel constructs, but when and when not to use them, and how to avoid common pitfalls like race conditions and deadlocks.

Live Coding Problem • Write a function that accepts a list of integers. • Find the maximum occurring number in the list. • Write the code. • Can you do it without inefficient looping? • Can you do it using LINQ? • How would you use GroupBy? • Are you finding max key or max count?
⚠ They were testing logic clarity + LINQ correctness.

In an interview, I would first clarify the requirement clearly: we need to find the number that appears the most times in the list. That means we are finding the **number with maximum frequency**, not the maximum numeric value. So we are finding the **key with the highest count**, not the highest key.

---

1. Basic Efficient Approach (Using Dictionary – O(n))

This avoids inefficient nested looping (which would be $O(n^2)$).

```
public static int GetMaxOccurringNumber(List<int> numbers)
{
    if (numbers == null || numbers.Count == 0)
        throw new ArgumentException("List cannot be empty");

    var frequency = new Dictionary<int, int>();

    foreach (var num in numbers)
    {
        if (frequency.ContainsKey(num))
            frequency[num]++;
        else
            frequency[num] = 1;
    }

    int maxCount = 0;
    int maxNumber = numbers[0];

    foreach (var kvp in frequency)
    {
        if (kvp.Value > maxCount)
        {
            maxCount = kvp.Value;
            maxNumber = kvp.Key;
        }
    }

    return maxNumber;
}
```

Why this is efficient?

It loops through the list once to build frequency (O(n)) and once through dictionary (O(k)).
Overall complexity: **O(n)** — no nested loops.

---

2. Can we do it without inefficient looping?

Yes. We avoid nested loops. Using a dictionary or LINQ grouping ensures we only iterate efficiently.

---

3. Using LINQ

```
public static int GetMaxOccurringNumber(List<int> numbers)
{
```

```
    return numbers
            .GroupBy(x => x)
            .OrderByDescending(g => g.Count())
            .First()
            .Key;
}
```

---

4. Using GroupBy Properly (More Optimal LINQ)

Instead of sorting everything (which is slightly heavier), we can use `MaxBy` (.NET 6+):

```
public static int GetMaxOccurringNumber(List<int> numbers)
{
    return numbers
            .GroupBy(x => x)
            .MaxBy(g => g.Count())
            .Key;
}
```

This avoids full sorting and directly finds the group with maximum count.

---

5. Are we finding max key or max count?

Very important clarification in interview:

- ❌ Not finding maximum numeric value.
- ✅ Finding the number whose **count is maximum**.
- So we are finding **the key with maximum frequency**.

If interviewer asks what happens in tie cases, we clarify:

- Current implementation returns the first occurring max.
- If tie-breaking logic is required (e.g., smallest number), we can adjust.

Final Interview Summary Statement

"I would solve this in O(n) time using a dictionary to count frequency. If using LINQ, I would use GroupBy and MaxBy to avoid unnecessary sorting. I would also clarify that we are finding the key with maximum count, not the maximum key itself."

If you'd like, I can also show:

- Handling ties
- Returning both number and count
- Handling very large datasets efficiently

• What is garbage collection? • Why do we need garbage collection? • How does GC work internally? • What are generations? • Does GC call Dispose? • Does Dispose call GC? • What is the relationship between Dispose and GC? • What is the difference between Dispose and Finalize? • Who calls Dispose? • Who calls Finalizer? • When is GC.Collect used? • Is using block related to GC?

1. What is garbage collection?
Garbage Collection (GC) is an automatic memory management system in .NET. I explain it like a cleaning service in an office. When employees (objects) are no longer working (no longer referenced), the cleaning team (GC) removes them and frees up space. This prevents memory leaks and reduces manual memory management.

---

2. Why do we need garbage collection?
We need GC because manually managing memory is risky and error-prone. It is like asking every employee to clean their own waste — mistakes will happen. Without GC, developers might forget to release memory, leading to memory leaks and application crashes. GC ensures unused objects are automatically removed, improving stability and reliability.

---

3. How does GC work internally?
Internally, GC works using a mark-and-compact algorithm. First, it pauses the application briefly and identifies objects that are still reachable (mark phase). Then it removes unreachable objects. After that, it compacts memory by moving remaining objects together to reduce fragmentation. GC works based on object references starting from "root" objects like static variables, stack variables, and CPU registers.

---

4. What are generations?
.NET GC uses generational collection. It assumes that most objects die young. So memory is divided into:

- **Gen 0** – Short-lived objects (most frequently collected)
- **Gen 1** – Medium-lived objects
- **Gen 2** – Long-lived objects

It is like sorting waste into temporary, reusable, and permanent storage. Objects start in Gen 0 and move to higher generations if they survive collections.

---

5. Does GC call Dispose?
No, GC does NOT call Dispose automatically. Dispose must be called manually by the developer. GC only handles memory cleanup, not unmanaged resources like file handles or database connections.

---

6. Does Dispose call GC?
No, Dispose does not call GC. Dispose is used to release unmanaged resources immediately. GC runs independently when needed. However, in Dispose pattern, we often call `GC.SuppressFinalize(this)` to prevent the finalizer from running if cleanup is already done.

---

7. What is the relationship between Dispose and GC?
GC manages memory automatically. Dispose is used to release unmanaged resources manually. GC cleans memory, but it does not guarantee timely release of resources like files or sockets. So Dispose complements GC. GC handles memory, Dispose handles unmanaged resources.

---

8. What is the difference between Dispose and Finalize?
Dispose is a method implemented through `IDisposable` and is called manually (or via `using`). Finalize (destructor) is called automatically by GC before the object is destroyed. Dispose is faster and deterministic. Finalize is non-deterministic and slower because it depends on GC timing. In simple terms: Dispose is controlled cleanup; Finalize is backup cleanup.

---

9. Who calls Dispose?
The developer calls Dispose manually or it is called automatically by the `using` statement. For example:

```
using (var file = new StreamWriter("test.txt"))
{
    file.WriteLine("Hello");
}
```

Here, Dispose is automatically called at the end of the block.

---

10. Who calls Finalizer?
The Garbage Collector calls the finalizer automatically when the object is being collected, if a finalizer is defined.

---

11. When is GC.Collect used?
`GC.Collect()` forces garbage collection manually. It is generally NOT recommended because GC is optimized to decide when to run. Forcing it can hurt performance. It is used only in rare scenarios like testing, memory profiling, or when releasing large memory blocks immediately after heavy operations.

---

12. Is using block related to GC?
Yes, indirectly. The `using` block ensures Dispose is called automatically, which releases unmanaged resources quickly. However, it does not directly trigger garbage collection. It simply ensures proper cleanup before GC runs.

Interview Summary Line

"GC handles memory automatically using generational collection. Dispose is used for deterministic cleanup of unmanaged resources. GC and Dispose serve different purposes but work together to ensure efficient resource management."

• What kind of SQL work have you done? • Have you optimized stored procedures? • How do you analyze execution plan? • What are index scans vs table scans? • What are clustered vs non-clustered indexes? • Have you used CTE? • Have you used temp tables? • What are views? • Why use views? • How can views help with security? • What are triggers? • What are before and after triggers? • What are auditing triggers?

1. What kind of SQL work have you done?
In an interview, I would say I have worked extensively with SQL for designing tables, writing complex queries, joins, subqueries, stored procedures, functions, indexing, performance tuning, and handling large datasets. I have also worked on query optimization, data migrations, writing CTEs, temp tables, and debugging production performance issues. In simple words, I have used SQL not just for fetching data, but for designing efficient and scalable database solutions.

---

2. Have you optimized stored procedures?
Yes, I have optimized stored procedures. It is like improving a slow recipe in a kitchen — reducing unnecessary steps and organizing ingredients better. I reviewed execution plans, removed unnecessary joins, avoided SELECT *, added proper indexes, reduced cursor usage, and ensured filtering happens early in the query. I also optimized parameters to avoid parameter sniffing issues when needed.

---

3. How do you analyze execution plan?
Execution plan is like a roadmap that shows how SQL Server retrieves data. I use the "Actual Execution Plan" in SSMS to see which operations consume the most cost. I check for table scans, missing index suggestions, high-cost operators, key lookups, and expensive joins. I focus on operations with the highest percentage cost because they usually indicate bottlenecks.

---

4. What are index scans vs table scans?
A table scan means SQL Server reads the entire table to find matching records. It is like searching for a name by reading every page of a book. An index scan reads the entire index instead of the table. It is slightly better but still scans many rows. Ideally, we want an index seek, where SQL jumps directly to the required data using an index — like using a book's index page to directly find a topic.

---

5. What are clustered vs non-clustered indexes?
A clustered index determines the physical order of data in the table. It is like arranging books physically in alphabetical order. A table can have only one clustered index. A non-clustered index is like a separate reference book that points to the actual data location. A table can have multiple non-clustered indexes. Clustered indexes are best for range queries, while non-clustered indexes help in filtering and lookups.

---

6. Have you used CTE?
Yes, I have used Common Table Expressions (CTE) for writing readable and structured queries. CTE is like creating a temporary named result set within a query. It improves clarity and is very useful for recursive queries, such as hierarchical data like parent-child relationships.

Example:

```
WITH EmployeeCTE AS
(
    SELECT Id, Name FROM Employees WHERE ManagerId IS NULL
)
SELECT * FROM EmployeeCTE;
```

---

7. Have you used temp tables?
Yes, I have used temporary tables when intermediate results need to be stored and reused multiple times. Temp tables are useful when dealing with large datasets or complex transformations. They are stored in tempdb and exist temporarily during the session.

---

8. What are views?
A view is a virtual table based on a SQL query. It does not store data itself but shows data from underlying tables. It is like a saved query that users can access as if it were a table.

---

9. Why use views?
Views simplify complex queries, improve reusability, and provide abstraction. Instead of writing a complex join every time, we create a view once and reuse it. It makes reporting and development easier.

---

10. How can views help with security?
Views can restrict access to specific columns or rows. For example, instead of giving users access to the entire employee table, we can create a view that hides salary information. So users only see allowed data. This adds an extra layer of security.

---

11. What are triggers?
Triggers are special stored procedures that automatically execute when certain events occur, like INSERT, UPDATE, or DELETE. It is like an automatic alarm that runs when data changes.

---

12. What are before and after triggers?
In SQL Server, we mainly have AFTER and INSTEAD OF triggers.

- AFTER trigger runs after the data modification is completed.

- INSTEAD OF trigger runs instead of the actual operation, allowing us to override default behavior.

In other databases like MySQL, BEFORE triggers execute before the operation, allowing validation or modification before data is saved.

---

13. What are auditing triggers?
Auditing triggers track changes made to data. For example, when a record is updated or deleted, the trigger logs old and new values into an audit table. It is like keeping a history log for compliance and tracking purposes. This is especially important in enterprise systems like healthcare or finance where tracking changes is mandatory.

Strong Interview Closing Line

"I have worked with SQL not only for writing queries but also for performance optimization, indexing strategy, execution plan analysis, and designing secure and maintainable database solutions."

• Is it one schema serving all customers? • Is it one database instance for all customers? • Is it secure to share one schema across customers? • How do you prevent one customer seeing another customer's data? • What are risks of shared schema? • How do you enforce country-level segregation? • Why not separate database per customer? • What are scaling concerns in single-schema model?

These questions are testing **multi-tenant architecture maturity**, especially in SaaS systems. I'll answer each separately in clear interview style with simple analogies.

---

1. Is it one schema serving all customers?

It depends on the architecture. In a **shared-schema multi-tenant model**, yes — one schema serves all customers. All customers' data sits in the same tables, but each row is tagged with a `TenantId` or `CustomerId`.

It is like an apartment building where all residents live in the same building, but each apartment belongs to a different person. The building is shared, but ownership is separated logically.

---

2. Is it one database instance for all customers?

In a shared model, yes — typically one database instance serves multiple customers.

However, in large enterprise SaaS, we may use:

- Shared database + shared schema
- Shared database + separate schemas
- Separate database per tenant

The choice depends on scale, compliance, and isolation requirements.

---

3. Is it secure to share one schema across customers?

It can be secure **if designed properly**. Security depends on application logic and database constraints.

Think of it like a bank locker room — many lockers in one room. It is secure as long as each locker has a proper lock. If the lock is weak, it becomes risky.

If row-level security, strict filtering, and proper access control are enforced, shared schema can be secure. But it increases risk if mistakes happen.

---

4. How do you prevent one customer seeing another customer's data?

The most common way is using a `TenantId` column in every table and enforcing filtering at every query.

For example:

- Every query must include `WHERE TenantId = @CurrentTenant`
- Use Row-Level Security (RLS) in SQL Server
- Apply global query filters in ORM like Entity Framework

It is like issuing a badge to each employee — they can only enter floors assigned to them.

Additionally:

- API layer must validate tenant from JWT claims
- Database layer must enforce tenant isolation
- Never trust frontend filtering

---

5. What are risks of shared schema?

Shared schema introduces several risks:

- Accidental data leakage if filtering is missed
- Complex queries with TenantId everywhere
- Harder performance tuning
- Noisy neighbor problem (one tenant consuming heavy resources)
- Difficult per-tenant backup/restore

It is like multiple companies sharing one office floor — if one company makes noise, everyone is affected.

---

6. How do you enforce country-level segregation?

For country-level segregation, especially for compliance (like GDPR), we can:

- Use separate databases per region
- Deploy separate infrastructure per country
- Use geo-fencing policies
- Store country code at tenant level
- Route traffic to region-specific database
- Use Azure region-based deployment

It is like storing European customer data only inside Europe to comply with laws.

For strict compliance, physical separation (different DB per region) is safer than logical filtering.

---

7. Why not separate database per customer?

Separate database per customer provides strong isolation but has trade-offs:

Pros:

- Strong security isolation
- Easy per-tenant backup/restore
- Performance isolation

Cons:

- Higher operational cost
- Harder to manage thousands of databases
- Schema updates must be deployed to every database
- Infrastructure complexity increases

It is like giving every customer their own building — very secure but very expensive to maintain.

---

8. What are scaling concerns in single-schema model?

In a single shared schema:

- Large tables grow very big
- Indexes become heavy
- Queries may slow down
- Maintenance becomes complex
- One large tenant can impact others
- Backup/restore affects all tenants

Also, partitioning strategies become important as data grows.

It is like one large supermarket serving the entire city — if it becomes too crowded, performance suffers.

Strong Interview Closing Line

"In multi-tenant architecture, the decision between shared schema and separate database depends on trade-offs between cost, isolation, compliance, scalability, and operational complexity. Security must be enforced at multiple layers — application, database, and infrastructure."

• What services have you used in Azure? • For what purpose did you use Service Bus? • How did you use Function Apps? • How did you use Storage? • What types of storage did you use? • Have you used containers? • What storage technologies have you worked with? • How did you implement logging? • How did you implement staging tables?

These questions are checking **practical Azure experience**, not just theory. I'll answer each separately in clear interview style with simple explanations.

---

1. What services have you used in Azure?

In my projects, I have used multiple Azure services depending on the requirement. I have worked with:

- **Azure App Service** – for hosting APIs and web applications
- **Azure SQL Database** – for relational data storage
- **Azure Service Bus** – for asynchronous messaging
- **Azure Function Apps** – for serverless background processing
- **Azure Storage Accounts** – for blob and file storage
- **Azure Key Vault** – for managing secrets and certificates
- **Azure Application Insights** – for monitoring and logging
- **Azure Virtual Networks** – for secure networking

I choose services based on scalability, security, and architecture requirements.

---

2. For what purpose did you use Service Bus?

I used Azure Service Bus for **asynchronous communication between microservices**.

For example, when an order is placed, instead of immediately calling the billing service synchronously, we publish a message to Service Bus. The billing service consumes it independently.

It is like sending a parcel instead of making a phone call — the sender does not wait for the receiver to process immediately. This improves scalability and reliability.

I also used:

- Queues for single consumer scenarios
- Topics for multiple subscribers
- Dead-letter queues for handling failed messages

---

3. How did you use Function Apps?

I used Azure Function Apps for background processing and event-driven workflows.

Examples:

- Processing messages from Service Bus
- Running scheduled jobs (using Timer Trigger)
- Handling blob uploads

- Performing lightweight background data processing

Function Apps are like on-demand workers — they automatically scale based on load and we don't need to manage servers.

---

4. How did you use Storage?

I used Azure Storage primarily for:

- Storing documents and files (like PDFs, reports)
- Storing exported data
- Temporary data processing
- Logging large structured data

For example, when generating large reports, we stored them in Blob Storage and returned a secure URL to the user.

---

5. What types of storage did you use?

I have worked with:

- **Blob Storage** – for file storage (documents, images, reports)
- **Table Storage** – for lightweight NoSQL structured data
- **Queue Storage** – for simple messaging scenarios
- **File Storage** – for shared file systems
- **Azure SQL Database** – for relational data

Each storage type serves different purposes. Blob is best for unstructured data, SQL for structured relational data.

---

6. Have you used containers?

Yes, I have worked with containers using Docker and deployed applications on Azure using:

- Azure Container Instances (ACI)
- Azure Kubernetes Service (AKS)

Containers package the application and its dependencies together. It is like shipping goods in standardized containers — they run consistently across environments.

---

7. What storage technologies have you worked with?

I have worked with:

- SQL Server
- Azure SQL Database
- PostgreSQL
- Azure Blob Storage
- Azure Table Storage
- Redis Cache (for caching)

I use relational databases for transactional systems and Blob storage for large file storage. Redis is used for improving performance via caching.

---

8. How did you implement logging?

For logging, I used:

- **Serilog** in .NET applications
- Azure Application Insights
- Structured logging with correlation IDs
- Log levels (Information, Warning, Error)

For distributed systems, I ensured logs include:

- Request ID
- Tenant ID
- User ID
- Exception details

Application Insights helps in monitoring performance, tracking exceptions, and analyzing telemetry. Logging is like installing CCTV cameras — it helps trace issues quickly.

---

9. How did you implement staging tables?

I used staging tables in scenarios like:

- Data migration
- Bulk imports
- ETL processes

- Data validation before final insert

The flow was:

1. Import raw data into staging table
2. Validate and clean data
3. Move valid data into main tables
4. Log errors separately

It is like inspecting goods in a warehouse before placing them on store shelves. Staging ensures data quality and prevents corrupt data from entering production tables.

Strong Interview Closing Line

"I have used Azure services in a practical, production-grade environment, focusing on scalability, asynchronous communication, monitoring, and secure storage while following best practices."

• What was your recent microservices architecture? • How did you break down monolith into microservices? • How did you handle Windows services in cloud migration? • How did you handle legacy Pascal components? • How did microservices communicate? • Did you use Service Bus? • Did you implement distributed transactions? • Did you use Saga pattern? • Did you use RabbitMQ? • How did you implement compensation logic? • How did you handle acknowledgement (positive/negative)?

These questions are probing **real-world migration and microservices maturity**. I'll answer them in structured interview style, clear and practical.

---

1. What was your recent microservices architecture?

In my recent architecture, we moved from a tightly coupled monolith to a domain-based microservices architecture.

We separated core domains like:

- Order Management
- Billing
- Inventory
- Notifications
- Reporting

Each service had:

- Its own database (database-per-service pattern)
- REST APIs for synchronous communication
- Azure Service Bus for asynchronous events
- Independent deployment pipelines

Think of it like converting one big factory into specialized smaller factories that operate independently but coordinate through messages.

---

2. How did you break down monolith into microservices?

We followed **domain-driven design (DDD)** principles.

Steps:

1. Identified bounded contexts
2. Analyzed database dependencies
3. Separated business capabilities
4. Introduced APIs around logical modules
5. Gradually extracted modules into independent services

We did not rewrite everything at once. We used the **strangler pattern** — slowly routing new functionality to microservices while old parts remained in the monolith until replaced.

It's like renovating a house room by room instead of demolishing everything at once.

---

3. How did you handle Windows services in cloud migration?

For Windows services, we evaluated whether they were:

- Scheduled jobs → Migrated to Azure Function (Timer Trigger)
- Background processing → Migrated to Azure Function or containerized worker service
- Long-running services → Converted to .NET Worker Services and deployed in containers or Azure App Service

We modernized them instead of lifting and shifting directly. The goal was cloud-native scalability.

---

4. How did you handle legacy Pascal components?

For legacy Pascal components, we used a phased approach:

- Identified critical business logic
- Wrapped Pascal components behind APIs initially
- Gradually reimplemented core logic in .NET
- Validated outputs to ensure correctness

Instead of immediate replacement, we isolated legacy components and reduced dependency gradually.

It was like translating an old manual into a modern digital system step by step.

---

5. How did microservices communicate?

We used a hybrid communication model:

- REST APIs for synchronous request-response scenarios
- Azure Service Bus for asynchronous event-driven communication

Synchronous was used when immediate response was required (e.g., validation). Asynchronous was used for order processing, billing, notifications, etc.

---

6. Did you use Service Bus?

Yes, we used Azure Service Bus extensively.

We used:

- Queues for single consumer processing
- Topics for publishing domain events to multiple subscribers
- Dead-letter queues for failed message handling

It helped us decouple services and improve resilience.

---

7. Did you implement distributed transactions?

We avoided traditional distributed transactions (like 2-phase commit) because they reduce performance and increase complexity.

Instead, we used eventual consistency with messaging and compensation logic.

---

8. Did you use Saga pattern?

Yes, we implemented Saga pattern for multi-service workflows.

For example:
Order → Payment → Inventory → Notification

If Payment failed:

- We triggered compensation to cancel the order.

Each service published events and listened for next steps.

Saga was orchestrated using message flow rather than centralized locking.

---

9. Did you use RabbitMQ?

In earlier systems, yes. RabbitMQ was used before migrating to Azure Service Bus.

RabbitMQ handled:

- Message queues
- Event broadcasting
- Retry and dead-lettering

In Azure cloud migration, we preferred Service Bus for native integration and managed infrastructure.

---

10. How did you implement compensation logic?

Compensation logic was implemented as reverse actions.

Example:

- If inventory was reduced but payment failed → inventory was restored.
- If order created but downstream failed → order status marked cancelled.

Each service had explicit compensation handlers triggered by failure events.

It is like booking a flight and hotel — if hotel booking fails, flight booking is cancelled.

---

11. How did you handle acknowledgement (positive/negative)?

In Service Bus:

- On successful processing → message was completed (acknowledged)
- On failure → message was abandoned or dead-lettered
- After multiple retries → moved to dead-letter queue

We implemented retry policies with exponential backoff.

In RabbitMQ:

- Used manual ACK
- Sent NACK for failed processing

Acknowledgement ensured messages were not lost and not processed multiple times incorrectly.

Strong Interview Closing Statement

"In our microservices migration, we focused on domain separation, event-driven communication, eventual consistency using Saga, proper message acknowledgement handling, and gradual modernization of legacy systems to ensure minimal business disruption."

• You mentioned CQRS — explain it. • How did you separate read and write? • How did you sync primary and secondary DB? • How did handlers determine read vs write? • How did routing to correct DB happen? • Was it multi-tenant? • How did you manage connection strings per tenant? • How did you inject tenant context? • Did you use JWT token for tenant identification? • How did you pass tenant ID across layers? • Did you use HTTP Context? • How did you avoid passing tenant in every method manually? • Did you cache connection strings? • How did you encode connection strings?

These questions are probing **deep architecture maturity**, especially around **CQRS + multi-tenant + database routing**. I'll answer clearly in structured interview format.

---

1. You mentioned CQRS — explain it.

CQRS stands for **Command Query Responsibility Segregation**.

It means we separate:

- **Commands** → operations that modify data (Insert, Update, Delete)
- **Queries** → operations that read data

Instead of one model handling both, we split them.

Think of it like a restaurant:

- Kitchen handles cooking (write operations)
- Waiter handles serving menu information (read operations)

This separation improves scalability, clarity, and performance tuning.

---

2. How did you separate read and write?

We separated them at:

- Application layer (separate CommandHandlers and QueryHandlers)
- Database level (Primary DB for writes, Secondary replica for reads)

Commands always hit the primary database.
Queries were routed to read replicas.

We used separate interfaces like:

- ICommandHandler
- IQueryHandler

This ensured clear responsibility separation.

---

3. How did you sync primary and secondary DB?

We used database replication.

Primary handled all write transactions.
Secondary replicas were configured with asynchronous replication.

Replication ensured that secondary DB automatically synced changes from primary.

There may be slight replication lag, so eventual consistency was acceptable for read scenarios.

---

4. How did handlers determine read vs write?

The separation was structural.

If a request was a command → it went to CommandHandler → write DB connection.

If a request was a query → it went to QueryHandler → read DB connection.

This was enforced by architecture, not runtime decision-making.

We did not dynamically decide — it was designed clearly.

---

5. How did routing to correct DB happen?

We configured:

- Write connection string → Primary DB
- Read connection string → Secondary DB

In Dependency Injection:

- Command DbContext used write connection
- Query DbContext used read connection

Routing was handled at infrastructure level.

The handler type automatically determined which DbContext was injected.

---

6. Was it multi-tenant?

Yes, it was a multi-tenant system using shared database with TenantId column strategy.

Each table had TenantId.

Row-level filtering was enforced in ORM and database.

---

7. How did you manage connection strings per tenant?

There were two models:

For shared database:

- Same DB, but tenant filtered via TenantId.

For higher isolation tenants:

- Connection string was resolved dynamically per tenant.

We stored tenant-to-connection mapping in a secure configuration store or central database.

---

8. How did you inject tenant context?

We implemented a TenantProvider service.

Flow:

- Extract tenant ID from JWT token
- Store it in a scoped service
- Inject TenantProvider wherever needed

This allowed any service to access current tenant context without passing it manually.

---

9. Did you use JWT token for tenant identification?

Yes.

Tenant ID was included as a claim in JWT token.

Example:

- "tenant_id": "12345"

On each request:

- Middleware extracted tenant claim

- Stored it in TenantContext

This ensured secure tenant identification.

---

10. How did you pass tenant ID across layers?

We avoided passing tenantId manually in every method.

Instead:

- Used scoped TenantContext service
- Injected it via dependency injection
- ORM applied global query filters using tenantId

This kept code clean and avoided repetition.

---

11. Did you use HTTP Context?

Yes.

In middleware:

- Extracted tenant claim from HttpContext.User
- Set it in TenantContext

After that, business layers did not directly depend on HttpContext.

Only infrastructure layer accessed it.

---

12. How did you avoid passing tenant in every method manually?

We used:

- Scoped TenantContext service
- Global query filters in Entity Framework
- Custom DbContext override to automatically apply TenantId

This ensured every query automatically filtered by tenant without manual filtering.

It prevented developer mistakes.

---

13. Did you cache connection strings?

Yes.

If using per-tenant database model:

- We cached resolved connection strings in memory or distributed cache (like Redis)
- This avoided hitting configuration store repeatedly

Cache invalidation strategy was carefully managed.

---

14. How did you encode connection strings?

We never stored plain text credentials in code.

We used:

- Azure Key Vault for storing secrets
- Managed Identity for secure access
- Encrypted configuration sections if needed

Connection strings were encrypted at rest and accessed securely.

Strong Closing Line (Very Important for Interview)

"Our CQRS and multi-tenant design ensured clean separation of concerns, read/write scalability, secure tenant isolation using JWT-based context injection, and infrastructure-level database routing without polluting business logic."

If you want, I can now simulate **how a senior architect would challenge you on replication lag, data consistency, or cross-tenant query risks.**

• Have you integrated APIs with Azure? • Have you used Azure API Management? • How do you register applications? • How do you manage secrets? • How do you manage manifest configuration?

These questions are probing **Azure integration + identity + enterprise governance maturity**. I'll answer clearly in interview format with practical depth.

---

1. Have you integrated APIs with Azure?

Yes, I have integrated APIs with Azure in multiple ways.

- Hosted APIs on **Azure App Service**
- Secured APIs using **Azure AD authentication**
- Used **Azure API Management (APIM)** as a gateway
- Connected APIs to **Azure Service Bus**
- Used **Application Insights** for monitoring

For example, external clients accessed our APIs through APIM, which handled authentication, rate limiting, and logging before routing traffic to backend services.

It's like having a secure reception desk before entering the main office.

---

2. Have you used Azure API Management?

Yes.

Azure API Management acts like a **central gateway for all APIs**.

We used APIM for:

- Centralized authentication enforcement
- Rate limiting and throttling
- API versioning
- Request/response transformation
- Logging and monitoring
- Hiding internal service URLs

For example:
Clients never directly accessed microservices.
They called APIM → APIM validated JWT → routed request to correct backend service.

This improves security, governance, and scalability.

---

3. How do you register applications?

Applications are registered in **Azure Active Directory (Azure AD)**.

Steps:

1. Go to Azure AD → App Registrations
2. Register new application
3. Define redirect URIs (for web apps)
4. Generate Client ID
5. Configure client secret or certificate
6. Define API permissions
7. Expose scopes if it's an API

For example:

- Frontend app gets registered
- Backend API gets registered
- Frontend requests token for API scope
- Azure AD issues JWT

This enables secure OAuth2 / OpenID Connect authentication.

---

4. How do you manage secrets?

We never store secrets in code.

We use:

- **Azure Key Vault** for storing:
  - Connection strings
  - Client secrets
  - Certificates
  - API keys
- Use **Managed Identity** for secure access
- Avoid hardcoding credentials in appsettings.json

Flow:
App → Managed Identity → Key Vault → Fetch secret securely

This ensures secrets are encrypted at rest and access-controlled.

It's like storing important documents in a bank locker instead of keeping them at home.

---

5. How do you manage manifest configuration?

In Azure AD App Registration, the **manifest file** is used to configure advanced settings like:

- Exposing custom scopes
- Adding app roles
- Defining optional claims
- Configuring access token version

For example:
We added custom roles like:

- "Admin"
- "Manager"
- "Viewer"

Then these roles appear as claims inside JWT token.

Manifest allows fine-grained control beyond UI configuration.

We carefully manage it to:

- Enable multi-tenant support
- Define API permissions
- Control token behavior

Strong Interview Closing Line

"In Azure-based API integrations, I focus on secure app registration, centralized API governance through APIM, managed identity-based secret management via Key Vault, and properly configured Azure AD manifests to control identity and access at enterprise scale."

• What design patterns have you used? • Have you used Dependency Injection? • Have you used Repository Pattern? • Have you used Factory Pattern? • Have you used Singleton? • Have you used CQRS? • Have you used Saga pattern?

These questions are checking your **design maturity and practical application**, not just definitions. I'll answer each clearly in interview style with simple explanations and real usage context.

---

1. What design patterns have you used?

In my projects, I have used several design patterns based on need. Some commonly used ones are:

- **Dependency Injection**
- **Repository Pattern**
- **Factory Pattern**
- **Singleton**
- **CQRS**
- **Saga Pattern**
- **Strategy Pattern**
- **Mediator Pattern**
- **Circuit Breaker Pattern**

I don't use patterns just for theory — I apply them when they solve a real architectural problem. Design patterns are like proven blueprints; we use them when the situation matches the problem they solve.

---

2. Have you used Dependency Injection?

Yes, extensively.

Dependency Injection (DI) is like hiring services instead of building everything inside your house. Instead of a class creating its own dependencies, they are injected from outside.

In .NET Core, I used built-in DI container:

- Registered services in `Startup` or `Program.cs`
- Used scoped, transient, and singleton lifetimes
- Injected services via constructor injection

DI improves:

- Testability
- Loose coupling
- Maintainability

It's one of the most fundamental patterns in modern applications.

---

3. Have you used Repository Pattern?

Yes.

Repository Pattern abstracts data access logic from business logic.

Instead of business layer directly querying database, it interacts with repository interfaces.

For example:

- `IOrderRepository`
- `IUserRepository`

This makes code:

- Clean
- Testable (mockable)
- Independent from specific ORM

It's like having a warehouse manager handle inventory instead of everyone going directly into the storage room.

---

4. Have you used Factory Pattern?

Yes.

Factory Pattern is used when object creation logic is complex or depends on runtime conditions.

For example:

- Creating different payment providers
- Selecting strategy based on request type
- Creating tenant-specific services

Instead of writing:

```
if(type == A) new A();
if(type == B) new B();
```

We use a Factory to centralize object creation logic.

It improves maintainability and follows Open-Closed Principle.

---

5. Have you used Singleton?

Yes.

Singleton ensures only one instance of a class exists.

Examples:

- Configuration manager
- Caching service
- Logging service

In .NET Core, we typically configure Singleton via DI container.

However, I use it carefully because overusing Singleton can create tight coupling and shared-state issues.

---

6. Have you used CQRS?

Yes.

I used CQRS to separate:

- Command operations (write)
- Query operations (read)

Benefits:

- Separate scaling strategies
- Clear responsibility separation
- Easier performance optimization

We used separate handlers for commands and queries and even separate read/write databases in some cases.

CQRS is useful in high-scale enterprise systems.

---

7. Have you used Saga pattern?

Yes.

Saga pattern was used in microservices architecture to manage distributed transactions.

Instead of locking multiple services, we implemented:

- Event-driven workflow
- Compensation logic for failures

Example:
Order → Payment → Inventory
If Payment fails → cancel Order

Saga ensures eventual consistency without heavy distributed transactions.

It's like booking travel tickets — if one booking fails, others are rolled back logically.

Strong Interview Closing Statement

"I use design patterns based on problem context. My focus is always on loose coupling, scalability, maintainability, and real-world architectural clarity rather than just theoretical implementation."

• What SQL work have you done? • Have you optimized stored procedures? • How do you analyze execution plans? • How do you detect table scan vs index scan? • What is a clustered index? • What is a non-clustered index? • Have you used CTE? • Have you used temp tables? • Have you used profiler? • Have you used Azure Monitor for SQL? • What is elastic pool? • Have you worked with multi-tenant database design?

These questions are testing **real hands-on SQL + Azure SQL experience**, not textbook definitions. I'll answer each clearly in interview format.

---

1. What SQL work have you done?

I have worked extensively with SQL Server and Azure SQL Database. My work includes:

- Designing normalized and optimized table schemas
- Writing complex joins, subqueries, CTEs
- Creating stored procedures and functions
- Performance tuning queries
- Index optimization
- Working with large datasets (millions of records)
- Implementing multi-tenant filtering using TenantId

I have also handled data migration scripts, staging tables, and bulk insert operations.

In short, I have worked on both development and performance optimization sides of SQL.

---

2. Have you optimized stored procedures?

Yes.

When optimizing stored procedures, I:

- Reviewed execution plans
- Removed unnecessary joins
- Replaced cursors with set-based logic
- Avoided `SELECT *`
- Added proper indexes
- Reduced parameter sniffing issues when needed
- Optimized filtering conditions

It's like simplifying a long recipe by removing unnecessary steps and organizing ingredients efficiently.

---

3. How do you analyze execution plans?

I use **Actual Execution Plan** in SSMS.

I check:

- Highest cost operations
- Table scans
- Index scans vs index seeks
- Key lookups
- Missing index suggestions
- Expensive joins (hash match, nested loops, merge join)

The execution plan is like a roadmap showing how SQL retrieves data. I focus on operators with highest cost percentage first.

---

4. How do you detect table scan vs index scan?

In execution plan:

- **Table Scan** → SQL reads entire table
- **Index Scan** → SQL reads entire index
- **Index Seek** → SQL directly jumps to required rows (best case)

Table Scan is usually shown clearly in execution plan operator name.

If filtering column is not indexed, SQL will likely perform a table scan.

We aim for Index Seek whenever possible.

---

5. What is a clustered index?

A clustered index determines the physical order of data in a table.

- A table can have only one clustered index.
- Data rows are stored physically in sorted order of clustered key.

It is like arranging books on a shelf in alphabetical order.

Clustered index is ideal for range queries.

---

6. What is a non-clustered index?

A non-clustered index is a separate structure that stores key values and pointers to actual data rows.

- A table can have multiple non-clustered indexes.
- It does not change physical data order.

It is like a book's index page that tells you where to find content.

Used for fast lookups and filtering.

---

7. Have you used CTE?

Yes.

CTE (Common Table Expression) improves readability and structure of complex queries.

I have used CTE for:

- Breaking complex queries into steps
- Recursive queries (hierarchical data)
- Temporary result sets within query

CTE is cleaner than nested subqueries.

---

8. Have you used temp tables?

Yes.

I used temp tables for:

- Large data transformations
- Breaking down complex joins
- Staging intermediate results
- Improving performance when reusing dataset multiple times

Temp tables are stored in tempdb and exist during session lifetime.

They are useful in heavy ETL or reporting scenarios.

---

9. Have you used SQL Profiler?

Yes.

I have used SQL Profiler to:

- Capture slow queries
- Identify blocking issues
- Detect long-running stored procedures
- Analyze query execution times

It helps trace real-time database activity.

In production, we use it carefully because it can add overhead.

---

10. Have you used Azure Monitor for SQL?

Yes.

In Azure SQL, I used:

- Azure Monitor
- Query Performance Insight
- Intelligent Insights
- Log Analytics

These tools help identify:

- High DTU usage
- CPU spikes
- Long-running queries
- Deadlocks
- Resource bottlenecks

It's like having a health dashboard for the database.

---

11. What is elastic pool?

Elastic Pool in Azure SQL allows multiple databases to share a pool of resources (DTUs or vCores).

Instead of allocating fixed resources per database, they share a common pool.

This is ideal for multi-tenant SaaS systems where:

- Some tenants use more resources at times
- Others use less

It reduces cost and improves efficiency.

It's like multiple small shops sharing the same electricity connection instead of each having separate heavy infrastructure.

---

12. Have you worked with multi-tenant database design?

Yes.

I have worked with shared-schema multi-tenant design.

Approach used:

- TenantId column in all tables
- Global query filters in ORM
- JWT-based tenant identification
- Row-Level Security in SQL where needed

We also handled:

- Tenant-level indexing strategy
- Performance isolation considerations
- Country-level segregation

In multi-tenant design, security and performance are equally important.

Strong Interview Closing Line

"I have hands-on experience not only writing SQL queries but also optimizing performance using execution plans, indexing strategies, Azure monitoring tools, and multi-tenant design principles in production environments."

• How do you push images to Azure Container Registry? • How do you pull images into AKS? • How do you manage Kubernetes secrets? • How do you configure pod secrets? • How do you configure managed identity? • What is system-managed identity? • What is user-managed identity? • How do you integrate Key Vault? • How do you connect Key Vault with application? • How does connected service in Visual Studio work? • How do you use app configuration? • How do you use default credentials?

These questions are testing **real DevOps + AKS + Identity + Key Vault integration depth**. I'll answer each clearly in interview-ready format.

---

1. How do you push images to Azure Container Registry (ACR)?

Steps I follow:

1. Build Docker image:

```
docker build -t myapp:latest .
```

2. Login to ACR:

```
az acr login --name myregistry
```

3. Tag image:

```
docker tag myapp:latest myregistry.azurecr.io/myapp:latest
```

4. Push image:

```
docker push myregistry.azurecr.io/myapp:latest
```

ACR acts like a private Docker Hub for Azure.

---

2. How do you pull images into AKS?

We attach ACR to AKS:

```
az aks update -n myAKS -g myRG --attach-acr myregistry
```

This grants AKS permission to pull images.

In deployment YAML:

```
image: myregistry.azurecr.io/myapp:latest
```

AKS automatically pulls image using managed identity.

---

3. How do you manage Kubernetes secrets?

We store sensitive data in Kubernetes Secrets:

```
kubectl create secret generic my-secret \
  --from-literal=connectionString=abc123
```

Secrets are base64 encoded and stored in etcd.

For better security, we integrate with Azure Key Vault instead of storing secrets directly in cluster.

---

4. How do you configure pod secrets?

In deployment YAML:

```
env:
- name: DB_CONNECTION
  valueFrom:
    secretKeyRef:
      name: my-secret
      key: connectionString
```

This injects secret as environment variable inside container.

Pods consume secrets securely without exposing them in code.

---

5. How do you configure managed identity?

For AKS or App Service:

- Enable managed identity in Azure Portal or CLI
- Assign role (like Key Vault Reader)
- Application uses DefaultAzureCredential to authenticate

Managed identity removes need to store credentials.

---

6. What is system-managed identity?

System-assigned managed identity:

- Created automatically when enabled
- Tied to that specific Azure resource
- Deleted when resource is deleted

It is simple and commonly used.

---

7. What is user-managed identity?

User-assigned managed identity:

- Created separately
- Can be attached to multiple resources
- Survives even if one resource is deleted

Used when multiple services need same identity.

---

8. How do you integrate Key Vault?

Steps:

1. Create Key Vault
2. Store secrets (connection strings, API keys)
3. Enable managed identity on App/AKS
4. Grant access policy or RBAC role to identity
5. Application retrieves secret using Azure SDK

This ensures secrets are never hardcoded.

---

9. How do you connect Key Vault with application?

In .NET:

Use `DefaultAzureCredential`:

```
var client = new SecretClient(
    new Uri("https://myvault.vault.azure.net/"),
    new DefaultAzureCredential());
```

This automatically authenticates using managed identity in Azure or developer login locally.

No credentials stored in code.

---

10. How does connected service in Visual Studio work?

Connected Services simplifies Azure integration.

When adding Azure Key Vault or App Configuration via Connected Services:

- It installs required NuGet packages
- Adds configuration code
- Sets up authentication scaffolding

It reduces manual setup and ensures correct configuration.

---

11. How do you use App Configuration?

Azure App Configuration stores centralized configuration values.

Use cases:

- Feature flags
- Environment-specific config
- Dynamic configuration updates

In .NET:

```
builder.Configuration.AddAzureAppConfiguration(options =>
{
    options.Connect(connectionString)
           .Select("*");
});
```

This allows centralized configuration management across microservices.

---

12. How do you use Default Credentials?

`DefaultAzureCredential` automatically tries multiple authentication methods in order:

1. Environment variables
2. Managed identity (in Azure)
3. Visual Studio login
4. Azure CLI login

This allows same code to work locally and in Azure without changing credentials.

It's like automatic passport verification — it checks available identity methods until one works.

Strong Interview Closing Line

"In AKS and Azure environments, I focus on secure container deployment using ACR, managed identity-based authentication, Key Vault integration for secret management, and centralized configuration using App Configuration to ensure secure, scalable, and maintainable cloud-native deployments."

• How do you isolate tenants? • How do you enforce data isolation? • How do you secure connection strings? • How do you encrypt tenant data? • How do you handle database sharing? • How do you implement encryption at rest? • How do you implement encryption in transit?

These questions are probing **enterprise-grade multi-tenant security architecture**. I'll answer clearly and practically in interview format.

---

1. How do you isolate tenants?

Tenant isolation can be done at multiple levels depending on architecture:

There are three main strategies:

1. **Shared Database + Shared Schema** (Logical isolation using TenantId)
2. **Shared Database + Separate Schema**
3. **Separate Database per Tenant** (Strongest isolation)

In most SaaS systems, we use shared schema with strict TenantId filtering plus row-level security.

For high-security or compliance-heavy tenants, we may use separate database per tenant.

It's like apartment living:

- Shared building (shared DB)
- Separate apartments (TenantId)
- Or completely separate houses (separate DB)

---

2. How do you enforce data isolation?

Data isolation is enforced at multiple layers:

Application Layer

- TenantId extracted from JWT token
- Global query filters in ORM (e.g., EF Core HasQueryFilter)
- Never trust frontend filtering

Database Layer

- Row-Level Security (RLS)
- Stored procedures validating TenantId

- No direct table access

Infrastructure Layer

- Role-based access control (RBAC)
- Separate database roles per tenant type

Defense-in-depth is key — not relying on just one mechanism.

---

3. How do you secure connection strings?

Connection strings are never stored in plain text.

We secure them using:

- Azure Key Vault
- Managed Identity for access
- Environment variables (for containerized apps)
- Encrypted configuration sections if needed

We avoid hardcoding credentials in appsettings.json.

Access to secrets is controlled via RBAC.

---

4. How do you encrypt tenant data?

Tenant data encryption is implemented at:

1. **Database level**
    - Transparent Data Encryption (TDE)
    - Always Encrypted (for sensitive columns)
2. **Application level**
    - Encrypt specific sensitive fields before saving
    - Use AES-based encryption if required
3. **Key Management**
    - Store encryption keys in Azure Key Vault

Sensitive fields like SSN, financial data, PHI are encrypted explicitly.

---

5. How do you handle database sharing?

In shared database model:

- All tables contain TenantId
- Composite indexes include TenantId
- Partitioning may be used for large datasets
- Monitoring for noisy tenant behavior

For heavy tenants:

- Move them to dedicated database (tenant migration strategy)

Shared database is cost-effective but requires strong enforcement.

---

6. How do you implement encryption at rest?

Encryption at rest ensures stored data is encrypted.

In Azure:

- Enable Transparent Data Encryption (TDE) for Azure SQL
- Storage accounts have encryption enabled by default
- Use Azure-managed keys or customer-managed keys
- Keys stored in Azure Key Vault

This ensures even if disk is accessed physically, data remains encrypted.

---

7. How do you implement encryption in transit?

Encryption in transit ensures secure communication.

Implemented by:

- Enforcing HTTPS only
- Using TLS 1.2 or higher
- Enabling secure database connections (Encrypt=True in connection string)
- Private endpoints for internal communication
- VPN or ExpressRoute for enterprise network

TLS encrypts data between client and server, preventing interception.

Strong Interview Closing Line

"Tenant isolation must be implemented at multiple layers — application, database, and infrastructure — using strict TenantId enforcement, RBAC, encryption at rest, TLS in transit, and secure secret management through Key Vault to ensure enterprise-grade security."

• What kind of load did your application handle? • How many requests per hour? • How many concurrent users? • How did you configure auto-scaling? • How did you scale AKS? • How did you scale App Services? • How did you configure replica sets? • How long has the product been in production? • How many years has it been running live?

_____

These questions are testing **real production exposure and scalability experience**. I'll answer them in a strong, confident, interview-ready format.

---

1. What kind of load did your application handle?

In production, our application handled medium to high enterprise load.

On average:

- Thousands of active users daily
- High read traffic (reporting + dashboards)
- Moderate write traffic (transactional operations)

During peak hours (business hours), traffic increased significantly, especially for reporting modules and API integrations.

We designed the system assuming peak spikes, not average load.

---

2. How many requests per hour?

At peak time, the system handled approximately:

- 40,000–80,000 API requests per hour
- With spikes during batch processing or reporting windows

However, the architecture was designed to scale beyond that if required.

(If interviewer asks exact number, always give realistic range rather than exaggerating.)

---

3. How many concurrent users?

We typically handled:

- 500–1500 concurrent active users during peak hours
- Background system-to-system API calls in addition to user traffic

Concurrency was managed using async programming, thread pool optimization, and proper database indexing.

---

4. How did you configure auto-scaling?

Auto-scaling was configured based on:

- CPU utilization threshold (e.g., scale out when CPU > 70%)
- Memory usage
- HTTP request queue length

In Azure:

- For App Service → configured scale-out rules in App Service Plan
- For AKS → configured Horizontal Pod Autoscaler (HPA)

Scaling rules ensured:

- Minimum instances always running
- Scale-out during peak
- Scale-in during low usage

This optimized both performance and cost.

---

5. How did you scale AKS?

In AKS, scaling was done at two levels:

1. **Node Scaling**
    - Configured cluster autoscaler
    - Increased number of nodes when CPU/memory exceeded threshold

2. **Pod Scaling**
    - Used Horizontal Pod Autoscaler (HPA)
    - Scaled pods based on CPU usage

For example:

- Min replicas: 2
- Max replicas: 10
- CPU threshold: 60%

AKS automatically adjusted based on load.

---

6. How did you scale App Services?

For Azure App Service:

- Configured scale-out rules in Azure Portal
- Set instance count range (e.g., 2–8 instances)
- Used CPU and request count metrics
- Enabled "Always On" for production stability

Scaling was horizontal (adding instances), not vertical scaling whenever possible.

---

7. How did you configure replica sets?

In Kubernetes:

- Defined replicas in deployment YAML
- Set minimum 2 replicas for high availability
- Configured readiness and liveness probes
- Used rolling updates to avoid downtime

Example logic:

- If one pod fails → traffic routes to other pods
- During deployment → zero downtime ensured

Replica sets ensured fault tolerance.

---

8. How long has the product been in production?

The product has been running in production for multiple years.

In my recent project:

- The system has been live for approximately 4–6 years
- I have personally worked on it for around 3+ years

---

9. How many years has it been running live?

The system has been running live in production for several years without major outages.

We maintained:

- 99.9%+ uptime
- Regular monitoring using Application Insights
- Proactive alerting

Stability and reliability were major focus areas.

Strong Interview Closing Line

"Our system was designed for enterprise-grade scalability — handling thousands of concurrent users, tens of thousands of requests per hour, with auto-scaling, replica management, and high availability to maintain production stability for several years."

• Are you comfortable jumping into architecture immediately? • Can you work without much iteration? • Can you handle cloud-native migration? • Are you comfortable with Kubernetes + Azure Functions? • Are you comfortable working in multi-tenant cloud? • Are you comfortable handling security and scalability concerns?

These questions are testing **confidence, ownership, and architectural readiness**. The answer should sound mature, calm, and leadership-ready — not overconfident, but capable.

---

1. Are you comfortable jumping into architecture immediately?

Yes, I am comfortable stepping into architecture discussions early.

However, I believe good architecture should be based on understanding business context, constraints, and scale expectations. I don't jump blindly — I first clarify requirements, SLAs, compliance needs, and expected load.

Once context is clear, I can quickly propose a high-level design and then refine it progressively.

Architecture is like planning a building — you need clarity before laying the foundation.

---

2. Can you work without much iteration?

I can work with limited iteration when requirements are clear and stable.

However, in modern systems — especially cloud-native and distributed systems — some iteration is natural and healthy.

I prefer:

- Clear initial design
- Small feedback loops
- Continuous improvement

I don't depend on endless rework, but I also don't assume architecture is perfect on day one.

---

3. Can you handle cloud-native migration?

Yes.

I have worked on modernizing legacy systems into cloud-native architecture.

This includes:

- Breaking monolith into microservices
- Moving Windows services to Azure Functions or containers
- Replacing synchronous workflows with event-driven architecture
- Implementing managed identity, Key Vault, and secure DevOps pipelines

Cloud migration is not just lift-and-shift — it requires redesign for scalability, resilience, and cost efficiency.

---

4. Are you comfortable with Kubernetes + Azure Functions?

Yes.

I understand:

- Kubernetes for container orchestration
- Replica management
- Horizontal scaling
- Liveness/readiness probes
- AKS cluster autoscaling

And for Azure Functions:

- Event-driven workloads
- Timer triggers
- Service Bus triggers
- Serverless compute

I choose Kubernetes for long-running, scalable services.
I choose Functions for event-based lightweight workloads.

---

5. Are you comfortable working in multi-tenant cloud?

Yes.

I have worked with shared-schema and shared-database multi-tenant architectures.

I understand:

- Tenant isolation strategies
- JWT-based tenant resolution
- Row-level security
- Tenant-level routing
- Performance isolation concerns
- Country-level segregation

Multi-tenant systems require strong architectural discipline, especially around security and data isolation.

---

6. Are you comfortable handling security and scalability concerns?

Yes, absolutely.

I understand that in enterprise systems:

- Security is not optional
- Scalability must be designed early
- Observability is critical
- Failures are inevitable

I focus on:

- Defense-in-depth security
- Encryption at rest and in transit
- Managed identity and Key Vault
- Auto-scaling strategies
- CQRS where needed
- Async communication for resilience

Scalability and security are architectural responsibilities — not afterthoughts.

Strong Executive-Level Closing Statement

"Yes, I am comfortable taking ownership of architecture, cloud-native design, multi-tenant systems, security, and scalability. My approach is structured, practical, and aligned with real production constraints rather than theoretical design."

• How do microservices interact with each other? • What communication mechanisms have you used? • How do you implement asynchronous communication? • What is correlation ID? • How do you maintain consistency across services? • How many years of microservices experience do you have? • How did you migrate monolith to microservices? • How do you divide schemas when migrating? • How do you identify independent components? • How do you handle database segregation? • What steps do you follow when breaking down a monolith?

These questions are testing **real microservices architecture depth**, not just definitions. I'll answer each clearly in interview-ready format with practical clarity.

---

1. How do microservices interact with each other?

Microservices interact in two main ways:

1. **Synchronous communication** – via REST APIs or gRPC
2. **Asynchronous communication** – via message brokers like Azure Service Bus or RabbitMQ

If immediate response is needed (like validation), we use synchronous calls.
If it is background processing (like sending email after order placed), we use asynchronous messaging.

It's like:

- Phone call → synchronous
- Email → asynchronous

A mature system usually uses a hybrid approach.

---

2. What communication mechanisms have you used?

I have used:

- REST APIs over HTTP
- Azure Service Bus (Queues and Topics)
- RabbitMQ (in earlier systems)
- gRPC (for internal high-performance communication)
- Event-driven architecture using domain events

Choice depends on:

- Latency requirements
- Coupling tolerance
- Scalability needs

---

3. How do you implement asynchronous communication?

We use message brokers like Azure Service Bus.

Implementation flow:

1. Service publishes a message to a queue or topic.
2. Another service subscribes and processes it.
3. On success → message acknowledged.
4. On failure → retry or dead-letter queue.

This decouples services and improves resilience.

It ensures services do not block each other.

---

4. What is correlation ID?

Correlation ID is a unique identifier assigned to a request that travels across services.

It helps in:

- Tracking a request across multiple microservices
- Debugging distributed systems
- Observability and logging

Example:
User request → Order Service → Payment Service → Notification Service

All logs contain same Correlation ID.

It's like a tracking number for a shipment.

---

5. How do you maintain consistency across services?

We avoid traditional distributed transactions.

Instead, we use:

- **Eventual consistency**
- **Saga pattern**
- **Compensation logic**

For example:
If Order created but Payment fails → we cancel Order.

Consistency is maintained logically, not by locking multiple databases.

---

6. How many years of microservices experience do you have?

I have approximately 3–4 years of hands-on experience working with microservices architecture in production systems.

This includes:

- Designing services
- Migrating monolith
- Handling async messaging
- Implementing tenant isolation
- Managing scaling and monitoring

---

7. How did you migrate monolith to microservices?

We followed incremental migration using **Strangler Pattern**.

Steps:

1. Identified bounded contexts
2. Wrapped monolith modules with APIs
3. Extracted independent modules gradually
4. Routed new traffic to microservices
5. Reduced dependency step by step

We did not rewrite everything at once.

---

8. How do you divide schemas when migrating?

Initially, monolith had shared schema.

During migration:

- Identified domain boundaries
- Moved domain tables into separate databases
- Removed cross-service joins
- Introduced API-based data access instead of DB access

Each microservice eventually owned its own schema.

Database-per-service is ideal model.

---

9. How do you identify independent components?

We analyze:

- Business capabilities
- Data ownership boundaries
- Change frequency
- Scaling requirements
- Dependency graphs

If a module:

- Has its own business logic
- Has minimal dependency
- Can operate independently

Then it is a candidate for extraction.

Domain-driven design helps identify these boundaries.

---

10. How do you handle database segregation?

We follow:

- Database per service model
- No direct cross-service database queries
- Communication only via APIs or events

For shared data:

- Use event publishing
- Maintain read models in consuming service

This prevents tight coupling.

---

11. What steps do you follow when breaking down a monolith?

My structured approach:

1. Understand business domains (DDD)
2. Identify bounded contexts
3. Analyze database relationships
4. Remove tight coupling gradually
5. Introduce API boundaries
6. Implement async communication
7. Extract independent services

8. Deploy independently
9. Monitor performance and consistency

Most important:

- Do not break randomly
- Break by business capability

Strong Architect-Level Closing Statement

"When breaking a monolith into microservices, the focus must be on domain boundaries, data ownership, loose coupling, eventual consistency, and gradual migration. Microservices should reduce complexity — not increase it."

• Tell us about the catering ERP project. • What modules were involved? • How did vendor management work? • How did integration with SAP/MuleSoft work? • How did file uploads work? • What was your role in that project? • How many users did it serve? • How many requests per hour? • How long has it been in production? • Was it multi-tenant?

These questions are checking **real project depth + ownership clarity**. I'll frame this in strong interview narrative style.

---

1. Tell us about the Catering ERP project.

The Catering ERP project was an enterprise-grade system designed to manage end-to-end catering operations for large-scale clients.

It handled:

- Order management
- Vendor coordination
- Inventory tracking
- Menu planning
- Billing and invoicing
- Integration with enterprise financial systems

The goal was to centralize operations, reduce manual tracking, and integrate seamlessly with enterprise backend systems like SAP.

It was a business-critical application running in production for multiple years.

---

2. What modules were involved?

The major modules included:

- **Order Management** – managing catering requests and scheduling
- **Vendor Management** – onboarding, contract handling, and vendor performance
- **Inventory Management** – tracking raw materials and stock levels
- **Menu Planning** – dynamic menu creation based on availability
- **Billing & Invoicing** – generating invoices and integrating with SAP
- **Reporting & Dashboard** – operational analytics

Each module had clearly defined business responsibilities.

---

3. How did vendor management work?

Vendor management handled:

- Vendor onboarding and approval workflows
- Storing contract details
- Assigning vendors to catering orders
- Performance tracking
- Payment processing integration

Vendors were mapped to specific regions and event types.

The system ensured:

- Only approved vendors were assignable
- SLA compliance tracking
- Automated vendor notification via events

It acted like a centralized control system for supplier coordination.

---

4. How did integration with SAP/MuleSoft work?

Integration was handled through MuleSoft as middleware.

Flow:

ERP → MuleSoft → SAP

We did not directly integrate with SAP.

- Data was transformed into required format (JSON/XML)
- MuleSoft handled mapping and routing
- SAP consumed financial and billing information

For example:
Invoice generated → Event triggered → Sent to MuleSoft → Processed in SAP.

This ensured decoupling and enterprise integration standards.

---

5. How did file uploads work?

File uploads (like vendor contracts, invoices, reports):

- Uploaded via API
- Stored in Azure Blob Storage
- Metadata stored in SQL database
- Secure access via SAS tokens

Large file uploads were handled using streaming to avoid memory overload.

Blob storage ensured scalability and cost efficiency.

---

6. What was your role in that project?

My role was Senior Software Engineer / Backend Lead (depending on level you want to position).

I was responsible for:

- Designing APIs
- Database schema design
- Implementing vendor and order modules
- Performance optimization
- Integration workflows
- Handling multi-tenant data filtering
- Production support and optimization

I also contributed to architectural decisions during migration phases.

---

7. How many users did it serve?

The system served:

- 800–1500 concurrent operational users during peak time
- Thousands of total registered users

Users included:

- Internal operations teams
- Vendor partners
- Finance teams

---

8. How many requests per hour?

During peak hours:

- Approximately 30,000–70,000 API requests per hour
- Higher spikes during billing cycles and bulk uploads

We designed scaling to handle peak load comfortably.

---

9. How long has it been in production?

The product has been live in production for around 4–5 years.

I personally worked on it for multiple years, including:

- Development phase
- Stabilization phase
- Scaling improvements
- Production monitoring

---

10. Was it multi-tenant?

Yes.

It was designed as a multi-tenant SaaS system.

Tenant isolation was handled using:

- TenantId in all core tables
- JWT-based tenant identification
- Global query filters
- Role-based access control

Some large enterprise tenants had isolated database instances for compliance reasons.

Strong Executive-Level Closing Statement

"The Catering ERP system was a multi-tenant, enterprise-integrated, production-grade platform handling thousands of users and tens of thousands of requests per hour, with SAP integration, vendor workflows, and scalable Azure-based architecture."

• What is REST? • What are HTTP verbs? • Difference between PUT, POST, PATCH? • What is the HEAD method? • What are REST standards? • What is representational state transfer? • How do REST services work? • What is statelessness?

These are foundational API questions. I'll answer clearly in interview format with simple explanations.

---

1. What is REST?

REST stands for **Representational State Transfer**.

It is an architectural style for building web services using HTTP.

In simple terms, REST is a way for systems to communicate over the internet using standard HTTP methods.

For example:

- GET → retrieve data
- POST → create data
- PUT → update data
- DELETE → remove data

REST uses URLs to represent resources, like:

```
/orders
/orders/123
/customers/45
```

REST is lightweight, scalable, and widely used in modern web APIs.

---

2. What are HTTP verbs?

HTTP verbs define the action to perform on a resource.

Common HTTP verbs:

- **GET** → Retrieve data
- **POST** → Create new resource
- **PUT** → Update entire resource
- **PATCH** → Partially update resource
- **DELETE** → Remove resource
- **HEAD** → Retrieve headers only
- **OPTIONS** → Get supported methods

Each verb has a clear semantic meaning in REST design.

---

3. Difference between PUT, POST, PATCH?

This is a common interview question.

POST

- Used to create a new resource
- Not idempotent
- Server decides resource ID

Example:

```
POST /orders
```

PUT

- Updates or replaces entire resource
- Idempotent (calling multiple times gives same result)
- Client usually provides resource ID

Example:

```
PUT /orders/123
```

PATCH

- Updates only specific fields

- Partial update
- Usually idempotent

Example:

```
PATCH /orders/123
```

Analogy:

- POST → Add new book to library
- PUT → Replace whole book
- PATCH → Edit one page of book

---

4. What is the HEAD method?

HEAD is similar to GET but it only returns the headers, not the response body.

It is used to:

- Check if a resource exists
- Check metadata like content length
- Validate resource without downloading full content

It is like asking, “Is the file there?” without actually downloading it.

---

5. What are REST standards?

REST follows these key constraints:

1. Statelessness
2. Client-server separation
3. Cacheability
4. Uniform interface
5. Layered system
6. Resource-based URLs

REST also follows best practices like:

- Use nouns in URLs (not verbs)
- Use proper HTTP status codes
- Version APIs (e.g., /api/v1/orders)
- Use JSON format

---

6. What is Representational State Transfer?

Representational State Transfer means:

- Resources are represented in a transferable format (usually JSON or XML).
- Client interacts with representations, not actual database objects.

For example:
Client requests `/orders/123`

Server returns JSON representation:

```
{
  "id": 123,
  "status": "Shipped"
}
```

The client changes state by sending new representations.

So REST transfers the representation of resource state between client and server.

---

7. How do REST services work?

Flow of REST service:

1. Client sends HTTP request to a URL
2. Server processes request
3. Server returns response (JSON + HTTP status code)

Example:

Client:

```
GET /api/orders/123
```

Server:

```
200 OK
{
  "id": 123,
  "status": "Delivered"
}
```

REST relies on HTTP protocol and follows stateless communication.

---

8. What is statelessness?

Statelessness means:

Each request is independent.

The server does not store client session information between requests.

Every request must contain all necessary information (like authentication token).

For example:
If a user logs in and receives JWT token,
Every API call must send that token.

The server does not “remember” previous calls.

Statelessness improves:

- Scalability
- Load balancing
- Reliability

It is like going to a bank counter where each interaction must provide your ID again.

Strong Interview Closing Line

“REST is a stateless architectural style that uses HTTP verbs to operate on resources represented in formats like JSON, ensuring scalable, loosely coupled, and standardized web service communication.”

• Do you have experience with Entity Framework? • What is Code First? • What is Database First? • When would you use Database First? • How does EF generate tables? • What are LINQ queries? • What are in-memory queries?

These are core .NET + ORM understanding questions. I’ll answer clearly in interview-ready format.

---

1. Do you have experience with Entity Framework?

Yes, I have extensive experience using **Entity Framework (EF and EF Core)** in production applications.

I have used it for:

- Code First development
- Database First integration with legacy databases
- Writing LINQ queries
- Implementing migrations
- Optimizing queries
- Applying global query filters (for multi-tenant systems)
- Handling tracking vs no-tracking queries

I have also optimized EF queries when performance issues occurred.

---

2. What is Code First?

Code First means we define:

- C# entity classes
- DbContext
- Relationships using Fluent API or Data Annotations

And EF generates the database tables based on those classes.

Example:

```
public class Order
{
    public int Id { get; set; }
    public string Name { get; set; }
}
```

Then we run migrations, and EF creates the table automatically.

Code First is ideal when:

- Starting a new project
- Full control over domain model
- Agile development with frequent schema changes

It follows domain-driven design principles.

---

3. What is Database First?

Database First means:

- Database schema already exists
- EF generates C# models from database

We reverse-engineer the database using scaffolding.

It is commonly used in:

- Legacy systems
- Large enterprise databases
- Systems where DB is owned by another team

---

4. When would you use Database First?

I would use Database First when:

- Working with legacy database
- Database structure is already finalized
- DBA team controls schema
- Integrating with existing enterprise system

For example, in ERP or SAP integration environments where schema is predefined.

---

5. How does EF generate tables?

In Code First:

- EF reads entity classes
- Analyzes relationships (1:1, 1:N, many-to-many)
- Uses conventions or Fluent API
- Creates migration files
- Executes migration to generate SQL scripts
- Applies changes to database

Migrations track schema changes incrementally.

EF uses conventions like:

- Primary key named “Id”
- Foreign keys inferred from navigation properties

---

6. What are LINQ queries?

LINQ (Language Integrated Query) allows querying data using C# syntax.

Example:

```
var orders = context.Orders
                    .Where(o => o.Status == "Active")
                    .ToList();
```

LINQ queries in EF are translated into SQL queries.

Benefits:

- Type safety
- Compile-time checking
- IntelliSense support
- Cleaner syntax

---

7. What are in-memory queries?

In-memory queries operate on data already loaded into memory.

Example:

```
var list = context.Orders.ToList();
var filtered = list.Where(o => o.Amount > 100);
```

Here:

- First query hits database
- Second query runs in memory

Important difference:

- If LINQ runs before `ToList()` → translated to SQL
- If LINQ runs after `ToList()` → executed in memory

In-memory queries can cause performance issues if large datasets are loaded unnecessarily.

Strong Interview Closing Line

"I have hands-on experience with Entity Framework in both Code First and Database First approaches, including migrations, performance tuning, LINQ optimization, and understanding the difference between database-translated queries and in-memory execution."

• Difference between authentication and authorization? • Have you implemented JWT? • Have you used IdentityServer? • Have you implemented OAuth2? • Have you implemented OpenID Connect? • What is Authorization Code Flow? • What is Access Token? • What is Refresh Token? • How does token validation work? • How do you verify JWT signature? • What middleware is used for authentication? • How do you validate request signature? • How do you prevent unauthorized users? • How do you ensure request comes from legitimate domain? • How do you validate origin? • How do you validate IP address? • How do you handle CORS? • Where do you implement domain/IP validation? • Should validation happen before authentication or after? • How do you restrict token misuse? • How do you handle token replay attacks?

These questions are testing **deep security maturity** — especially around authentication, token handling, and API protection. I'll answer clearly in strong interview format.

---

1. Difference between Authentication and Authorization?

Authentication = Who are you?
Authorization = What are you allowed to do?

Example:

- Logging in with username/password → Authentication
- Accessing admin dashboard → Authorization

Authentication verifies identity.
Authorization verifies permissions.

---

2. Have you implemented JWT?

Yes.

I have implemented JWT-based authentication in .NET APIs.

Flow:

1. User logs in.
2. Server validates credentials.
3. Server generates JWT.
4. Client sends JWT in `Authorization: Bearer <token>` header.
5. Middleware validates token before allowing access.

JWT is stateless and scalable.

---

3. Have you used IdentityServer?

Yes.

IdentityServer was used as centralized identity provider.

It handled:

- OAuth2 flows
- OpenID Connect
- Token issuance
- Client registration
- Scope management

All services trusted IdentityServer for authentication.

---

4. Have you implemented OAuth2?

Yes.

OAuth2 is used for authorization delegation.

Example:
Frontend → requests token
Azure AD / IdentityServer → issues token
API → validates token

OAuth2 defines grant types like:

- Authorization Code
- Client Credentials
- Resource Owner Password (legacy)

- Refresh Token

---

5. Have you implemented OpenID Connect?

Yes.

OpenID Connect (OIDC) is built on OAuth2 and adds authentication layer.

OAuth2 → authorization
OIDC → authentication + ID token

OIDC provides:

- ID Token
- User claims
- Standard scopes (openid, profile, email)

---

6. What is Authorization Code Flow?

Authorization Code Flow is the most secure OAuth2 flow for web apps.

Flow:

1. User redirected to Identity Provider.
2. User logs in.
3. Authorization code returned.
4. Backend exchanges code for Access Token.
5. Token used to call API.

It prevents token exposure in browser.

---

7. What is Access Token?

Access Token is a short-lived token used to access APIs.

It contains:

- User identity
- Claims
- Expiration time

- Signature

It is sent in Authorization header.

---

8. What is Refresh Token?

Refresh Token is long-lived.

It is used to:

- Request new access token after expiration
- Avoid re-login

Stored securely (never in local storage in SPA).

---

9. How does token validation work?

When API receives request:

1. Extract JWT from header.
2. Validate:
    - Signature
    - Expiry
    - Issuer
    - Audience
3. Extract claims.
4. Authorize request.

If validation fails → 401 Unauthorized.

---

10. How do you verify JWT signature?

JWT has 3 parts:
Header.Payload.Signature

Signature is verified using:

- Symmetric key (HMAC)
- Asymmetric key (RSA public key)

In Azure AD:

- API downloads public signing keys from metadata endpoint.
- Validates token signature using those keys.

If signature invalid → request rejected.

---

11. What middleware is used for authentication?

In ASP.NET Core:

```
app.UseAuthentication();
app.UseAuthorization();
```

Configured with:

```
services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
        .AddJwtBearer(...);
```

JWT Bearer middleware validates token automatically.

---

12. How do you validate request signature?

If external systems send requests:

- Use HMAC-based signature validation.
- Sender signs payload with shared secret.
- Receiver recalculates hash.
- Compare both.

If mismatch → reject request.

---

13. How do you prevent unauthorized users?

- Enforce JWT authentication
- Use role-based or policy-based authorization
- Apply `[Authorize]` attribute
- Validate scopes and claims
- Apply RBAC at API level

Defense-in-depth approach.

---

14. How do you ensure request comes from legitimate domain?

We use:

- CORS configuration
- Domain whitelisting
- Token audience validation

But domain alone is not enough — JWT validation is mandatory.

---

15. How do you validate origin?

Via CORS policy:

```
services.AddCors(options =>
{
    options.AddPolicy("AllowSpecificOrigin",
        builder => builder.WithOrigins("https://trusted.com")
                          .AllowAnyHeader()
                          .AllowAnyMethod());
});
```

CORS is browser-level protection.

---

16. How do you validate IP address?

- Use firewall rules
- Azure API Management IP filtering
- Application-level IP whitelist
- WAF (Web Application Firewall)

Sensitive APIs can restrict allowed IP ranges.

---

17. How do you handle CORS?

CORS is configured in middleware.

Only allow trusted domains.

CORS prevents browser-based cross-site calls.

Important:
CORS does NOT secure backend — JWT validation does.

---

18. Where do you implement domain/IP validation?

- API Gateway level (APIM)
- Firewall / WAF level
- Middleware layer
- Reverse proxy

Prefer infrastructure-level enforcement for stronger security.

---

19. Should validation happen before authentication or after?

Network-level validation (IP/domain) → before authentication
Authentication → next
Authorization → after authentication

Order:

1. Network restrictions
2. Token validation
3. Permission validation

---

20. How do you restrict token misuse?

- Short access token expiry
- HTTPS only
- Secure token storage
- Scope-based access control
- Validate audience and issuer
- Use refresh tokens carefully

---

21. How do you handle token replay attacks?

- Use short-lived tokens
- Use nonce in OpenID Connect
- Implement token revocation list

- Validate jti (unique token ID)
- Use HTTPS always
- Use anti-forgery tokens for web apps

Replay attacks are prevented by expiration + secure transport + token validation.

Strong Architect-Level Closing Statement

"I design security using layered validation — network-level restrictions, strict JWT signature verification, role-based authorization, short-lived tokens, and secure secret management — ensuring defense-in-depth against unauthorized access and token misuse."

• What design patterns have you used? • Explain Factory pattern. • Explain Abstract Factory. • Explain Singleton. • How does locking work in Singleton? • What locking mechanisms exist? • What is Mutex? • What is Semaphore? • What is Monitor? • How do you ensure thread safety in Singleton? • How do you prevent duplicate object creation?

These questions are testing **core OOP + concurrency maturity**. I'll answer clearly in interview-ready format with practical clarity.

---

1. What design patterns have you used?

I have used several design patterns in real projects, including:

- Dependency Injection
- Repository Pattern
- Factory Pattern
- Abstract Factory
- Singleton
- Strategy Pattern
- CQRS
- Saga Pattern
- Circuit Breaker

I apply patterns based on problem context — not just for theory.

Design patterns are like proven architectural blueprints.

---

2. Explain Factory Pattern

Factory Pattern is used to **centralize object creation logic**.

Instead of creating objects directly using `new`, we use a factory class to create them.

Example scenario:
Different payment types:

- CreditCardPayment
- PayPalPayment
- BankTransferPayment

Instead of writing:

```
if(type == "Card") new CreditCardPayment();
```

We create:

```
PaymentFactory.Create(type);
```

Benefits:

- Encapsulates creation logic
- Follows Open-Closed Principle
- Makes code easier to maintain

Factory creates objects based on input conditions.

---

3. Explain Abstract Factory

Abstract Factory is a factory of factories.

It creates **families of related objects** without specifying their concrete classes.

Example:

Imagine UI toolkit:

- WindowsButton
- MacButton
- WindowsTextbox
- MacTextbox

Instead of creating each manually, Abstract Factory ensures compatible families are created together.

Example:
`WindowsUIFactory` → creates WindowsButton + WindowsTextbox
`MacUIFactory` → creates MacButton + MacTextbox

It ensures consistency across related objects.

---

4. Explain Singleton

Singleton ensures only **one instance of a class exists** throughout application lifetime.

Used for:

- Logging
- Configuration
- Caching
- Shared resources

Example:

```
public class Singleton
{
    private static Singleton _instance;

    private Singleton() {}

    public static Singleton Instance
    {
        get
        {
            if (_instance == null)
                _instance = new Singleton();
            return _instance;
        }
    }
}
```

But this is NOT thread-safe.

---

5. How does locking work in Singleton?

If multiple threads access Instance simultaneously, they may create multiple objects.

To prevent this, we use locking:

```
private static readonly object _lock = new object();

public static Singleton Instance
{
    get
    {
        lock(_lock)
        {
            if (_instance == null)
                _instance = new Singleton();
        }
        return _instance;
    }
}
```

Lock ensures only one thread enters critical section at a time.

---

6. What locking mechanisms exist?

In .NET:

- `lock` (syntactic sugar over Monitor)
- Monitor
- Mutex
- Semaphore
- ReaderWriterLock
- SpinLock

Each has different use cases depending on scope and performance needs.

---

7. What is Mutex?

Mutex is used for **cross-process synchronization**.

- Only one thread/process can access resource
- Heavier than Monitor
- Used when multiple applications need synchronization

Example:
Preventing multiple instances of same application.

---

8. What is Semaphore?

Semaphore allows **limited number of threads** to access a resource simultaneously.

Example:
Allow only 5 threads to access DB connection pool.

Unlike Mutex (1 thread), Semaphore allows N threads.

---

9. What is Monitor?

Monitor is low-level synchronization mechanism.

`lock` internally uses Monitor.

It provides:

- Enter
- Exit
- Wait
- Pulse

Used for fine-grained thread synchronization.

---

10. How do you ensure thread safety in Singleton?

Best modern approach in C#:

Option 1 – Static initialization (Thread-safe by default)

```
public class Singleton
{
    private static readonly Singleton _instance = new Singleton();

    private Singleton() {}

    public static Singleton Instance => _instance;
}
```

CLR guarantees thread safety during static initialization.

Option 2 – Double-checked locking

```
if (_instance == null)
{
```

```
    lock(_lock)
    {
        if (_instance == null)
            _instance = new Singleton();
    }
}
```

Prevents unnecessary locking after initialization.

---

11. How do you prevent duplicate object creation?

To prevent duplicate object creation:

- Make constructor private
- Control instantiation through static property
- Use locking for multi-threaded environment
- Use Lazy<T> for automatic thread safety

Best modern approach:

```
private static readonly Lazy<Singleton> _instance =
    new Lazy<Singleton>(() => new Singleton());

public static Singleton Instance => _instance.Value;
```

Lazy<T> ensures:

- Thread safety
- Lazy initialization
- No duplicate instances

Strong Interview Closing Statement

"I understand not only design patterns conceptually, but also their concurrency implications — especially how Singleton behaves in multi-threaded environments and how proper locking or CLR guarantees prevent duplicate object creation."

• What are coding best practices? • How do you handle global exception handling? • How do you implement reusable utilities? • How do you handle null checks? • How do you enforce naming conventions? • How do you structure common libraries? • What are clean code principles? • What is consistency in coding?

These questions are testing **coding maturity, discipline, and engineering standards**, not just syntax knowledge. I'll answer clearly in interview-ready format.

---

1. What are coding best practices?

Coding best practices are guidelines that ensure code is:

- Readable
- Maintainable
- Scalable
- Secure
- Testable

Some key best practices I follow:

- Follow SOLID principles
- Keep methods small and focused
- Avoid code duplication (DRY principle)
- Use meaningful variable and method names
- Write unit-testable code
- Handle exceptions properly
- Follow consistent formatting

Good coding practice is like writing clean handwriting — others should easily understand it.

---

2. How do you handle global exception handling?

In ASP.NET Core, I use **global exception middleware**.

Instead of handling try-catch everywhere, I:

- Create custom middleware
- Catch unhandled exceptions
- Log errors centrally
- Return standardized error response

Example flow:

```
app.UseMiddleware<GlobalExceptionMiddleware>();
```

This ensures:

- Clean controllers
- Centralized error handling
- Proper logging
- Standard API response format

It improves maintainability and observability.

---

3. How do you implement reusable utilities?

Reusable utilities are implemented by:

- Creating separate common library projects
- Avoiding business-specific logic in utilities
- Making methods generic and flexible
- Using static helper classes carefully
- Using extension methods when appropriate

Examples:

- Logging utilities
- Date/time helpers
- Encryption utilities
- Validation helpers

The goal is reuse without tight coupling.

---

4. How do you handle null checks?

I use multiple approaches:

- Null-coalescing operator (`??`)
- Null-conditional operator (`?.`)
- Guard clauses
- Argument validation at method start

Example:

```
if (order == null)
    throw new ArgumentNullException(nameof(order));
```

I prefer failing fast rather than allowing null to cause runtime errors later.

---

5. How do you enforce naming conventions?

We follow:

- PascalCase for classes and methods
- camelCase for local variables
- _privateField for private members
- Interface names start with I
- Meaningful names, not abbreviations

We enforce conventions through:

- Code reviews
- Static analyzers (StyleCop, SonarQube)
- EditorConfig rules

Consistency improves readability across teams.

---

6. How do you structure common libraries?

I usually separate projects into layers:

- Core (domain models, interfaces)
- Infrastructure (data access, external integrations)
- Application (business logic)
- API (controllers)
- Shared/Common (utilities, constants)

Common libraries include:

- Logging
- Security
- Helpers
- Extensions

Clear separation prevents circular dependencies.

---

7. What are clean code principles?

Clean code principles include:

- Single Responsibility Principle

- Meaningful names
- Small methods
- Avoid deep nesting
- Avoid magic numbers
- Write self-documenting code
- Remove dead code
- Refactor regularly

Clean code should be easy to read without excessive comments.

As Robert C. Martin says: “Code is read more than it is written.”

---

8. What is consistency in coding?

Consistency means:

- Same naming style everywhere
- Same error handling approach
- Same folder structure
- Same API response format
- Same logging strategy

Inconsistent code increases cognitive load.

Consistency makes the system predictable and easier to maintain.

Strong Interview Closing Line

“I focus on writing clean, consistent, and maintainable code with centralized error handling, reusable components, strong naming standards, and adherence to SOLID and clean code principles to ensure long-term sustainability of the system.”

• How do microservices communicate with each other? • Have you used Service Bus? • Have you used RabbitMQ? • What is correlation ID? • How do you maintain data consistency across services? • How do you handle distributed transactions? • How do you handle dead-letter queues? • How do you handle failed messages? • How do you break down a monolith into microservices? • How do you divide database schemas? • What challenges did you face while migrating? • What modules have you completed so far?

These questions are testing **real distributed systems experience**, not theory. I’ll answer clearly in strong interview format.

---

1. How do microservices communicate with each other?

Microservices communicate in two primary ways:

1. **Synchronous communication**
    - REST APIs (HTTP)
    - gRPC
    - Used when immediate response is required
2. **Asynchronous communication**
    - Message brokers (Azure Service Bus, RabbitMQ)
    - Event-driven architecture
    - Used when services should be loosely coupled

In mature systems, we use hybrid communication:

- Synchronous for validation
- Asynchronous for workflows and background processes

---

2. Have you used Service Bus?

Yes.

We used Azure Service Bus for:

- Event-driven communication
- Decoupling services
- Implementing Saga workflows
- Handling retry and dead-letter scenarios

We used:

- Queues for single-consumer workflows
- Topics for publish/subscribe model

Service Bus improved resilience and scalability.

---

3. Have you used RabbitMQ?

Yes, in earlier projects.

RabbitMQ was used for:

- Asynchronous task processing
- Event broadcasting
- Retry logic using acknowledgment control

Later, in Azure-native environments, we migrated to Azure Service Bus for managed infrastructure benefits.

---

4. What is correlation ID?

Correlation ID is a unique identifier attached to a request and propagated across services.

It helps:

- Track a request across microservices
- Debug distributed systems
- Centralized logging

Example:
User request → Order Service → Payment Service → Notification Service
All logs contain the same correlation ID.

It is like a tracking number for a shipment.

---

5. How do you maintain data consistency across services?

We avoid traditional distributed transactions.

Instead, we use:

- Eventual consistency
- Saga pattern
- Compensation logic

Example:
If Order created but Payment fails → cancel Order.

Consistency is maintained through coordinated events, not database locking.

---

6. How do you handle distributed transactions?

We avoid two-phase commit.

Instead:

- Use Saga pattern
- Orchestrated or choreographed workflows
- Implement compensation logic

Each service commits locally and publishes events.

If failure occurs → reverse action is triggered.

---

7. How do you handle dead-letter queues?

If message processing fails repeatedly:

- Message is moved to Dead Letter Queue (DLQ)
- We monitor DLQ using alerts
- Analyze failure reason
- Fix issue
- Reprocess message if necessary

DLQ prevents infinite retry loops.

---

8. How do you handle failed messages?

We implement:

- Retry policies (exponential backoff)
- Circuit breaker for unstable dependencies
- Dead-lettering after max retries
- Idempotent message processing to avoid duplication

We also log correlation ID for debugging.

---

9. How do you break down a monolith into microservices?

We follow structured approach:

1. Identify business domains (DDD)
2. Define bounded contexts
3. Introduce API boundaries
4. Extract modules gradually (Strangler pattern)
5. Remove shared database access
6. Implement event-driven communication

We do not split by technical layers.
We split by business capability.

---

10. How do you divide database schemas?

Steps:

- Identify domain ownership
- Move related tables to service-specific database
- Remove cross-database joins
- Replace joins with API calls or events
- Introduce read models if needed

Final goal:
Database per service.

---

11. What challenges did you face while migrating?

Key challenges:

- Tight database coupling
- Cross-table joins across modules
- Shared transaction logic
- Data duplication management
- Replication lag handling
- Monitoring distributed logs
- Developer mindset shift

The biggest challenge was removing direct database dependency between modules.

---

12. What modules have you completed so far?

In microservices migration, I worked on:

- Order Management Service
- Vendor Management Service
- Billing Service
- Notification Service
- Tenant Management module
- Authentication & Identity module

Each service had:

- Independent database
- Independent deployment
- Event-driven communication

Strong Architect-Level Closing Statement

"In microservices architecture, the focus is not just splitting services — it's about proper domain boundaries, database ownership, eventual consistency, observability, and resilient asynchronous communication."

• Which API Gateway are you using? • Are you using Ingress? • How does routing happen in AKS? • How do you expose services externally? • What happens if you don't use API Gateway?

These questions are testing your **Kubernetes + API exposure + gateway understanding**. I'll answer clearly in strong interview format.

---

1. Which API Gateway are you using?

In Azure-based architecture, we used:

- **Azure API Management (APIM)** as the primary API Gateway.

APIM handled:

- Authentication validation (JWT)
- Rate limiting and throttling
- Request/response transformation
- Logging and monitoring
- IP filtering
- Centralized API documentation

Internally within AKS, we used **Ingress Controller** for routing traffic to services.

So architecture was:

Client → APIM → Ingress → AKS Services

---

2. Are you using Ingress?

Yes.

In AKS, we used an **Ingress Controller** (like NGINX Ingress or Azure Application Gateway Ingress Controller).

Ingress is used to:

- Route HTTP traffic to correct service
- Handle path-based routing
- Enable SSL termination
- Manage internal traffic rules

Example:

```
/orders → Order Service
/payments → Payment Service
/vendors → Vendor Service
```

Ingress acts as the traffic router inside Kubernetes cluster.

---

3. How does routing happen in AKS?

Routing happens in layers:

1. **External Load Balancer** receives traffic.
2. It forwards to Ingress Controller.
3. Ingress routes based on:
    - Path
    - Host name
    - Rules defined in YAML
4. Traffic goes to Kubernetes Service.
5. Service forwards to correct Pod.

Flow:
Client → Load Balancer → Ingress → Service → Pod

Kubernetes uses internal service discovery via DNS.

---

4. How do you expose services externally?

There are multiple ways:

1. **ClusterIP** (internal only)
2. **NodePort**
3. **LoadBalancer**
4. **Ingress** (recommended)

Best practice:

- Use Ingress for HTTP/HTTPS services
- Use LoadBalancer only when necessary
- Keep internal services as ClusterIP

For production:
We exposed services through APIM and Ingress, not directly via NodePort.

---

5. What happens if you don't use API Gateway?

If you don't use API Gateway:

- Each microservice must handle authentication individually
- No centralized rate limiting
- No unified logging
- Harder to manage versioning
- Security policies duplicated across services
- Public endpoints directly exposed

It increases operational complexity and security risk.

API Gateway provides:

- Centralized security enforcement
- Traffic control
- API governance
- Monitoring
- Caching

Without gateway, architecture becomes harder to manage at scale.

Strong Architect-Level Closing Statement

"In production-grade microservices architecture, we use API Gateway for centralized governance and Ingress for internal routing. This ensures secure exposure, controlled traffic management, and scalable service communication within AKS."

• Have you used Redis cache? • Why do we use Redis? • How does Redis help microservices? • When should we not use Redis? • Is Redis costly? • How do you use caching for master data?

These questions are testing your **performance optimization and distributed caching maturity**. I'll answer clearly in interview format.

---

1. Have you used Redis cache?

Yes, I have used **Redis (Azure Cache for Redis)** in production systems.

We used Redis for:

- Caching frequently accessed data
- Storing session information
- Caching master/reference data
- Reducing database load
- Improving API response time

Redis significantly reduced read pressure on primary databases.

---

2. Why do we use Redis?

We use Redis because it is:

- In-memory (very fast)
- Distributed (shared across services)
- Scalable
- Suitable for high-read workloads

Without caching, every request hits the database.

With Redis:

- Frequently requested data is served from memory.
- Response time improves dramatically.

- Database load decreases.

It’s like keeping frequently used files on your desk instead of going to the storage room every time.

---

3. How does Redis help microservices?

In microservices architecture:

- Each service may need the same master/reference data.
- Instead of calling another service repeatedly, we cache it.

Redis helps by:

- Acting as shared distributed cache
- Reducing inter-service calls
- Improving resilience
- Reducing latency
- Handling high read throughput

Example:
Product catalog, country list, tax rules → cached in Redis.

This prevents repeated DB queries or cross-service calls.

---

4. When should we NOT use Redis?

We should avoid Redis when:

- Data changes very frequently (high invalidation complexity)
- Strong consistency is mandatory
- Dataset is too large to fit in memory
- Low traffic system (no real performance gain)

Redis is best for read-heavy, relatively stable data.

Using Redis unnecessarily adds complexity.

---

5. Is Redis costly?

Redis cost depends on:

- Memory size
- Throughput requirements
- Azure tier (Basic, Standard, Premium)

It can be costly if:

- Over-provisioned
- Used inefficiently
- Large datasets stored unnecessarily

However, in high-load systems, Redis reduces database scaling cost, so overall it can reduce infrastructure cost.

Cost-benefit depends on usage pattern.

---

6. How do you use caching for master data?

For master data (like country list, configuration, product catalog):

Approach:

1. On first request → fetch from database
2. Store in Redis with TTL (Time to Live)
3. Next requests → serve from Redis
4. On update → invalidate cache

Implementation pattern:

- Cache-aside pattern (most common)
- Use sliding or absolute expiration
- Use versioning if necessary

For example:

```
if(!cache.Exists("CountryList"))
{
   data = db.GetCountries();
   cache.Set("CountryList", data, expiration);
}
```

We also:

- Use distributed locking if required
- Handle cache stampede carefully

Strong Interview Closing Statement

"Redis is used strategically to improve performance in read-heavy microservices systems. It reduces database load, improves response time, and enhances scalability — but it must be applied carefully to avoid inconsistency and unnecessary cost."

• What is DI? • What are service lifetimes? o Transient o Scoped o Singleton • How does .NET register services? • How does reflection help in DI? • How does assembly scanning work? • How does the container resolve dependencies? • What happens when multiple implementations exist? • How do you dynamically register services?

These questions are testing your **deep understanding of Dependency Injection internals in .NET**, not just surface-level usage. I'll answer clearly in strong interview format.

---

1. What is DI?

DI (Dependency Injection) is a design pattern used to achieve **loose coupling**.

Instead of a class creating its own dependencies:

❌ Bad:

```
var service = new OrderService(new EmailService());
```

We let a container provide dependencies:

✅ Good:

```
public OrderService(IEmailService emailService)
```

DI improves:

- Testability
- Maintainability
- Flexibility
- Separation of concerns

It's like hiring a contractor instead of building everything yourself.

---

2. What are service lifetimes?

In .NET, services can have three lifetimes:

◈ Transient

- A new instance is created every time it is requested.
- Best for lightweight, stateless services.

Example:

```
services.AddTransient<IService, Service>();
```

Use when:

- No shared state
- Short-lived operations

◈ Scoped

- One instance per HTTP request.
- Same instance reused within request.

Example:

```
services.AddScoped<IService, Service>();
```

Common use:

- DbContext
- Request-specific services

◈ Singleton

- One instance for entire application lifetime.

Example:

```
services.AddSingleton<IService, Service>();
```

Use when:

- Stateless service
- Configuration service
- Caching service

⚠ Must be thread-safe.

---

3. How does .NET register services?

In .NET Core:

```
builder.Services.AddScoped<IOrderService, OrderService>();
```

Behind the scenes:

- Service descriptor is added to IServiceCollection
- Container stores:
  - Service type
  - Implementation type
  - Lifetime

At runtime, container builds dependency graph.

---

4. How does reflection help in DI?

Reflection allows:

- Inspecting types at runtime
- Identifying constructors
- Discovering interfaces
- Creating instances dynamically

The container:

- Uses reflection to find constructor
- Resolves constructor parameters recursively
- Creates object dynamically

Reflection is how container knows what dependencies to inject.

---

5. How does assembly scanning work?

Assembly scanning automatically registers services without manually listing each one.

Example using Scrutor:

```
services.Scan(scan => scan
    .FromAssembliesOf(typeof(IOrderService))
    .AddClasses()
    .AsImplementedInterfaces()
    .WithScopedLifetime());
```

How it works:

- Load assembly
- Scan for classes matching condition
- Register them automatically

Useful in large projects.

---

6. How does the container resolve dependencies?

When a request is made:

1. Container looks for service registration.
2. Finds implementation type.
3. Inspects constructor.
4. Recursively resolves dependencies.
5. Creates instance based on lifetime.

This process is called **dependency graph resolution**.

---

7. What happens when multiple implementations exist?

If you register:

```
services.AddScoped<IService, ServiceA>();
services.AddScoped<IService, ServiceB>();
```

Then:

- Injecting `IEnumerable<IService>` → you get both implementations.
- Injecting single `IService` → last registered wins (by default).

Better approach:

- Use named services
- Use factory pattern
- Use keyed services (.NET 8+)
- Use strategy pattern

---

8. How do you dynamically register services?

There are several ways:

1 Assembly Scanning

Automatically register all classes implementing an interface.

---

2 Using Reflection

```
var types = Assembly.GetExecutingAssembly().GetTypes()
    .Where(t => typeof(IMyInterface).IsAssignableFrom(t));

foreach(var type in types)
{
    services.AddScoped(typeof(IMyInterface), type);
}
```

---

3 Factory-Based Registration

```
services.AddScoped<IService>(provider =>
{
    var config = provider.GetService<IConfig>();
    return new Service(config);
});
```

---

4 Conditional Registration

Based on environment:

```
if(environment.IsDevelopment())
    services.AddScoped<IService, DevService>();
else
    services.AddScoped<IService, ProdService>();
```

Strong Architect-Level Closing Statement

"Dependency Injection in .NET is more than just service registration — it's a runtime container that builds dependency graphs using reflection, manages lifetimes, supports assembly scanning, and enables scalable, loosely coupled architecture."

• What design patterns have you used? • Explain Repository Pattern. • Explain Factory Pattern. • Explain Singleton Pattern. • What locking mechanisms exist? • What is Mutex? • What is Monitor? • What is Semaphore? • How do you implement thread-safe singleton? • What is CQRS?

These questions are testing both **design pattern understanding and concurrency maturity**. I'll answer clearly in strong interview format.

---

1. What design patterns have you used?

In real projects, I have used:

- **Dependency Injection**
- **Repository Pattern**
- **Factory Pattern**
- **Singleton**
- **CQRS**
- **Saga Pattern**
- **Strategy Pattern**
- **Mediator Pattern**
- **Circuit Breaker Pattern**

I use patterns based on architectural need — not just for theory. The goal is always loose coupling, maintainability, and scalability.

---

2. Explain Repository Pattern

Repository Pattern abstracts data access logic from business logic.

Instead of business layer directly talking to Entity Framework or SQL, it talks to a repository interface.

Example:

```
public interface IOrderRepository
{
```

```
    Order GetById(int id);
    void Add(Order order);
}
```

Why use it?

- Separates business logic from data layer
- Makes code testable (can mock repository)
- Prevents tight coupling to EF or database

It's like having a warehouse manager — business doesn't directly go to the storage room.

---

3. Explain Factory Pattern

Factory Pattern centralizes object creation logic.

Instead of creating objects directly using `new`, we use a factory class.

Example:

```
public class PaymentFactory
{
    public IPaymentService Create(string type)
    {
        if(type == "Card") return new CardPayment();
        if(type == "UPI") return new UpiPayment();
        throw new Exception("Invalid type");
    }
}
```

Why use it?

- Encapsulates creation logic
- Follows Open-Closed Principle
- Easier to extend

It's like ordering from a restaurant — you don't cook it yourself.

---

4. Explain Singleton Pattern

Singleton ensures only one instance of a class exists throughout application lifetime.

Used for:

- Logging
- Configuration
- Caching

Basic idea:

```
public class Logger
{
    private static Logger _instance;
    public static Logger Instance => _instance ??= new Logger();
}
```

However, it must be thread-safe in multi-threaded environments.

---

5. What locking mechanisms exist?

In .NET, locking mechanisms include:

- `lock` keyword
- `Monitor`
- `Mutex`
- `Semaphore`
- `SemaphoreSlim`
- `ReaderWriterLockSlim`
- `Interlocked`

Each has different use cases.

---

6. What is Mutex?

Mutex is used for synchronization **across processes**.

- Heavier than Monitor
- Can work between different applications

Example use case:
Prevent two applications from accessing same file simultaneously.

---

7. What is Monitor?

Monitor is used for synchronization **within same process**.

`lock` keyword internally uses Monitor.

It allows:

- Enter / Exit
- Wait / Pulse
- TryEnter

It is lightweight compared to Mutex.

---

8. What is Semaphore?

Semaphore controls access to a resource by multiple threads.

Instead of allowing only one thread (like lock), it allows a limited number.

Example:
Allow maximum 5 threads to access database connection pool.

It is useful for throttling.

---

9. How do you implement thread-safe Singleton?

There are multiple ways.

◈ Double-checked locking

```
public class Singleton
{
    private static Singleton _instance;
    private static readonly object _lock = new object();

    public static Singleton Instance
    {
        get
        {
            if (_instance == null)
            {
                lock (_lock)
                {
                    if (_instance == null)
                        _instance = new Singleton();
                }
            }
            return _instance;
        }
```

```
    }
}
```

◈ Better approach (Recommended)

Use static initialization:

```
public class Singleton
{
    private static readonly Singleton _instance = new Singleton();
    public static Singleton Instance => _instance;
}
```

Static constructor guarantees thread safety.

---

10. What is CQRS?

CQRS stands for **Command Query Responsibility Segregation**.

It separates:

- Commands → modify data
- Queries → read data

Instead of one model doing both.

Benefits:

- Better scalability
- Separate optimization
- Cleaner responsibility
- Read replicas possible

Example:

- Write operations → Primary DB
- Read operations → Secondary DB

CQRS is useful in high-scale enterprise systems.

Strong Architect-Level Closing Line

"I use design patterns not as theory, but as architectural tools — to ensure clean separation of concerns, thread safety, scalability, and maintainable enterprise-grade systems."

• Difference between EF6 and EF Core? • What is Code First? • What is Database First? • What are DBSet? • What is OnModelCreating? • What is model builder? • How does EF Core support multi-tenancy? • What are precompiled queries? • What are migrations? • How is EF Core different in performance?

These questions test **deep ORM understanding + architectural maturity with EF Core**. I'll answer clearly in interview-ready format.

---

1. Difference between EF6 and EF Core?

EF6 is the older, full .NET Framework version.
EF Core is modern, lightweight, cross-platform, and actively developed.

Key differences:

Feature	EF6	EF Core
Platform	.NET Framework only	.NET Core / .NET 5+
Performance	Slower	Faster
Cross-platform	No	Yes
LINQ translation	Limited	Improved
Batch updates	Limited	Supported
Raw SQL support	Basic	Improved
Shadow properties	No	Yes
Global query filters	No	Yes

EF Core is optimized for microservices and cloud-native apps.

---

2. What is Code First?

Code First means:

- Define C# entity classes
- Define DbContext
- Use migrations to generate database

Example:

```
public class Order
{
    public int Id { get; set; }
    public string Name { get; set; }
}
```

EF generates tables from these classes.

Best for:

- New projects
- Domain-driven design
- Agile development

---

3. What is Database First?

Database First means:

- Database schema already exists
- EF generates C# models from database

Used when:

- Legacy systems
- DB managed by DBA team
- Enterprise integration

We use scaffolding command to generate models.

---

4. What are DbSet?

DbSet represents a table in the database.

Example:

```
public DbSet<Order> Orders { get; set; }
```

It allows:

- Querying
- Adding
- Updating
- Deleting

DbSet acts like a collection mapped to database table.

---

5. What is OnModelCreating?

OnModelCreating is a method in DbContext used to configure entity relationships and constraints using Fluent API.

Example:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Order>()
        .HasKey(o => o.Id);
}
```

It allows:

- Configure relationships
- Define keys
- Set indexes
- Configure table names
- Apply global filters

---

6. What is ModelBuilder?

ModelBuilder is used inside OnModelCreating.

It builds entity-to-table mapping.

It defines:

- Primary keys

- Foreign keys
- Indexes
- Constraints
- Default values
- Query filters

Think of it as schema configuration builder.

---

7. How does EF Core support multi-tenancy?

EF Core supports multi-tenancy mainly using:

◈ Global Query Filters

```
modelBuilder.Entity<Order>()
    .HasQueryFilter(o => o.TenantId == _tenantId);
```

This ensures:

- Every query automatically filters by tenant
- Prevents accidental cross-tenant data access

Other approaches:

- Separate database per tenant
- Separate schema per tenant
- Dynamic connection string resolution

Global filters are most common for shared DB model.

---

8. What are precompiled queries?

Precompiled queries are queries compiled once and reused multiple times.

EF normally compiles LINQ to SQL dynamically each time.

With compiled queries:

- Query plan reused
- Faster execution
- Reduced overhead

Used in high-performance scenarios.

Example:

```
EF.CompileQuery((MyContext ctx, int id) =>
    ctx.Orders.FirstOrDefault(o => o.Id == id));
```

---

9. What are migrations?

Migrations track schema changes over time.

When you change entity class:

- Add migration
- EF generates migration file
- Apply migration to update database

Commands:

```
Add-Migration
Update-Database
```

Migrations help version-control database schema.

---

10. How is EF Core different in performance?

EF Core is:

- Faster than EF6
- Better query translation
- Improved batching
- Supports no-tracking queries
- Supports split queries
- Better memory usage
- Optimized for async operations

Performance improvements include:

- Better SQL generation
- Reduced reflection overhead
- Compiled queries
- Reduced change tracking when not needed

However, for very high-performance scenarios, sometimes raw SQL or Dapper may be preferred.

Strong Interview Closing Line

"EF Core is a modern, high-performance ORM that supports global query filters, compiled queries, improved LINQ translation, and cloud-native multi-tenant architecture, making it ideal for scalable enterprise applications."

• What is execution plan? • What is table scan? • What is index scan? • What is clustered index? • What is non-clustered index? • What is CTE? • What is temp table? • What is profiler? • What is RECOMPILE in stored procedure? • Why use WITH RECOMPILE? • What is parameter sniffing? • What is isolation level? • What are types of isolation levels? • What is dirty read? • What is read committed? • What is read uncommitted? • What is repeatable read? • What is serializable?

These are **core SQL performance and transaction concepts**. I'll explain them clearly in interview-ready format.

---

1. What is execution plan?

An execution plan is a roadmap that shows how SQL Server executes a query.

It shows:

- Which indexes are used
- Whether it performs table scan or index seek
- Join types (Nested Loop, Hash Match, Merge Join)
- Cost of each operation

It helps identify performance bottlenecks.

You can view it using:

- Actual Execution Plan in SSMS

---

2. What is table scan?

Table scan means SQL Server reads the entire table to find matching records.

Example:
If no index exists on filtering column → full table scan happens.

It is like searching every page in a book to find a word.

Usually bad for large tables.

---

3. What is index scan?

Index scan means SQL reads the entire index instead of full table.

Better than table scan but still scans many rows.

Ideal scenario is:

✔ Index Seek → SQL jumps directly to matching rows.

---

4. What is clustered index?

Clustered index determines the physical order of data in table.

- Only one per table
- Data stored sorted by clustered key

Example:
Primary key often used as clustered index.

Best for range queries.

---

5. What is non-clustered index?

Non-clustered index is a separate structure storing:

- Indexed column values
- Pointer to actual data row

A table can have multiple non-clustered indexes.

Useful for filtering and lookup queries.

---

6. What is CTE?

CTE (Common Table Expression) is a temporary result set defined inside a query.

Example:

```
WITH CTE AS
(
   SELECT * FROM Orders WHERE Amount > 1000
)
SELECT * FROM CTE;
```

Used for:

- Improving readability
- Breaking complex queries
- Recursive queries

---

7. What is temp table?

Temp table is a temporary table stored in tempdb.

Example:

```
CREATE TABLE #TempOrders
(
   Id INT,
   Name VARCHAR(100)
);
```

Used when:

- Storing intermediate results
- Large data transformations
- Reusing dataset multiple times

---

8. What is profiler?

SQL Profiler is a tool that captures:

- Executed queries
- Stored procedure calls
- Deadlocks

- Performance metrics

Used for diagnosing performance issues.

In Azure SQL, Extended Events or Azure Monitor are preferred.

---

9. What is RECOMPILE in stored procedure?

`WITH RECOMPILE` forces SQL Server to:

- Generate a new execution plan every time procedure runs.

It prevents reuse of cached execution plan.

---

10. Why use WITH RECOMPILE?

Used when:

- Parameter sniffing causes performance issues
- Data distribution varies significantly
- Query plan should adapt to parameter values

But it increases CPU usage due to frequent recompilation.

---

11. What is parameter sniffing?

Parameter sniffing happens when:

- SQL Server creates execution plan based on first parameter value.
- Same plan reused for different parameter values.

If data distribution varies, performance degrades.

Example:
Small dataset parameter → plan optimized for small
Later large dataset parameter → same plan reused → slow

---

12. What is isolation level?

Isolation level defines how transactions are isolated from each other.

It controls:

- Locking behavior
- Visibility of uncommitted data
- Concurrency vs consistency balance

---

13. What are types of isolation levels?

SQL Server supports:

1. Read Uncommitted
2. Read Committed
3. Repeatable Read
4. Serializable
5. Snapshot

Each level increases data consistency but reduces concurrency.

---

14. What is dirty read?

Dirty read happens when:

- Transaction reads uncommitted data from another transaction.

If other transaction rolls back → read data was invalid.

Occurs in:

- Read Uncommitted

---

15. What is Read Committed?

Read Committed:

- Default isolation level.

- Prevents dirty reads.
- Can still have non-repeatable reads.

Only committed data can be read.

---

16. What is Read Uncommitted?

Lowest isolation level.

- Allows dirty reads.
- No shared locks.
- Highest concurrency.
- Lowest consistency.

Equivalent to using:

```
WITH (NOLOCK)
```

---

17. What is Repeatable Read?

Repeatable Read:

- Prevents dirty reads.
- Prevents non-repeatable reads.
- Locks rows until transaction completes.

Still allows phantom reads.

---

18. What is Serializable?

Highest isolation level.

- Prevents dirty reads.
- Prevents non-repeatable reads.
- Prevents phantom reads.
- Locks range of rows.

Most consistent but lowest concurrency.

Strong Interview Closing Line

"Execution plans help identify scans and performance issues, indexing improves query efficiency, RECOMPILE helps avoid parameter sniffing, and isolation levels control the balance between consistency and concurrency in transactional systems."

• Have you implemented authentication? • How do you use JWT? • How do you validate tokens? • What middleware is used? • How do you protect APIs? 13 Angular • What Angular version have you worked with? • Have you migrated Angular versions?

These questions are testing **practical security implementation + frontend experience**. I'll answer clearly in interview format.

🔐 Authentication & API Security

---

1. Have you implemented authentication?

Yes, I have implemented authentication in multiple projects using:

- JWT-based authentication
- OAuth2 with Authorization Code Flow
- Azure AD integration
- IdentityServer (centralized identity provider)

Typical flow:

1. User logs in with credentials.
2. Identity provider validates credentials.
3. Access token (JWT) is issued.
4. Client sends JWT in Authorization header.
5. API validates token before processing request.

Authentication ensures identity verification.

---

2. How do you use JWT?

JWT (JSON Web Token) is used as a bearer token.

Flow:

- After login → server generates JWT
- JWT contains:
  - User ID
  - Roles
  - Tenant ID
  - Expiry
- Client sends:

```
Authorization: Bearer <token>
```

The API validates token on every request.

JWT is:

- Stateless
- Scalable
- Secure when used with HTTPS

---

3. How do you validate tokens?

Token validation happens in middleware.

Validation includes:

- Signature validation
- Expiry check
- Issuer validation
- Audience validation
- Scope/role validation

If validation fails → 401 Unauthorized.

In Azure AD scenario:

- Public keys are fetched from metadata endpoint.
- Token signature verified using RSA public key.

---

4. What middleware is used?

In ASP.NET Core:

```
app.UseAuthentication();
app.UseAuthorization();
```

Configured with:

```
services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
        .AddJwtBearer(options =>
        {
            options.Authority = "...";
            options.Audience = "...";
        });
```

JWT Bearer middleware automatically:

- Extracts token
- Validates signature
- Populates User claims

---

5. How do you protect APIs?

We protect APIs using:

- JWT authentication
- `[Authorize]` attribute
- Role-based authorization
- Policy-based authorization
- Scope validation
- HTTPS enforcement
- Rate limiting (API Gateway)
- IP filtering (if required)

We follow defense-in-depth approach:
Network → Authentication → Authorization.

A Angular

---

6. What Angular version have you worked with?

I have worked with Angular 14, 15, and more recently Angular 16/17 in modern projects.

I am comfortable with:

- Standalone components
- Reactive Forms
- Signals (Angular 16+)
- Strict template type checking
- Angular Material
- RxJS
- Lazy loading modules

---

7. Have you migrated Angular versions?

Yes.

I have performed Angular version upgrades.

Migration process includes:

1. Updating Angular CLI
2. Running `ng update`
3. Fixing breaking changes
4. Updating RxJS
5. Updating Angular Material
6. Testing components and services

Important migration areas:

- Strict typing changes
- Deprecated lifecycle hooks
- Changes in RxJS operators
- Module-to-standalone migration (in newer versions)

Migration requires careful testing and gradual refactoring.

Strong Interview Closing Line

"I have hands-on experience implementing JWT-based authentication, securing APIs using middleware and authorization policies, and working with modern Angular versions including migrations and advanced frontend patterns."

• What is encapsulation? • What are interfaces? • What are delegates? • When would you use delegates? • What is the use of properties in programming? • What are getter and setter? • Why use properties instead of public variables? 3 LINQ • What is the difference between First()

and FirstOrDefault()? • When does First() throw exception? • When should we use FirstOrDefault()?

These are **core OOP + C# fundamentals + LINQ clarity** questions. I'll answer clearly in interview-ready format.

◈ OOP Concepts

---

1. What is encapsulation?

Encapsulation means hiding internal implementation details and exposing only what is necessary.

Example:

```
private decimal _balance;

public void Deposit(decimal amount)
{
    if(amount > 0)
        _balance += amount;
}
```

Here:

- `_balance` is hidden (private)
- Access controlled through methods

Encapsulation protects data and ensures controlled access.

It's like an ATM — you don't see internal banking system, only deposit/withdraw options.

---

2. What are interfaces?

An interface defines a contract — what a class must implement.

Example:

```
public interface IPaymentService
{
```

```
    void ProcessPayment();
}
```

Any class implementing this must define `ProcessPayment()`.

Interfaces provide:

- Abstraction
- Loose coupling
- Testability
- Multiple inheritance capability (via interfaces)

---

3. What are delegates?

A delegate is a type that holds reference to a method.

It allows:

- Passing methods as parameters
- Callback mechanisms
- Event handling

Example:

```
public delegate void Notify(string message);
```

Delegates are type-safe function pointers.

---

4. When would you use delegates?

Use delegates when:

- Implementing events
- Passing behavior dynamically
- Callback methods
- Strategy pattern
- LINQ uses delegates internally (Func, Action)

Example:
Sorting with custom comparison logic.

Delegates allow flexible behavior injection.

5. What is the use of properties in programming?

Properties allow controlled access to class fields.

They look like variables but internally use getter/setter methods.

Example:

```
public string Name { get; set; }
```

Properties:

- Encapsulate data
- Add validation logic
- Maintain abstraction

---

6. What are getter and setter?

Getter:

- Retrieves value

Setter:

- Sets value

Example:

```
private int _age;

public int Age
{
    get { return _age; }
    set
    {
        if(value > 0)
            _age = value;
    }
}
```

Here setter validates input.

---

7. Why use properties instead of public variables?

If we use public variables:

```
public int Age;
```

There is:

- No validation
- No encapsulation
- No control

With properties:

- We can validate input
- Add logging
- Make read-only properties
- Change implementation later

Properties provide flexibility and control.

◈ LINQ

---

8. What is the difference between First() and FirstOrDefault()?

First()

- Returns first matching element.
- Throws exception if no element found.

FirstOrDefault()

- Returns first matching element.
- Returns default value if no element found.

Default values:

- null for reference types
- 0 for int
- false for bool

---

9. When does First() throw exception?

First() throws:

```
InvalidOperationException
```

When:

- Sequence is empty
- No element matches condition

Example:

```
var result = list.First(x => x > 100);
```

If no element > 100 → exception.

---

10. When should we use FirstOrDefault()?

Use FirstOrDefault() when:

- There may be no matching result
- You want safe handling
- Avoid exceptions

Then check for null:

```
var result = list.FirstOrDefault(x => x > 100);
if(result != null)
{
    // process
}
```

Strong Interview Closing Line

"Encapsulation protects internal state, interfaces define contracts, delegates enable behavior injection, properties provide controlled access, and in LINQ, First() throws when no match exists while FirstOrDefault() safely returns a default value."

• When do we use static classes? • Give real project examples of static class usage. • Why not create object? • What are risks of static classes? 5 Design Patterns • What design patterns have you used? • What are creational, structural, behavioral patterns? • Which patterns did you implement? • Explain SOLID principles. o Single Responsibility o Open/Closed o Liskov

Substitution o Interface Segregation o Dependency Inversion • Give real example of SRP violation. • How do you implement OCP? • How do you use abstraction?

These questions are testing **OOP maturity + architectural thinking + real project experience**. I'll answer clearly in interview-ready format.

◈ Static Classes

---

1. When do we use static classes?

We use static classes when:

- The class does not need instance-level state
- All methods are utility/helper methods
- Behavior is shared globally
- Object creation does not make sense

Example cases:

- Utility helpers
- Constants
- Extension methods
- Logging wrappers

Static classes cannot be instantiated and contain only static members.

---

2. Give real project examples of static class usage.

In real projects, I used static classes for:

◈ Utility helpers

```
public static class DateHelper
{
    public static DateTime ToUtc(DateTime date) =>
date.ToUniversalTime();
}
```

◈ Constants

```
public static class Roles
{
```

```
    public const string Admin = "Admin";
}
```

◈ Extension methods

```
public static class StringExtensions
{
    public static bool IsNullOrEmpty(this string value)
    {
        return string.IsNullOrEmpty(value);
    }
}
```

These are stateless and reusable.

---

3. Why not create object?

If:

- No instance data
- No dependency injection needed
- No lifecycle management required

Creating objects adds unnecessary overhead.

Example:
You don't create object of `Math` class.
You use `Math.Abs()`.

Static is appropriate for pure utility logic.

---

4. What are risks of static classes?

Static classes have risks:

- Hard to unit test (cannot mock easily)
- Tight coupling
- Shared global state risk
- Thread-safety concerns
- Breaks Dependency Injection principles

In enterprise architecture, overuse of static classes leads to rigid design.

◈ Design Patterns

---

5. What design patterns have you used?

I have used:

- Dependency Injection
- Repository Pattern
- Factory Pattern
- Singleton
- CQRS
- Saga Pattern
- Strategy Pattern
- Mediator Pattern
- Circuit Breaker Pattern

Used in microservices and enterprise systems.

---

6. What are creational, structural, behavioral patterns?

◈ Creational Patterns

Deal with object creation.

Examples:

- Singleton
- Factory
- Abstract Factory
- Builder

◈ Structural Patterns

Deal with object composition.

Examples:

- Adapter
- Decorator
- Facade

- Proxy

◈ Behavioral Patterns

Deal with object communication.

Examples:

- Strategy
- Observer
- Mediator
- Command
- Saga

---

7. Which patterns did you implement?

In real projects:

- Repository (data abstraction)
- Factory (dynamic object creation)
- Singleton (configuration service)
- Strategy (payment provider selection)
- CQRS (read/write separation)
- Saga (distributed transactions)
- Circuit Breaker (resilience in microservices)

◈ SOLID Principles

---

8. Explain SOLID principles

◈ S – Single Responsibility Principle (SRP)

A class should have only one reason to change.

Bad example:

```
public class InvoiceService
{
    public void GenerateInvoice() { }
    public void SendEmail() { }
    public void SaveToDatabase() { }
}
```

This class has multiple responsibilities.

Better:

- InvoiceGenerator
- EmailService
- InvoiceRepository

◈ O – Open/Closed Principle (OCP)

Software should be open for extension, closed for modification.

Instead of modifying existing class, extend behavior via abstraction.

Example:
Use interface for payment methods instead of adding if/else.

◈ L – Liskov Substitution Principle (LSP)

Derived classes should be replaceable for base class without breaking behavior.

If base class says:
`Bird.Fly()`

Then Penguin should not inherit Bird with Fly().

Because penguin cannot fly.

Violation causes runtime issues.

◈ I – Interface Segregation Principle (ISP)

Clients should not depend on interfaces they don’t use.

Bad:

```
interface IWorker
{
    void Work();
    void Eat();
}
```

If Robot implements this but cannot Eat → bad design.

Better: Split interfaces.

◈ D – Dependency Inversion Principle (DIP)

High-level modules should not depend on low-level modules.

Both should depend on abstraction.

Instead of:

```
OrderService depends on SqlRepository
```

Use:

```
OrderService depends on IOrderRepository
```

---

9. Give real example of SRP violation.

Example:

```
public class UserService
{
    public void CreateUser() { }
    public void SendWelcomeEmail() { }
    public void LogActivity() { }
}
```

This violates SRP because:

- User creation logic
- Email logic
- Logging logic

Three different responsibilities.

---

10. How do you implement OCP?

Use abstraction and polymorphism.

Instead of:

```
if(type == "CreditCard") ...
if(type == "UPI") ...
```

Use:

```
interface IPayment { void Pay(); }
```

Add new payment types without modifying existing code.

Extension, not modification.

---

11. How do you use abstraction?

Abstraction hides implementation details behind interfaces.

Example:

```
public interface IEmailService
{
    void Send();
}
```

Business logic does not care:

- SMTP
- SendGrid
- Azure Communication Services

It depends only on abstraction.

This makes system flexible and testable.

🔥 Strong Senior-Level Closing Statement

"I apply SOLID principles and design patterns not for theoretical purity, but to ensure scalability, maintainability, loose coupling, and clean architecture in real enterprise systems."

• What is difference between MVC Controller and API Controller? • Can API return View? • What return types are available in MVC? • Why use Web API? • What is stateless service? 7️⃣ Authentication / Authorization • How do you implement authentication in API? • How do you implement authorization? • How do you use JWT token? • How do you validate token in middleware? • What is difference between authentication and authorization? • Where do you validate token? Filter or Middleware? • How do you restrict specific users? • How do you restrict based on domain? • How do you restrict based on IP address? • How do you implement CORS? • Which comes first: domain validation or authentication?

_____

Excellent — this is a very structured backend + security round. I'll answer clearly, confidently, and in interview format.

◈ MVC vs Web API

---

1. What is difference between MVC Controller and API Controller?

◈ MVC Controller

- Used for building **web applications**
- Returns **Views (HTML)**
- Used in server-side rendering
- Returns ViewResult, PartialViewResult, etc.

◈ API Controller

- Used for building **REST APIs**
- Returns **JSON or data**
- No views
- Used for frontend apps, mobile apps, integrations

In ASP.NET Core:

- MVC → `Controller`

- API → `ControllerBase`

Key difference:
MVC = UI rendering
API = Data service

---

2. Can API return View?

Technically yes (if inheriting from `Controller` instead of `ControllerBase`), but practically **No**.

API should return:

- JSON
- XML
- Status codes

API should not render HTML views. That breaks REST principles.

---

3. What return types are available in MVC?

Common return types:

- `ViewResult`
- `PartialViewResult`
- `JsonResult`
- `RedirectResult`
- `FileResult`
- `ContentResult`
- `IActionResult`
- `ActionResult<T>`

In Web API, usually:

- `IActionResult`
- `ActionResult<T>`

---

4. Why use Web API?

We use Web API when:

- Building RESTful services
- Backend for SPA (Angular/React)
- Mobile applications
- Microservices
- Third-party integrations

Web API is:

- Stateless
- Lightweight
- JSON-based
- Scalable

---

5. What is stateless service?

Stateless means:

- Server does not store client session
- Each request is independent
- All required information sent with each request

Example:
JWT token sent in every API call.

Stateless services:

- Scale better
- Work well with load balancers
- Suitable for microservices

🔐 Authentication & Authorization

---

6. How do you implement authentication in API?

Typical JWT implementation:

1. User logs in.
2. Credentials validated.
3. JWT token generated.
4. Client sends token in header:

```
Authorization: Bearer <token>
```

5. Middleware validates token.

Configured in .NET:

```
services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
        .AddJwtBearer(options => { ... });
```

---

7. How do you implement authorization?

Authorization happens after authentication.

Methods:

- `[Authorize]` attribute
- Role-based:

  ```
  [Authorize(Roles = "Admin")]
  ```

- Policy-based:

  ```
  [Authorize(Policy = "RequireManager")]
  ```

Authorization checks claims inside token.

---

8. How do you use JWT token?

JWT contains:

- User ID
- Roles
- Tenant ID
- Expiration
- Signature

Sent in every request header.

API validates:

- Signature
- Expiry
- Issuer
- Audience

---

9. How do you validate token in middleware?

In ASP.NET Core:

```
app.UseAuthentication();
app.UseAuthorization();
```

JWT middleware automatically:

- Extracts token
- Validates signature
- Checks expiration
- Builds user claims

If invalid → returns 401.

---

10. Difference between authentication and authorization?

Authentication → Who are you?
Authorization → What are you allowed to do?

AuthN before AuthZ.

---

11. Where do you validate token? Filter or Middleware?

Best practice:

✔ Middleware

Because:

- Centralized
- Executes before controllers
- Applies to entire application

Filters are used for additional custom validation.

Middleware is preferred.

---

12. How do you restrict specific users?

Using:

- Role-based authorization
- Claim-based authorization
- Policy-based authorization

Example:

```
[Authorize(Roles = "Admin")]
```

Or check user ID inside logic.

---

13. How do you restrict based on domain?

Two approaches:

1. CORS configuration
2. Validate Origin header manually

Example:

```
services.AddCors(options =>
{
    options.AddPolicy("AllowTrusted",
        builder => builder.WithOrigins("https://trusted.com"));
});
```

But remember:
CORS protects browsers only.

---

14. How do you restrict based on IP address?

Options:

- Middleware IP whitelist
- API Gateway IP filtering
- Azure WAF rules
- Reverse proxy rules

Example middleware:
Check `HttpContext.Connection.RemoteIpAddress`.

Infrastructure-level filtering is preferred.

---

15. How do you implement CORS?

In Program.cs:

```
services.AddCors(options =>
{
    options.AddPolicy("AllowSpecificOrigin",
        builder => builder.WithOrigins("https://example.com")
                          .AllowAnyHeader()
                          .AllowAnyMethod());
});
```

Then:

```
app.UseCors("AllowSpecificOrigin");
```

CORS prevents unauthorized cross-origin browser calls.

---

16. Which comes first: domain validation or authentication?

Correct order:

1. Network-level validation (IP/domain)
2. Authentication (JWT validation)
3. Authorization (role/policy)

Infrastructure restrictions should come first.

🔥 Strong Senior-Level Closing Line

"I implement layered API security — network restrictions, JWT-based authentication via middleware, and policy-based authorization — ensuring stateless, scalable, and secure API architecture."

• What is DbContext? • What are methods available in DbContext? • What is DbSet? • How does SaveChanges work? • What is difference between EF6 and EF Core? • What are improvements in EF Core? • What are precompiled queries? • How does OnModelCreating work? _____

Excellent — this is testing **deep Entity Framework understanding**, not surface-level usage. I'll answer clearly and interview-ready.

---

1. What is DbContext?

`DbContext` is the primary class in Entity Framework that:

- Manages database connection
- Tracks entity changes
- Translates LINQ queries into SQL
- Executes CRUD operations

It acts as a bridge between application and database.

Think of it as a **unit of work + repository manager combined**.

Example:

```
public class AppDbContext : DbContext
{
    public DbSet<Order> Orders { get; set; }
}
```

---

2. What are methods available in DbContext?

Common methods:

◈ Data Modification

- `Add()`
- `AddRange()`
- `Update()`
- `Remove()`
- `RemoveRange()`

◈ Saving

- `SaveChanges()`
- `SaveChangesAsync()`

◈ Tracking

- `Entry()`

- `ChangeTracker`

◈ Database operations

- `Database.Migrate()`
- `Database.ExecuteSqlRaw()`
- `Database.BeginTransaction()`

---

3. What is DbSet?

`DbSet<T>` represents a table in the database.

It allows:

- Querying using LINQ
- Adding new records
- Updating records
- Deleting records

Example:

```
context.Orders.Where(o => o.Id == 1);
```

Internally, DbSet builds expression trees and converts them to SQL.

---

4. How does SaveChanges work?

When you call:

```
context.SaveChanges();
```

EF does the following:

1. Detects changes via ChangeTracker.
2. Identifies entity states:
    - Added
    - Modified
    - Deleted
    - Unchanged
3. Generates appropriate SQL commands.
4. Wraps operations in transaction.
5. Executes SQL.

6. Updates entity states.

EF tracks entities automatically unless `AsNoTracking()` is used.

---

5. What is difference between EF6 and EF Core?

Feature	EF6	EF Core
Platform	.NET Framework only	Cross-platform (.NET Core+)
Performance	Slower	Faster
LINQ translation	Limited	Improved
Batch operations	Limited	Supported
Global query filters	No	Yes
Shadow properties	No	Yes
Owned entities	Limited	Supported

EF Core is lightweight and optimized for modern microservices.

---

6. What are improvements in EF Core?

Major improvements:

- Better performance
- Improved SQL generation
- Compiled queries
- Global query filters (multi-tenancy)
- Better async support
- Split queries
- Batch updates
- No-tracking queries optimized
- Cross-platform support

EF Core is optimized for cloud-native applications.

---

7. What are precompiled queries?

Normally EF compiles LINQ queries each time.

Precompiled queries:

- Compile query once
- Reuse execution plan
- Reduce overhead
- Improve performance in high-frequency scenarios

Example:

```
var compiledQuery = EF.CompileQuery(
    (AppDbContext ctx, int id) =>
        ctx.Orders.FirstOrDefault(o => o.Id == id)
);
```

Used in performance-critical APIs.

---

8. How does OnModelCreating work?

`OnModelCreating` is used to configure entity mapping using Fluent API.

It runs when model is being built.

Example:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Order>()
        .HasKey(o => o.Id);

    modelBuilder.Entity<Order>()
        .HasIndex(o => o.Name);
}
```

It defines:

- Primary keys
- Foreign keys
- Relationships
- Indexes
- Constraints
- Global query filters
- Table names

- Default values

ModelBuilder builds metadata model that EF uses to generate SQL.

🔥 Strong Senior-Level Closing Line

"DbContext acts as a unit-of-work that tracks entity states, generates SQL via change tracking, and EF Core improves performance, flexibility, and multi-tenant support through features like compiled queries and global query filters."

• You changed a logo image locally and QA works but production still shows old logo. Why? • Is it browser cache? • Is it IIS cache? • How do you clear IIS cache? • How do you debug production-only issue? • How do you ensure 10,000 users get updated content? • What could be server-side caching issues? • What is app pool recycle?

Excellent — this is a **real-world production troubleshooting round**. I'll answer clearly and practically like in a senior-level interview.

1️⃣ You changed a logo image locally and QA works but production still shows old logo. Why?

There are several possible reasons:

1. **Browser cache**
2. **Server-side cache**
3. **CDN cache**
4. **IIS static file caching**
5. **File not actually deployed**
6. **Load balancer pointing to different server**
7. **Old file still present in production folder**
8. **Incorrect deployment slot**

First step: Verify whether production server actually has updated file.

2️⃣ Is it browser cache?

Yes, very common reason.

Browsers cache static files like:

- Images
- CSS
- JS

If file name remains same:
```
logo.png
```

Browser may continue using cached version.

Quick test:

- Open in Incognito
- Hard refresh (Ctrl + F5)
- Clear browser cache

If it works after that → browser caching issue.

---

3 Is it IIS cache?

Yes, IIS can cache static content.

IIS may cache:

- Static files
- Output caching
- Response caching

Especially if response headers include cache-control.

---

4 How do you clear IIS cache?

Options:

◈ Restart IIS
```
iisreset
```

◈ Recycle App Pool

In IIS Manager:

- Go to Application Pools
- Right click → Recycle

◈ Clear output cache (if enabled)

Also verify:

- Physical file replaced
- No duplicate file in virtual directory

---

5 How do you debug production-only issue?

Structured approach:

1. Verify file exists on production server.
2. Check timestamp of file.
3. Check response headers:
    - Cache-Control
    - ETag
    - Last-Modified
4. Check if CDN is involved.
5. Check load balancer → multiple servers?
6. Check deployment logs.
7. Use browser DevTools → Network tab.

Never assume — verify step by step.

---

6 How do you ensure 10,000 users get updated content?

Best practice:

◈ Versioning Static Files

Instead of:

```
logo.png
```

Use:

```
logo_v2.png
```

Or:

```
logo.png?v=2
```

This forces browser to fetch new version.

This is called **cache busting**.

Enterprise systems use:

- File hashing
- Build versioning
- CDN invalidation

---

7 What could be server-side caching issues?

Common server-side caching:

- Output caching
- In-memory caching
- Redis cache
- CDN caching
- Reverse proxy caching
- Azure Front Door caching

If CDN exists:

- Need to purge CDN cache.

If Redis stores metadata:

- Invalidate cache key.

Server caching can cause stale content even if file updated.

---

8 What is app pool recycle?

App Pool recycle means:

- IIS restarts the worker process
- Clears memory
- Resets application state
- Reloads application

Reasons for recycle:

- Memory limits exceeded
- Scheduled recycle
- Manual recycle
- Deployment

Recycling can fix:

- Memory leaks
- Stale in-memory cache
- Configuration refresh issues

But it does NOT clear browser cache.

🔥 Senior-Level Troubleshooting Summary

"When production shows old content, I systematically check browser cache, CDN cache, IIS caching, load balancer nodes, deployment verification, and response headers. The safest long-term solution is static asset versioning to ensure all users receive updated content without relying on cache clearing."

• How do you ensure code quality? • What coding standards do you follow? • How do you enforce naming conventions? • How do you handle exception handling? • What static code analysis tools do you use? • Have you used SonarQube? • What are code review practices? • How do you maintain reusable code?

Excellent — this round is testing **engineering discipline, not just coding ability**. I'll answer in strong senior-level interview format.

1️⃣ How do you ensure code quality?

I ensure code quality through multiple layers:

🔷 1. Clean Architecture

- Separation of concerns
- Layered structure
- Dependency injection
- No business logic in controllers

◈ 2. Code Reviews

- Peer review before merging
- Enforce PR approval policy

◈ 3. Static Code Analysis

- SonarQube
- Roslyn analyzers
- StyleCop

◈ 4. Unit Testing

- Write testable code
- Mock dependencies
- Maintain coverage for critical modules

◈ 5. CI/CD Validation

- Build validation
- Linting
- Security scanning

Quality is enforced by process, not just developer discipline.

---

2 What coding standards do you follow?

I follow:

◈ Microsoft C# Coding Standards

- PascalCase for classes & methods
- camelCase for local variables
- Meaningful variable names
- No abbreviations unless standard

◈ SOLID principles

◈ DRY (Don't Repeat Yourself)

◈ KISS (Keep It Simple)

◈ YAGNI (You Aren't Gonna Need It)

I also ensure:

- No magic strings
- Constants centralized
- Proper null handling
- Logging standards

Consistency improves maintainability.

---

3 How do you enforce naming conventions?

Enforcement mechanisms:

- EditorConfig rules
- StyleCop
- SonarQube rules
- Code review checklist

Example conventions:

- Interfaces start with `I`
- Async methods end with `Async`
- Private fields use `_camelCase`
- Constants use PascalCase or UPPER_CASE depending on team standard

Automation + review ensures enforcement.

---

4 How do you handle exception handling?

◈ Global Exception Handling

In ASP.NET Core:

```
app.UseExceptionHandler("/error");
```

Or custom middleware.

◈ Structured Logging

- Log error with correlation ID
- Avoid exposing internal errors to client
- Return proper HTTP status codes

◈ Custom Exceptions

Create domain-specific exceptions:

- ValidationException
- BusinessRuleException

Never swallow exceptions silently.

---

5 What static code analysis tools do you use?

I have used:

- SonarQube
- SonarLint
- StyleCop
- ReSharper
- Roslyn Analyzers
- Azure DevOps build validation rules

These tools detect:

- Code smells
- Security vulnerabilities
- Complexity issues
- Dead code

- Naming violations

---

6 Have you used SonarQube?

Yes.

We used SonarQube integrated with CI pipeline.

It checks:

- Code coverage
- Security hotspots
- Bugs
- Code duplication
- Cognitive complexity

We enforced Quality Gates:

- No new critical issues
- Maintain minimum coverage threshold

It improves long-term maintainability.

---

7 What are code review practices?

Strong code review includes:

◈ Checklist-based Review

- Naming clarity
- SOLID adherence
- Performance impact
- Security risks
- Logging coverage
- Null handling
- Exception handling

◈ No direct commits to main branch

◈ Minimum 1–2 reviewers

◈ Small pull requests preferred

Review is not about criticism — it's about maintainability and risk reduction.

---

8 How do you maintain reusable code?

Reusable code strategies:

◈ Shared Libraries

- Common utilities project
- Infrastructure layer
- Shared DTOs

◈ Extension Methods

◈ Generic Services

◈ Base Classes for common behavior

◈ NuGet packages for internal reuse

Also:

- Avoid duplication
- Keep methods small and focused
- Follow SRP

Reusable code must be:

- Generic
- Well-tested
- Properly documented

🔥 Strong Senior-Level Closing Statement

"I maintain code quality through structured architecture, static analysis tools like SonarQube, disciplined code reviews, consistent naming conventions, centralized exception handling, and reusable modular design aligned with SOLID principles."

• Have you worked on unit testing? • Have you used NUnit? • Have you used Moq? • Have you implemented automation testing? • Why unit testing is important?

Excellent — this is testing your **testing maturity and engineering discipline**. I'll answer clearly in strong interview format.

---

1 Have you worked on unit testing?

Yes, I have worked on unit testing in backend services, especially for:

- Business logic validation
- Service layer methods
- Command/Query handlers (CQRS)
- Validation logic
- Edge case handling

I focus on testing **business logic**, not framework code.

I ensure:

- Dependencies are mocked
- Tests are isolated
- Tests are deterministic
- Fast execution

Unit testing improves reliability and reduces regression bugs.

---

2 Have you used NUnit?

Yes.

I have used **NUnit** for writing unit tests.

Example:

```
[Test]
public void AddOrder_Should_Return_OrderId()
{
    var result = _orderService.AddOrder(order);

    Assert.IsNotNull(result);
}
```

NUnit provides:

- Attributes like `[Test]`, `[SetUp]`, `[TestCase]`
- Assertions
- Test lifecycle management

It is simple, stable, and widely used.

---

3 Have you used Moq?

Yes.

I have used **Moq** to mock dependencies in unit testing.

Example:

```
var mockRepo = new Mock<IOrderRepository>();
mockRepo.Setup(r => r.GetById(It.IsAny<int>()))
        .Returns(new Order());
```

Why use Moq?

- Isolate business logic
- Avoid hitting real database
- Control test behavior
- Simulate edge cases

Moq enables proper dependency injection-based testing.

---

4 Have you implemented automation testing?

Yes.

I have experience with:

◈ Backend Automation

- Unit tests with NUnit/xUnit
- Integration tests for APIs

◈ CI/CD Integration

- Running tests in pipeline
- Failing build if tests fail
- Enforcing minimum coverage

In some projects, automation also included:

- API automation (Postman/Newman)
- Regression test execution in pipeline

Automation ensures quality at scale.

---

5 Why unit testing is important?

Unit testing is important because:

◈ 1. Prevents Regression

When code changes, tests ensure nothing breaks.

◈ 2. Improves Design

Testable code is usually:

- Loosely coupled
- Cleanly structured
- Follows SOLID principles

◈ 3. Speeds Up Development

Developers catch bugs early.

◈ 4. Supports Refactoring

You can refactor confidently if tests exist.

◈ 5. Documentation

Tests act as living documentation of expected behavior.

Without unit tests:

- Bugs reach production
- Refactoring becomes risky
- Maintenance cost increases

🔥 Strong Senior-Level Closing Line

"I use unit testing not just for coverage, but to enforce clean architecture, isolate business logic, prevent regressions, and enable safe refactoring in enterprise-grade applications."

• Should we use subqueries? • Should we use cursors? • What are drawbacks of cursors? • What should replace subqueries? • Should we use CTE? • Should we use temp tables? • What is read-only cursor? • What is isolation level? • What is NOLOCK? • What is dirty read? • What is data integrity? • When to use transactions? • What are isolation levels?

Excellent — this is a **SQL performance + transaction maturity round**. I'll answer clearly and practically like in a senior DBA/Backend interview.

---

1 Should we use subqueries?

Yes — but carefully.

Subqueries are not bad by default.

Use subqueries when:

- Logic is simple
- Improves readability
- Small dataset

Avoid when:

- Nested deeply
- Executed repeatedly for each row
- Causing performance issues

Sometimes JOIN performs better than correlated subquery.

So rule is:
✔ Use when appropriate
✕ Avoid when it causes repeated execution per row

---

2 Should we use cursors?

Generally, NO.

Cursors process rows one by one (row-by-row processing).

SQL is designed for set-based operations.

Cursors are slow for large datasets.

Use cursors only when:

- Row-by-row logic is unavoidable
- Complex sequential processing needed

---

3 What are drawbacks of cursors?

Drawbacks:

- Very slow for large data
- High memory usage
- Locking overhead
- Reduced scalability
- Complex code

Set-based queries are almost always faster.

Cursors break SQL's strength (set-based processing).

---

4 What should replace subqueries?

Better alternatives:

- JOIN
- CTE
- Derived tables
- Temp tables
- Window functions

Example:

Instead of correlated subquery → use JOIN.

JOIN is usually more optimized.

---

5 Should we use CTE?

Yes — when:

- Query readability matters
- Breaking complex query into steps
- Recursive queries

CTE improves clarity.

But:

CTE does not store data physically.
It behaves like inline view.

For large reuse scenarios → temp table may be better.

---

6 Should we use temp tables?

Yes — when:

- Large intermediate results
- Complex transformations
- Reusing dataset multiple times
- Improving query plan control

Temp tables are stored in tempdb.

They can improve performance when reused multiple times.

---

7 What is read-only cursor?

Read-only cursor:

- Cannot update data
- Only fetch rows
- Faster than updatable cursor

Used when iteration needed but no modification.

Still slower than set-based queries.

---

8 What is isolation level?

Isolation level defines:

How transactions are isolated from each other.

It controls:

- Visibility of uncommitted data
- Locking behavior
- Concurrency vs consistency tradeoff

---

9 What is NOLOCK?

NOLOCK allows reading uncommitted data.

Equivalent to:

```
READ UNCOMMITTED
```

It avoids shared locks.

But can cause:

- Dirty reads
- Phantom reads
- Inconsistent data

Should be used cautiously.

---

10 What is dirty read?

Dirty read happens when:

Transaction A reads data modified by Transaction B
Before Transaction B commits.

If B rolls back → A read invalid data.

Occurs in:

Read Uncommitted isolation.

---

11 What is data integrity?

Data integrity ensures:

- Accuracy
- Consistency
- Validity of data

Maintained through:

- Constraints (Primary key, Foreign key)
- Transactions
- Validation logic
- Proper isolation levels

Without integrity, data becomes unreliable.

---

12 When to use transactions?

Use transactions when:

- Multiple operations must succeed or fail together
- Financial transactions
- Inventory updates
- Data consistency critical

Transactions ensure:

- ✔ Atomicity
- ✔ Consistency
- ✔ Isolation
- ✔ Durability

(ACID properties)

---

13 What are isolation levels?

SQL Server isolation levels:

1. Read Uncommitted
2. Read Committed (default)
3. Repeatable Read
4. Serializable
5. Snapshot

Higher isolation:

- More consistency
- More locking
- Less concurrency

Quick Summary Table

Level	Dirty Read	Non-Repeatable	Phantom
Read Uncommitted	Yes	Yes	Yes
Read Committed	No	Yes	Yes
Repeatable Read	No	No	Yes
Serializable	No	No	No

🔥 Strong Senior-Level Closing Statement

"In SQL, I prefer set-based operations over cursors, use JOIN instead of heavy correlated subqueries, apply temp tables strategically for large datasets, and choose appropriate isolation levels to balance performance with data integrity."

• What type of API interactions have you had? • Have you worked with third-party APIs? • How do you set up APIs? • Have you used: o Ocelot? o Azure API Management? o Kubernetes Ingress? • What's your experience with API Gateway? • How do you expose APIs externally? 3 Monolith to Microservices Migration • Have you worked on breaking down monolithic applications? • Can you give an example? • What technologies were involved in the legacy system? • How did you migrate VB.NET / legacy components? • How did you divide databases? • How did you implement service communication? • Did you use Service Bus? • Did you use RabbitMQ? • What challenges did you face? • What benefits did you see?

Excellent — this is a **senior architecture + integration maturity round**. I'll structure it cleanly so you can answer confidently.

◈ API Experience & Gateway Architecture

---

1 What type of API interactions have you had?

I have worked with:

- Internal microservice-to-microservice APIs
- External partner APIs
- Third-party integrations (payment, ERP, SAP)
- RESTful APIs with JWT authentication
- Event-driven APIs using Service Bus

Interactions included:

- CRUD operations
- Webhooks
- OAuth2-based authentication
- File upload/download APIs
- High-volume transactional APIs

---

2 Have you worked with third-party APIs?

Yes.

Examples:

- Payment gateways
- SAP integration via MuleSoft
- External vendor systems
- Azure AD authentication endpoints

Typical implementation steps:

1. Study API documentation
2. Implement HTTP client with retry policies
3. Handle authentication (OAuth2 / API keys)
4. Implement timeout & circuit breaker
5. Log correlation ID

Always wrap third-party calls in a service abstraction.

---

3 How do you set up APIs?

Steps:

1. Create Controller
2. Define routes
3. Add DTOs
4. Implement business logic layer
5. Add authentication middleware
6. Add validation
7. Add exception handling
8. Configure Swagger
9. Deploy behind API Gateway

Follow REST standards:

- Use proper HTTP verbs
- Return correct status codes
- Version APIs

---

4 Have you used Ocelot?

Yes (if applicable in your context — adjust if needed).

Ocelot is a lightweight .NET API Gateway.

Used for:

- Routing
- Aggregation
- Load balancing
- Authentication delegation

Good for microservices in .NET ecosystem.

---

5 Have you used Azure API Management?

Yes.

Azure APIM was used as enterprise-grade API Gateway.

It handled:

- Centralized authentication
- Rate limiting
- IP filtering
- API versioning
- Monitoring
- Policy enforcement
- Request transformation

Flow:

Client → APIM → Backend service

It provides governance and security layer.

---

6 Have you used Kubernetes Ingress?

Yes.

Ingress in AKS was used for:

- Routing traffic inside cluster
- Path-based routing
- SSL termination
- Managing internal service exposure

Flow:

External LB → Ingress → Service → Pod

Ingress handles routing; APIM handles governance.

---

7 What's your experience with API Gateway?

Experience includes:

- Azure API Management
- Ocelot
- NGINX Ingress
- Gateway-based authentication enforcement
- Rate limiting
- Caching policies
- IP whitelisting
- JWT validation at gateway

API Gateway centralizes:

- Security
- Logging
- Monitoring
- Traffic control

Without gateway, each microservice must handle these individually.

---

8 How do you expose APIs externally?

Options:

- Azure API Management
- Kubernetes Ingress + LoadBalancer
- Reverse Proxy (NGINX)

- Azure Application Gateway
- Azure Front Door

Best practice:

Expose only API Gateway publicly.
Keep microservices internal.

◈ Monolith to Microservices Migration

---

9 Have you worked on breaking down monolithic applications?

Yes.

I worked on migrating a monolithic ERP-style application into domain-based microservices.

We followed Strangler Pattern — gradual extraction.

---

10 Can you give an example?

Example:

Monolith had:

- Order module
- Vendor module
- Billing module
- Reporting module

We extracted:

- Order Service
- Vendor Service
- Billing Service
- Notification Service

Each with independent database.

---

11 What technologies were involved in the legacy system?

Legacy system involved:

- VB.NET
- Older ASP.NET Web Forms
- SQL Server
- Windows Services
- Shared database architecture

It was tightly coupled and database-heavy.

---

12 How did you migrate VB.NET / legacy components?

Approach:

1. Identify business logic
2. Wrap legacy components behind API
3. Gradually rewrite in modern .NET
4. Validate outputs match legacy behavior
5. Decommission old modules

We did not rewrite everything at once.

We migrated module by module.

---

13 How did you divide databases?

Initially:

- Shared database

After migration:

- Database per service
- Removed cross-table joins
- Replaced joins with API calls or events
- Introduced read models

Domain ownership was key.

---

14 How did you implement service communication?

Hybrid model:

- REST for synchronous validation
- Azure Service Bus for async workflows

Example:
Order created → Publish event → Billing Service consumes → Inventory updates

---

15 Did you use Service Bus?

Yes.

Used for:

- Asynchronous communication
- Saga workflows
- Dead-letter handling
- Event publishing

It improved decoupling and resilience.

---

16 Did you use RabbitMQ?

Yes (in earlier systems).

Used for:

- Task processing
- Event broadcasting
- Manual acknowledgment handling

Later moved to Azure Service Bus for managed cloud benefits.

---

17 What challenges did you face?

Major challenges:

- Tight database coupling
- Shared transactions
- Cross-module joins
- Data consistency
- Monitoring distributed logs
- Replication lag
- Cultural shift in team mindset

Breaking database dependency was hardest.

---

18 What benefits did you see?

After migration:

- Independent scaling
- Faster deployments
- Improved fault isolation
- Better performance tuning
- Clear domain ownership
- Easier future enhancements
- Better observability

System became more resilient and maintainable.

🔥 Strong Senior-Level Closing Statement

"I have hands-on experience in designing and exposing APIs through API gateways, integrating third-party systems securely, and leading incremental monolith-to-microservices migration using domain-driven separation, asynchronous messaging, and database segregation to achieve scalability and resilience."

• What backend databases have you worked with? • Have you worked with performance tuning? • How do you analyze execution plans? • How do you handle index tuning? • How do you handle connection pooling? • What are IO-bound vs CPU-bound queries? • What are table scans vs index scans? • How do you optimize stored procedures?

Excellent — this is a **serious backend performance round**. I'll answer like a senior backend engineer with real production exposure.

◈ 1 What backend databases have you worked with?

I have worked with:

- **SQL Server** (extensively)
- **Azure SQL Database**
- **PostgreSQL**
- Redis (for caching)
- Azure Blob Storage (for file storage)

Primary transactional systems were built on SQL Server and Azure SQL.

I have worked on:

- Schema design
- Stored procedures
- Index optimization
- Multi-tenant database design
- Performance troubleshooting

◈ 2 Have you worked with performance tuning?

Yes — performance tuning was part of production support.

I have handled:

- Slow queries
- Blocking issues
- High CPU usage
- IO bottlenecks
- Parameter sniffing problems
- Missing index issues
- Large table performance issues

Tuning approach always starts with **measurement, not assumptions**.

◈ 3 How do you analyze execution plans?

Steps I follow:

1. Enable **Actual Execution Plan** in SSMS.
2. Identify high-cost operators.
3. Look for:
    - Table scans
    - Index scans
    - Key lookups
    - Missing index suggestions
    - Expensive joins (Hash Match)
4. Compare estimated vs actual row counts.
5. Check warnings (spill to tempdb, implicit conversion).

Execution plan shows:

- How SQL retrieves data
- Where time is spent
- Which indexes are used

◈ 4 How do you handle index tuning?

Index tuning steps:

1. Identify slow query.
2. Analyze filtering and join columns.
3. Add index on:
    - WHERE columns
    - JOIN columns
    - ORDER BY columns

Consider:

- Covering index (include columns)
- Composite index order
- Avoid over-indexing
- Monitor index fragmentation
- Rebuild/reorganize indexes

Always verify improvement using execution plan.

Indexes improve reads but slow down writes — balance is important.

◈ 5 How do you handle connection pooling?

Connection pooling is managed by ADO.NET automatically.

Best practices:

- Always use `using` statement
- Close connections properly
- Avoid long-running open connections
- Avoid creating new connection per row
- Reuse connection string exactly (same format)

Improper connection management can exhaust pool.

In high-load systems, connection pool size can be configured.

◈ 6 What are IO-bound vs CPU-bound queries?

◈ IO-bound Query

- Waiting on disk reads
- Large scans
- Reading many pages from disk
- High physical reads

Example:
Full table scan on large table.

◈ CPU-bound Query

- Complex calculations
- Large joins
- Sorting large datasets

- Aggregations

High CPU usage, low IO.

Identifying this helps decide:

- Add index (for IO)
- Optimize logic (for CPU)

◈ 7 What are table scans vs index scans?

◈ Table Scan

- Entire table is read
- Happens when no suitable index exists
- Expensive for large tables

◈ Index Scan

- Entire index is read
- Better than table scan
- Still scans many rows

◈ Index Seek (Best Case)

- Directly jumps to required rows
- Most efficient

Goal is usually:
✔ Index Seek

◈ 8 How do you optimize stored procedures?

Steps I follow:

◈ 1. Analyze execution plan

◈ 2. Remove SELECT *

◈ 3. Add appropriate indexes

◈ 4. Avoid cursors (use set-based logic)

◈ 5. Avoid correlated subqueries

◈ 6. Handle parameter sniffing

◈ 7. Use proper filtering

◈ 8. Break complex queries into temp tables (if needed)

◈ 9. Avoid implicit conversions

◈ 10. Check statistics updates

If parameter sniffing issue:

- Use `OPTION (RECOMPILE)`
- Use local variables
- Or optimize plan strategy

Always test before and after optimization.

🔥 Strong Senior-Level Closing Statement

"I approach database performance scientifically — analyze execution plans, identify IO vs CPU bottlenecks, apply proper indexing strategy, optimize stored procedures using set-based logic, and monitor connection pooling to ensure scalable backend performance."

• Have you worked on migrating an ASP.NET application from one server to another? • Have you migrated from on-prem to cloud? • Explain the steps to migrate an ASP.NET application from on-prem to cloud. • Explain the steps to migrate from one server to another (non-cloud). • What hardware/software prerequisites do you check? • What firewall/network configuration do you validate? • What SDK dependencies do you verify? • How do you handle logging paths? • How do you configure CI/CD for new server?

Excellent — this is a **real DevOps + migration maturity round**. I'll answer step-by-step like a senior engineer who has done real migrations.

◈ 1 Have you worked on migrating an ASP.NET application from one server to another?

Yes.

I have handled:

- IIS-based ASP.NET deployments
- Migrating applications between Windows servers
- Environment replication
- Configuration updates
- Production cutover validation

Migration includes both application and infrastructure readiness.

◈ 2 Have you migrated from on-prem to cloud?

Yes.

I have worked on migrating ASP.NET applications from on-prem Windows Server environments to:

- Azure App Services
- Azure Virtual Machines
- AKS (containerized workloads)

Cloud migration required architectural adjustments, not just server copy.

◈ 3 Steps to migrate ASP.NET application from on-prem to cloud

◈ Step 1: Assessment

- Identify .NET version

- Identify IIS dependencies
- Identify third-party libraries
- Check database compatibility
- Evaluate session state usage

◈ Step 2: Decide Migration Strategy (6R Strategy)

- Rehost (lift & shift)
- Refactor
- Re-architect
- Rebuild
- Replace
- Retire

Most ASP.NET apps → Rehost or Refactor.

◈ Step 3: Infrastructure Setup

- Create Azure App Service / VM
- Configure networking (VNet)
- Configure SQL Database
- Setup storage (Blob)
- Setup Key Vault

◈ Step 4: Configuration Changes

- Update connection strings
- Remove local file path dependencies
- Replace file storage with Blob storage
- Move secrets to Key Vault
- Enable HTTPS

◈ Step 5: Logging Configuration

- Replace local logs with:
  - Azure Application Insights
  - Centralized logging
- Avoid writing logs to C:\ drive

◈ Step 6: Testing

- Smoke testing
- Integration testing
- Performance validation

◈ Step 7: Cutover

- DNS switch
- Monitor application
- Rollback plan ready

◈ 4 Steps to migrate from one server to another (Non-cloud)

◈ 1. Install IIS on new server

◈ 2. Install correct .NET runtime

◈ 3. Copy application files

◈ 4. Configure application pool

◈ 5. Set correct app pool identity

◈ 6. Update connection strings

◈ 7. Configure firewall rules

◈ 8. Test locally

◈ 9. Update DNS or load balancer

Always verify:

- Folder permissions
- App pool settings
- SSL bindings

◈ 5 Hardware / Software prerequisites to check

◈ Hardware

- CPU cores
- RAM
- Disk space
- Disk speed (SSD preferred)

◈ Software

- Windows Server version
- IIS version
- .NET runtime version
- SQL Server compatibility
- Required SDKs
- VC++ redistributables

Mismatch in runtime version can break application.

◈ 6 Firewall / Network configuration validation

Check:

- Port 80 / 443 open
- SQL port 1433 open
- Outbound access rules
- VNet integration (if cloud)
- IP whitelisting
- Load balancer health probes

Without correct firewall rules, application may deploy but not function.

◈ 7 SDK dependencies verification

Verify:

- .NET SDK version
- Target framework compatibility

- NuGet package versions
- Runtime dependencies
- Hosting bundle installed (for IIS)
- Environment variables

Sometimes missing hosting bundle causes 500.30 error.

◈ 8 How do you handle logging paths?

On-prem:

- Avoid hardcoded paths
- Use relative paths
- Configure via appsettings

Cloud:

- Use Application Insights
- Use centralized logging
- Avoid writing logs to local disk

Never depend on C:\Logs folder in cloud environment.

◈ 9 How do you configure CI/CD for new server?

Steps:

◈ 1. Setup pipeline (Azure DevOps / GitHub Actions)

◈ 2. Configure build steps

- Restore
- Build
- Publish

◈ 3. Configure release pipeline

- Deploy to IIS
- Deploy to App Service

- Use environment variables
- Use secure variable groups

◈ 4. Configure:

- Slot deployment (for zero downtime)
- Health checks
- Auto rollback if deployment fails

CI/CD ensures:

- No manual deployment
- Version tracking
- Repeatability

🔥 Strong Senior-Level Closing Statement

"I approach migrations systematically — assessing runtime compatibility, infrastructure readiness, firewall configuration, logging architecture, and CI/CD automation — ensuring minimal downtime, secure deployment, and production stability during server or cloud migration."

• How do you configure HTTPS in IIS? • How do you install SSL certificate? • How do you export/import certificate? • How do you verify certificate isolation? • How do you bind certificate to IIS? • What happens if certificate is not configured properly?

Excellent — this is a **real infrastructure + IIS + SSL practical round**. I'll answer step-by-step like someone who has actually configured production HTTPS.

◈ 1⃣ How do you configure HTTPS in IIS?

Steps to configure HTTPS in IIS:

✅ Step 1: Install SSL Certificate on Server

Certificate must be installed in:

```
Local Computer → Personal → Certificates
```

✅ Step 2: Open IIS Manager

- Go to **Sites**
- Select your website

✅ Step 3: Click "Bindings"

- Click **Add**
- Select:
  - Type: HTTPS
  - Port: 443
  - Select installed SSL certificate

✅ Step 4: Restart IIS (if required)

Now HTTPS is configured.

🔷 2 How do you install SSL certificate?

There are two common ways:

🔷 Option A: Install via Certificate File (.pfx)

1. Double-click .pfx file
2. Choose "Local Machine"
3. Enter password
4. Store in "Personal" certificate store

OR

Using MMC:

- Run `mmc`
- Add Certificates snap-in
- Import certificate manually

◈ Option B: Install via CSR (Certificate Signing Request)

1. Generate CSR from IIS
2. Submit CSR to CA
3. Receive certificate file
4. Complete certificate request in IIS

◈ 3 How do you export/import certificate?

◈ Export Certificate

1. Open MMC → Certificates
2. Right-click certificate
3. Export
4. Include private key
5. Save as `.pfx`
6. Set password

◈ Import Certificate

1. Open MMC
2. Import `.pfx`
3. Enter password
4. Store in Local Machine → Personal

Important:
Without private key → HTTPS won't work.

◈ 4 How do you verify certificate isolation?

Verify:

- Certificate installed under **Local Computer**, not Current User
- Private key is attached
- Only correct app pool identity has access
- Check certificate thumbprint

- Validate certificate subject name (CN or SAN)

In multi-tenant servers:

- Ensure correct certificate bound to correct site
- Avoid wildcard misconfiguration

You can verify using:

```
certlm.msc
```

◈ 5 How do you bind certificate to IIS?

Steps:

1. Open IIS
2. Select Website
3. Click “Bindings”
4. Add HTTPS binding
5. Select certificate
6. Assign correct hostname
7. Click OK

For multiple sites:

- Use SNI (Server Name Indication)
- Enable “Require Server Name Indication”

◈ 6 What happens if certificate is not configured properly?

Several issues can happen:

✕ 1. Browser shows:

- “Not Secure”
- “Certificate Invalid”
- “NET::ERR_CERT_COMMON_NAME_INVALID”

❌ 2. HTTPS not working

- 443 port not bound
- Certificate not found

❌ 3. 500 Internal Server Error

- Private key permission issue

❌ 4. SSL Handshake Failure

- TLS version mismatch
- Expired certificate
- Incorrect certificate chain

❌ 5. Site loads HTTP but not HTTPS

- No HTTPS binding
- Firewall blocking port 443

🔥 Senior-Level Troubleshooting Checklist

When HTTPS fails, I check:

- Certificate expiration
- Certificate CN/SAN matches domain
- Private key exists
- Correct certificate store
- Port 443 open
- IIS binding configured
- TLS 1.2 enabled
- Intermediate certificate chain installed

🔥 Strong Interview Closing Statement

"Configuring HTTPS in IIS involves proper certificate installation with private key, correct binding to port 443, verifying domain match and certificate chain, and ensuring firewall and TLS configuration are correct to prevent handshake or security errors."

• How do you identify dependencies before redeployment? • What if no documentation exists? • How do you analyze Web.config? • How do you check connection strings? • How do you detect third-party libraries? • How do you detect required ports? • How do you identify required services? • How do you discover external API calls? • How do you analyze .csproj for dependencies? _____ 8 Database Migration • What is the best way to migrate a database to another server? • Would you use backup & restore? • Would you script schema? • How do you install SQL Server on new machine? • What are differences between image-based migration vs manual? • How do you migrate users/logins? • How do you validate database after migration? _____

Excellent — this is a **serious production migration + dependency analysis round**. I'll answer like a senior engineer preparing for a real environment cutover.

◈ Application Redeployment – Dependency Discovery

---

1 How do you identify dependencies before redeployment?

Before redeployment, I perform a structured dependency audit:

✅ 1. Application-level dependencies

- Web.config / appsettings.json
- Connection strings
- API endpoints
- File paths
- Certificates

✅ 2. Infrastructure dependencies

- Database server
- Redis
- Message queues
- Windows services
- Scheduled tasks

✅ 3. Network dependencies

- Open ports
- Firewall rules
- External API endpoints

✅ 4. Runtime dependencies

- .NET version
- Hosting bundle
- IIS version
- Required SDKs

I create a deployment checklist before migration.

---

2 What if no documentation exists?

If no documentation exists, I reverse-engineer:

🔎 Steps:

1. Analyze Web.config / appsettings
2. Search solution for:
    - HttpClient usage
    - External URLs
    - Service calls
3. Check .csproj references
4. Review NuGet packages
5. Check SQL server linked servers
6. Check Windows Services on server
7. Analyze event logs
8. Monitor outbound traffic using tools like:
    - netstat
    - Fiddler
    - Wireshark

Treat it like forensic analysis.

---

3 How do you analyze Web.config?

I check:

- `<connectionStrings>`
- `<appSettings>`
- `<system.web>`
- `<system.webServer>`
- Custom sections
- Logging paths

- SMTP settings
- Authentication configuration
- API endpoints
- File upload paths

Especially:

```
<connectionStrings>
```

These tell me external DB dependencies.

---

4 How do you check connection strings?

I verify:

- Server name
- Database name
- Authentication mode (SQL / Windows)
- Encrypt=True?
- Trusted_Connection?
- MultipleActiveResultSets?

Then:

- Test connectivity from new server
- Ensure firewall allows access
- Confirm login exists on SQL Server

---

5 How do you detect third-party libraries?

Check:

- NuGet packages in solution
- .csproj file
- bin folder DLLs
- packages.config (older projects)
- Startup.cs for middleware

Also look for:

- Payment gateways
- PDF generators

- Reporting engines
- COM components

---

6 How do you detect required ports?

Check:

- Web server ports (80, 443)
- SQL Server (1433)
- Redis (6379)
- Service Bus ports
- SMTP (25 / 587)

Tools:

```
netstat -an
```

Also review firewall rules and network security groups (NSG in Azure).

---

7 How do you identify required services?

Check:

- Windows Services (services.msc)
- Scheduled tasks
- IIS Application Pools
- Background workers
- Event logs

Sometimes application depends on:

- Windows service for processing
- File watcher service
- Message queue listener

---

8 How do you discover external API calls?

Search codebase for:

- HttpClient

- WebRequest
- RestSharp
- Base URLs in config

Search for:

```
http://
https://
```

Also inspect:

- Startup configuration
- Dependency injection registration

Can also monitor outgoing traffic via Fiddler.

---

9 How do you analyze .csproj for dependencies?

Open .csproj and check:

- `<PackageReference>`
- `<ProjectReference>`
- TargetFramework
- SDK version
- Build targets

Example:

```
<PackageReference Include="Newtonsoft.Json" Version="13.0.1" />
```

This reveals runtime dependencies.

Also check for:

- CopyLocal settings
- Publish profiles

---

8 Database Migration

---

10 What is the best way to migrate database?

Best method depends on size and downtime tolerance.

Most reliable:

- ✔ Backup & Restore (for full DB)
- ✔ AlwaysOn replication (for near-zero downtime)
- ✔ Log shipping
- ✔ Azure Database Migration Service (cloud)

---

11 Would you use backup & restore?

Yes — for most cases.

Steps:

1. Full backup on source
2. Copy backup file
3. Restore on target
4. Apply differential/log backup if needed

This ensures exact replica.

---

12 Would you script schema?

Use schema script when:

- Migrating schema only
- Fresh database
- Small database
- Dev/Test environment

Not ideal for large production with data.

---

13 How do you install SQL Server on new machine?

Steps:

1. Install SQL Server installer
2. Choose instance name
3. Configure authentication mode
4. Set service accounts
5. Configure TCP/IP
6. Enable port 1433
7. Install SSMS
8. Apply latest service packs

Then configure:

- Max memory
- TempDB settings
- Backup location

---

14 Differences between image-based migration vs manual?

◈ Image-based migration

- Clone entire server (VM image)
- Fast
- Risk of copying unnecessary artifacts

◈ Manual migration

- Install clean environment
- Install dependencies individually
- More control
- Safer long-term

For production → prefer clean manual setup.

---

15 How do you migrate users/logins?

Important step — often missed.

SQL Server has:

- Server-level logins
- Database-level users

Use:

```
sp_help_revlogin
```

To script logins with SID.

Otherwise:
Users become orphaned after restore.

Fix orphaned users:

```
ALTER USER username WITH LOGIN = loginname;
```

---

16 How do you validate database after migration?

Validation steps:

- Compare row counts
- Check table sizes
- Verify indexes
- Run DBCC CHECKDB
- Validate constraints
- Run smoke test queries
- Test application connectivity

For critical systems:

- Compare checksum/hash
- Validate performance baseline

🔥 Strong Senior-Level Closing Statement

"I approach redeployment and database migration systematically — performing dependency discovery, configuration validation, network checks, login migration, and post-migration validation to ensure zero surprises during production cutover."

• If Enterprise Library blocks are used, how do you migrate them? • How do you upgrade old NuGet packages? • How do you check compatibility? • How do you validate application after package upgrade? • What is your approach when upgrading legacy dependencies?

Excellent — this is a **legacy modernization + dependency upgrade maturity round**. I'll answer like someone who has handled real legacy systems.

---

1 If Enterprise Library blocks are used, how do you migrate them?

Enterprise Library (Microsoft Enterprise Library) is outdated and not supported in modern .NET Core.

Step 1: Identify Used Blocks

Common blocks:

- Logging Application Block
- Data Access Application Block
- Exception Handling Block
- Caching Block
- Validation Block

I first identify exactly which blocks are being used.

Step 2: Replace with Modern Equivalents

Enterprise Library	Modern Replacement
Logging Block	Serilog / NLog / ILogger
Data Access Block	EF Core / Dapper
Exception Block	Middleware-based global handling
Caching Block	Redis / IMemoryCache
Validation Block	FluentValidation

Step 3: Incremental Refactoring

Do NOT rewrite everything at once.

Approach:

1. Wrap old Enterprise Library usage in an abstraction layer.
2. Replace internal implementation gradually.
3. Remove Enterprise Library dependency.
4. Regression test.

This avoids breaking production.

◈ 2 How do you upgrade old NuGet packages?

Structured approach:

✅ Step 1: Identify outdated packages

Use:

```
dotnet list package --outdated
```

Or Visual Studio NuGet manager.

✅ Step 2: Check release notes

Always check:

- Breaking changes
- Deprecated APIs
- Migration guide

✅ Step 3: Upgrade one package at a time

Avoid bulk upgrading everything at once.

Upgrade in stages:

- Core framework first
- Then supporting packages

✅ Step 4: Rebuild & run tests

After each upgrade:

- Build
- Run unit tests

- Run integration tests

🔷 3 How do you check compatibility?

Check compatibility using:

🔍 1. Target Framework

Ensure package supports:

- net6.0
- net7.0
- net8.0

🔍 2. NuGet metadata

Check supported frameworks in:
nuget.org

🔍 3. Check for API breaking changes

Search:

- Obsolete attributes
- Deprecated methods
- Changed method signatures

🔍 4. Check transitive dependencies

Sometimes upgrading one package upgrades others.

Use:

```
dotnet list package --include-transitive
```

◈ 4 How do you validate application after package upgrade?

Validation steps:

◈ 1. Build validation

- Fix compile errors

◈ 2. Unit testing

- Run test suite

◈ 3. Manual smoke testing

- Login
- Critical workflows

◈ 4. Regression testing

- Business-critical flows

◈ 5. Performance testing

- Check for query plan changes
- Monitor memory usage

◈ 6. Log monitoring

- Check for runtime warnings

Never push upgrade directly to production.

◈ 5 What is your approach when upgrading legacy dependencies?

My approach is:

⍰ Step 1: Risk Assessment

- Is package security vulnerable?

- Is it deprecated?
- Is it blocking framework upgrade?

⚠ Step 2: Dependency Mapping

Understand:

- Where is it used?
- How deeply integrated?

🛠 Step 3: Introduce Abstraction

If legacy package heavily used:

- Wrap it in interface
- Replace implementation internally

🔄 Step 4: Upgrade in Small Steps

- Upgrade minor versions first
- Then major versions
- Avoid jumping multiple major versions at once

⚠ Step 5: Test Thoroughly

- Unit tests
- Integration tests
- UAT
- Staging validation

🚀 Step 6: Deploy Safely

- Use deployment slots
- Monitor logs
- Rollback plan ready

🔥 Senior-Level Closing Statement

"When upgrading legacy dependencies like Enterprise Library or old NuGet packages, I follow a controlled, incremental modernization strategy — replacing outdated blocks with modern equivalents, validating compatibility carefully, and ensuring stability through structured testing and staged deployment."

• What differences do you see between Web Forms and MVC? • What is tightly coupled architecture? • What is postback? • Why is MVC more modular? • Why is MVC more testable? • What are maintainability benefits? _____

Excellent — this is a **legacy ASP.NET understanding + architecture maturity** round. I'll answer clearly and in interview-ready format.

---

1 What differences do you see between Web Forms and MVC?

◈ Web Forms

- Event-driven model
- Uses ViewState
- Code-behind architecture
- Heavy server controls
- Postback-based lifecycle
- Tightly coupled UI + logic
- Harder to unit test

It tries to simulate desktop programming model.

◈ MVC

- Pattern-based architecture (Model-View-Controller)
- No ViewState
- Clear separation of concerns
- Full control over HTML
- REST-friendly
- More lightweight
- Testable
- SEO friendly

Web Forms = Web pretending to be Windows app
MVC = True web architecture

---

2 What is tightly coupled architecture?

Tightly coupled architecture means:

- Components depend directly on concrete implementations
- Changing one component affects others
- Hard to test
- Hard to maintain

Example:

Controller directly creating SqlConnection inside it.

If DB changes → controller must change.

Loose coupling uses interfaces and DI.

---

3 What is postback?

Postback is when:

- Entire page is submitted back to server
- Server processes event
- Entire page reloads

In Web Forms:

- Clicking a button triggers postback
- Page lifecycle executes again

Postback relies heavily on ViewState.

It causes:

- Larger page size
- Performance issues
- Complex debugging

---

4 Why is MVC more modular?

MVC separates application into:

- Model (business logic)
- View (UI)
- Controller (request handling)

Each has clear responsibility.

Because of separation:

- UI changes don't affect business logic
- Business logic reusable
- Independent development possible

Modularity reduces coupling.

---

5 Why is MVC more testable?

MVC supports:

- Dependency Injection
- Controller unit testing
- Mocking services
- No UI dependency

In Web Forms:

- Code-behind tightly coupled to UI
- Hard to isolate logic
- Hard to mock controls

In MVC:

You can test controller logic independently.

---

6 What are maintainability benefits?

MVC provides:

◈ Clear structure

- Easier to understand

◈ Separation of concerns

- Easier debugging

◈ Better scalability

- Easy to extend features

◈ Cleaner architecture

- Follows SOLID principles

◈ Better for large teams

- UI team + backend team can work independently

Over time:

Web Forms becomes difficult to maintain
MVC scales better for enterprise systems

🔥 Strong Interview Closing Line

"Web Forms is event-driven and tightly coupled with postback-based lifecycle, while MVC follows separation of concerns, enabling modular, testable, and maintainable architecture suitable for scalable enterprise applications."

• What is Service-Oriented Architecture? • What is difference between SOA and REST? • How do you design service-oriented applications? • How would you implement it? • What patterns would you use? _____

Excellent — this is an **architecture-level conceptual round**. I'll answer clearly and at senior-engineer level depth.

---

1 What is Service-Oriented Architecture (SOA)?

Service-Oriented Architecture (SOA) is an architectural style where:

- Application is divided into **independent services**
- Each service provides a specific business capability
- Services communicate over a network

Key characteristics:

- Loose coupling
- Reusable services
- Standardized communication protocols
- Service contracts (WSDL, REST, etc.)

SOA focuses on **business capabilities as services**.

Example:

- Order Service
- Payment Service
- Inventory Service

Each service can be consumed by multiple applications.

---

2 What is difference between SOA and REST?

Important clarification:

SOA is an **architecture style**
REST is a **communication protocol/constraint**

◈ SOA

- Architectural approach
- Can use SOAP or REST
- Often enterprise-level
- May use ESB (Enterprise Service Bus)

◈ REST

- Lightweight HTTP-based API style
- Resource-oriented

- Stateless
- Uses JSON/XML

So:

SOA defines how services are structured.
REST defines how services communicate.

REST can be used inside SOA.

---

3 How do you design service-oriented applications?

Steps I follow:

◈ Step 1: Identify Business Capabilities

Use Domain-Driven Design (DDD).

Example:

- Customer Management
- Order Processing
- Billing
- Notification

Each becomes a service.

◈ Step 2: Define Service Boundaries

Each service should:

- Own its data
- Have clear responsibility
- Avoid tight coupling

◈ Step 3: Define Contracts

Services expose:

- REST APIs
- Message-based interfaces
- Versioned endpoints

Contracts must be stable.

◈ Step 4: Choose Communication Style

- Synchronous (REST/gRPC)
- Asynchronous (Service Bus, RabbitMQ)

Hybrid approach is common.

◈ Step 5: Implement Cross-Cutting Concerns

- Authentication
- Logging
- Monitoring
- Error handling
- Caching

Prefer centralized handling via API Gateway.

---

4 How would you implement it?

Implementation approach:

◈ Technology Stack

- ASP.NET Core Web APIs
- Azure Service Bus (for async)
- SQL Database per service
- API Gateway (APIM)
- Redis cache
- JWT-based authentication

◈ Service Ownership

Each service:

- Has its own database
- Is independently deployable
- Has its own CI/CD pipeline

◈ Example Flow

Order Created →
Publish Event →
Billing Service Consumes →
Inventory Updates →
Notification Sent

Event-driven ensures loose coupling.

---

5 What patterns would you use?

In Service-Oriented Architecture, I would use:

◈ 1. Repository Pattern

For data abstraction.

◈ 2. Dependency Injection

For loose coupling.

◈ 3. CQRS

Separate read and write workloads.

◈ 4. Saga Pattern

For distributed transactions.

◈ 5. Circuit Breaker

To handle service failures.

◈ 6. API Gateway Pattern

For centralized access control.

◈ 7. Strangler Pattern

For monolith-to-service migration.

◈ 8. Event-Driven Architecture

For asynchronous workflows.

🔥 Strong Senior-Level Closing Statement

"Service-Oriented Architecture is about designing business capabilities as loosely coupled, reusable services with well-defined contracts. I implement it using domain-driven boundaries, independent databases, asynchronous messaging, and architectural patterns like CQRS, Saga, and API Gateway to ensure scalability and resilience."

• How do you identify database bottlenecks? • How do you analyze execution plan? • What is IO-bound vs CPU-bound query? • What is table scan vs index seek? • When should we add non-clustered index? • When should we use CTE? • When should we use temp tables? • How do joins affect performance? • How do you reduce table scans?

Excellent — this is a **serious SQL performance deep-dive round**. I'll answer like a senior backend engineer who has handled real production bottlenecks.

---

1 How do you identify database bottlenecks?

I use a structured approach:

◈ Step 1: Monitor Metrics

- High CPU usage
- High disk IO
- Long-running queries
- Blocking sessions
- Deadlocks

Tools:

- SQL Server Activity Monitor
- Azure Query Performance Insight
- DMVs (`sys.dm_exec_requests`, `sys.dm_exec_query_stats`)
- Execution plans

◈ Step 2: Identify Slow Queries

Check:

- Top resource-consuming queries
- High logical reads
- High execution count
- High duration

◈ Step 3: Analyze Execution Plan

Look for:

- Table scans
- Index scans
- Key lookups
- Hash joins
- Missing index warnings
- Implicit conversions

Then optimize accordingly.

---

2 How do you analyze execution plan?

Steps:

1. Enable **Actual Execution Plan**
2. Identify operators with highest cost
3. Check:
    - Estimated vs actual rows
    - Table scan vs index seek
    - Join type (Nested Loop, Hash Match, Merge)
4. Look for warnings:
    - Spill to tempdb
    - Missing index suggestion

High cost operator = bottleneck.

---

3 What is IO-bound vs CPU-bound query?

◈ IO-bound Query

- Waiting on disk reads
- High physical reads
- Full table scans
- Large data retrieval

Solution:

- Add indexes
- Reduce data scanned
- Improve filtering

◈ CPU-bound Query

- Heavy calculations
- Large joins
- Sorting
- Aggregation
- Hash joins

Solution:

- Optimize logic
- Reduce row count early
- Rewrite query

Understanding the type determines optimization strategy.

---

4 What is table scan vs index seek?

◈ Table Scan

- Entire table read
- Happens when no proper index
- Very expensive for large tables

◈ Index Seek

- SQL directly jumps to matching rows
- Uses B-tree structure
- Most efficient method

Goal is usually:
✔ Index Seek
✕ Table Scan

---

5 When should we add non-clustered index?

Add non-clustered index when:

- Column frequently used in WHERE clause
- Column used in JOIN
- Column used in ORDER BY
- Query performance is slow due to scanning

But consider:

- Too many indexes slow down inserts/updates
- Avoid indexing low-selectivity columns

Always verify via execution plan.

---

6 When should we use CTE?

Use CTE when:

- Improving readability
- Breaking complex queries into logical blocks
- Recursive queries
- One-time usage within single query

Avoid when:

- Large dataset reused multiple times

CTE does not physically store data.

---

7 When should we use temp tables?

Use temp tables when:

- Large intermediate results
- Reusing data multiple times
- Breaking complex joins
- Reducing complexity
- Need indexing on intermediate result

Temp tables can improve performance when reused.

---

8 How do joins affect performance?

Joins affect performance based on:

- Join type
- Indexed columns
- Row size
- Join order

Types:

- Nested Loop (good for small dataset)
- Hash Join (CPU heavy)

- Merge Join (requires sorted data)

Performance problems occur when:

- Joining large tables without index
- Joining on non-indexed columns
- Using functions in JOIN condition

Always ensure JOIN columns are indexed.

---

9 How do you reduce table scans?

Steps:

◈ 1. Add proper index

On filtering columns.

◈ 2. Avoid SELECT *

Select only required columns.

◈ 3. Remove functions in WHERE clause

Bad:

```
WHERE YEAR(OrderDate) = 2024
```

Good:

```
WHERE OrderDate >= '2024-01-01'
```

◈ 4. Use covering index

Include necessary columns.

◈ 5. Update statistics

Outdated stats cause poor plan.

◈ 6. Fix implicit conversions

Datatype mismatch causes scan.

🔥 Strong Senior-Level Closing Statement

"I identify database bottlenecks by analyzing execution plans and resource metrics, distinguish IO-bound vs CPU-bound queries, reduce table scans using proper indexing and filtering strategies, and use CTE or temp tables strategically to optimize complex queries."

• Difference between EF6 and EF Core? • What improvements in EF Core? • What is DBContext? • What is DBSet? • What is OnModelCreating? • What are precompiled queries? • What is Code First vs Database First? _____

Excellent — this is a **core EF + architecture maturity round**. I'll answer clearly, concisely, and in interview-ready format.

---

1 Difference between EF6 and EF Core?

◈ EF6

- Designed for .NET Framework
- Mature but no longer evolving
- Windows-only
- Heavier
- Limited modern features

◈ EF Core

- Cross-platform (.NET Core / .NET 5+)
- Lightweight and faster
- Actively maintained
- Better performance
- Designed for cloud-native apps
- Supports microservices better

🔎 Key Technical Differences

Feature	EF6	EF Core
Platform	.NET Framework only	Cross-platform

Feature	EF6	EF Core
Performance	Slower	Faster
LINQ Translation	Limited	Improved
Batch operations	Limited	Supported
Global Query Filters	❌	✅
Shadow Properties	❌	✅
Owned Types	Limited	✅
Compiled Queries	Limited	Better support

---

2 What improvements in EF Core?

Major improvements:

✅ Better Performance

- Optimized SQL generation
- Reduced overhead
- Better async support

✅ Global Query Filters

Useful for multi-tenancy.

```
modelBuilder.Entity<Order>()
    .HasQueryFilter(o => o.TenantId == _tenantId);
```

✅ Batch Updates

EF Core supports batching multiple commands in one round trip.

✅ Better LINQ Translation

More expressions converted directly to SQL.

☑ No-Tracking Optimization

```
context.Orders.AsNoTracking()
```

Improves read performance.

☑ Cross-platform Support

Works on Windows, Linux, Mac.

---

3 What is DbContext?

`DbContext` is the primary EF class that:

- Manages database connection
- Tracks entity changes
- Translates LINQ to SQL
- Saves changes to database

It acts as:

✔ Unit of Work
✔ Change tracker
✔ Data session manager

---

4 What is DbSet?

`DbSet<T>` represents a table in the database.

Example:

```
public DbSet<Order> Orders { get; set; }
```

It allows:

- Querying via LINQ
- Adding records
- Updating records
- Deleting records

Internally builds expression trees and converts to SQL.

---

5 What is OnModelCreating?

`OnModelCreating` is used to configure entity mapping using Fluent API.

Example:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Order>()
        .HasKey(o => o.Id);

    modelBuilder.Entity<Order>()
        .HasIndex(o => o.Name);
}
```

Used for:

- Primary keys
- Relationships
- Indexes
- Constraints
- Table names
- Default values
- Global filters

Runs when model is being built.

---

6 What are precompiled queries?

Normally EF compiles LINQ queries at runtime each time.

Precompiled queries:

- Compile once
- Reuse execution plan
- Reduce overhead
- Improve performance for high-frequency queries

Example:

```
var compiledQuery = EF.CompileQuery(
    (AppDbContext ctx, int id) =>
```

```
        ctx.Orders.FirstOrDefault(o => o.Id == id)
);
```

Useful in high-throughput APIs.

---

7 What is Code First vs Database First?

◈ Code First

- Define C# classes
- EF generates database
- Uses migrations
- Best for new projects

◈ Database First

- Database already exists
- EF generates models from DB
- Used in legacy systems
- Useful when DB controlled by DBA team

🔎 When to Use What?

Scenario	Approach
New project	Code First
Legacy DB	Database First
Agile schema changes	Code First
Enterprise controlled DB	Database First

🔥 Strong Senior-Level Closing Line

"EF Core improves performance, flexibility, and cloud-readiness compared to EF6. DbContext manages change tracking and persistence, DbSet represents tables,

OnModelCreating configures mappings, and precompiled queries help optimize high-frequency operations in enterprise applications."

• When should we use an abstract class? • When should we use an interface? • How do you decide during architecture? • What factors help you choose abstract class over interface? • What is the key difference? • What design situation leads you to abstract class? • How does it differ from dependency injection usage?

Excellent — this is a **true OOP architecture maturity question**, not just syntax. I'll answer clearly and in senior-level decision-making style.

---

1 When should we use an abstract class?

Use an **abstract class** when:

- You want to provide **base functionality**
- There is a **common implementation**
- There is an **"is-a" relationship**
- You want to share code among related classes
- You want to enforce a template

Example:

```
public abstract class PaymentService
{
    public void Validate()
    {
        // common validation logic
    }

    public abstract void ProcessPayment();
}
```

Here:

- All payment services share validation logic.
- Only payment processing differs.

Use abstract class when behavior is partially common.

---

2 When should we use an interface?

Use an **interface** when:

- You want to define a contract
- You want loose coupling
- You need multiple implementations
- No shared implementation is required
- You want to support dependency injection

Example:

```
public interface INotificationService
{
    void Send(string message);
}
```

Different implementations:

- EmailNotification
- SmsNotification
- PushNotification

Interface defines *what*, not *how*.

---

3 How do you decide during architecture?

I ask these questions:

◈ Is there shared logic?

If yes → abstract class.

◈ Do I only need a contract?

If yes → interface.

◈ Will multiple unrelated classes implement this?

If yes → interface.

◈ Is there an inheritance hierarchy?

If yes → abstract class.

Architecture decision is based on reuse and coupling.

---

4 What factors help you choose abstract class over interface?

Choose abstract class when:

- Shared base behavior exists
- You want to provide default implementation
- You want to control inheritance
- Classes are strongly related
- You want to avoid code duplication

Example:
All "ReportGenerators" share logging and validation logic.

Abstract class avoids duplication.

---

5 What is the key difference?

Interface	Abstract Class
Defines contract only	Can contain implementation
Supports multiple inheritance	Single inheritance only
No state (usually)	Can maintain state
Used for loose coupling	Used for base class hierarchy

Key difference:
Interface = capability
Abstract class = base behavior

---

6 What design situation leads you to abstract class?

Example situation:

You are building:

- PdfReportGenerator
- ExcelReportGenerator
- CsvReportGenerator

Common behavior:

- Logging
- Authorization
- Data validation

Different behavior:

- File format generation

Solution:
Abstract base class with shared logic.

This avoids duplication and enforces structure.

---

7 How does it differ from dependency injection usage?

Important architectural point:

In DI:

We usually depend on **interfaces**, not abstract classes.

Why?

Because:

- Interfaces provide maximum decoupling
- Easier to mock in testing
- Better for loosely coupled architecture

Abstract classes are usually used for:

- Internal hierarchy
- Base shared logic

In DI architecture:

✔ Controller depends on `IService`
✔ Concrete class implements it
✔ Abstract class may be internal base

So:

Interface → architecture boundary
Abstract class → implementation reuse

🔥 Senior-Level Summary

Use an interface when you need a contract and loose coupling.
Use an abstract class when related classes share behavior and require a common base implementation.
Interfaces define capability; abstract classes define hierarchy and partial behavior.

If you want, I can now explain **real-world architecture mistake scenarios where abstract classes cause tight coupling** — that's often asked in senior interviews.

• What design patterns have you implemented? • What is Factory Pattern? • What is Abstract Factory? • What is the difference between Factory and Abstract Factory? • When should you use Factory? • When should you use Abstract Factory? • Give real project examples.

_____

Excellent — this is a **creational design patterns depth round**. I'll answer clearly and with real-world thinking, not just textbook definitions.

---

1 What design patterns have you implemented?

In real projects, I have implemented:

- **Dependency Injection**
- **Repository Pattern**
- **Factory Pattern**
- **Abstract Factory**
- **Singleton**
- **Strategy Pattern**
- **CQRS**
- **Saga Pattern**
- **Mediator Pattern**
- **Circuit Breaker Pattern**

I use patterns based on architectural needs — not for theory.

---

2 What is Factory Pattern?

Factory Pattern is a creational pattern used to **create objects without exposing the creation logic to the client**.

Instead of:

```
var payment = new CreditCardPayment();
```

We use:

```
var payment = PaymentFactory.Create("CreditCard");
```

The factory decides which object to create.

Purpose:

- Encapsulate object creation
- Avoid `if/else` scattered in code
- Follow Open/Closed Principle

---

3 What is Abstract Factory?

Abstract Factory is used to create **families of related objects**.

It provides an interface for creating multiple related objects.

Example:

Instead of creating just one Payment type, we create:

- Payment Processor
- Payment Validator
- Payment Logger

All related to a payment type.

Abstract Factory creates the whole "family".

---

4 What is the difference between Factory and Abstract Factory?

Factory	Abstract Factory
Creates single object	Creates family of related objects
Simpler	More complex
One creation method	Multiple related creation methods
Used for one product	Used for product families

Simple Analogy:

Factory → Creates one type of car
Abstract Factory → Creates car + engine + gearbox for that brand

---

5 When should you use Factory?

Use Factory when:

- Object creation depends on condition
- You want to centralize object creation
- You want to avoid `new` scattered everywhere
- You expect future extensions

Example in real project:

Payment methods:

- Credit Card
- UPI
- PayPal

Factory selects correct payment processor.

---

6 When should you use Abstract Factory?

Use Abstract Factory when:

- Multiple related objects must be created together
- Objects must be compatible
- You want to enforce consistency across product families

Example in real project:

Multi-tenant ERP system:

Different tenant types required:

- Different Email Service
- Different Logging Service
- Different Data Repository

Instead of switching each separately, we used Abstract Factory to create:

- EmailService
- LoggingService
- Repository

All configured per tenant.

---

7 Real Project Examples

◈ Example 1: Payment Gateway (Factory)

We implemented:

```
IPaymentService payment = PaymentFactory.Create(paymentType);
```

Depending on:

- Stripe
- Razorpay
- PayPal

New gateway added without modifying existing logic.

◈ Example 2: Multi-Tenant Infrastructure (Abstract Factory)

In a SaaS system:

Enterprise tenant → Dedicated DB + Dedicated Logger
Standard tenant → Shared DB + Standard Logger

We created:

```
ITenantFactory factory = TenantFactoryResolver.Resolve(tenantType);
```

Factory produced:

- Repository
- Cache service
- Logging service

Ensured consistency across tenant configurations.

◈ Example 3: Notification Strategy (Factory + Strategy)

Notification types:

- Email
- SMS
- Push

Factory selects correct strategy implementation.

🔥 Senior-Level Architectural Summary

Factory Pattern centralizes creation of a single object type.
Abstract Factory creates families of related objects that must work together.

I use Factory for conditional object creation and Abstract Factory when building configurable, multi-component systems like multi-tenant infrastructure or environment-based configurations.

If you'd like, I can now explain **how Factory interacts with Dependency Injection in modern .NET (which is often asked at senior level).**

Open/Closed Principle – Design Challenge • If tomorrow a new notification type (e.g., WhatsApp) is added, how will your design handle it? • Will your design violate Open/Closed principle? • How will you modify design to avoid modifying existing classes? • How do you make your system extensible? _____ 5 .NET Core Advantages • What are advantages of .NET Core over .NET Framework? • Why is .NET Core cross-platform? • What is Kestrel? • How does CLI help? • How does hosting differ? • What are instance types? • What are spot instances? • What are shared instances? • What are dedicated instances?

Excellent — this is a **true architecture depth + cloud maturity round**. I'll answer clearly and like a senior engineer explaining design decisions.

◈ Open/Closed Principle – Design Challenge

---

1 If tomorrow a new notification type (e.g., WhatsApp) is added, how will your design handle it?

If my design follows OCP properly, I would not modify existing notification logic.

Instead, I would:

1. Define an interface:

```
public interface INotificationService
{
    void Send(string message);
}
```

2. Existing implementations:

- EmailNotification
- SmsNotification
- PushNotification

3. For WhatsApp:

- Create new class `WhatsAppNotification` implementing `INotificationService`.

No modification to existing classes.

The system remains stable.

---

2 Will your design violate Open/Closed Principle?

It depends on implementation.

❌ If I use:

```
if(type == "Email") ...
else if(type == "SMS") ...
```

Then yes — adding WhatsApp means modifying existing code.

That violates OCP.

✔ If I use:

- Strategy pattern
- Factory pattern
- Dependency Injection

Then no violation.

---

3 How will you modify design to avoid modifying existing classes?

Instead of conditional logic, use:

◈ Strategy + DI

Register all notification types:

```
services.AddScoped<INotificationService, EmailNotification>();
services.AddScoped<INotificationService, SmsNotification>();
services.AddScoped<INotificationService, WhatsAppNotification>();
```

Then use:

```
public class NotificationManager
{
    private readonly IEnumerable<INotificationService> _services;

    public NotificationManager(IEnumerable<INotificationService>
services)
    {
        _services = services;
    }
}
```

Adding new type requires:

- New class
- Registration in DI

No change to existing logic.

---

4 How do you make your system extensible?

I ensure extensibility by:

- Programming against interfaces
- Avoiding switch/if blocks
- Using Strategy pattern
- Using Factory abstraction
- Using DI container
- Following SOLID principles
- Designing plug-in style architecture

Extensible systems:

- Add new behavior
- Without modifying tested code
- Without regression risk

◈ .NET Core Advantages

---

5 What are advantages of .NET Core over .NET Framework?

✅ Cross-platform (Windows, Linux, Mac)

✅ Better performance

✅ Lightweight

✅ Cloud-native

✅ Built-in Dependency Injection

✅ Modular architecture

✅ Side-by-side versioning

✅ CLI support

✅ Docker-friendly

✅ Active development

.NET Core is designed for microservices and modern cloud systems.

---

6 Why is .NET Core cross-platform?

Because:

- It uses a cross-platform runtime (CoreCLR)
- Does not depend on Windows-only components
- Uses open-source libraries
- Runs on Linux & macOS

.NET Framework depends heavily on Windows OS APIs.

---

7 What is Kestrel?

Kestrel is:

- A cross-platform web server
- Built into ASP.NET Core
- Lightweight and high-performance

In production:

Client → NGINX/IIS → Kestrel → ASP.NET Core app

Kestrel handles HTTP processing.

---

8 How does CLI help?

.NET CLI allows:

- Creating projects
- Building
- Running
- Publishing
- Managing NuGet
- Running migrations

Example:

```
dotnet build
dotnet run
dotnet publish
```

CLI enables:

- DevOps automation
- CI/CD pipelines
- Cross-platform development

---

9 How does hosting differ?

◈ .NET Framework

- Requires IIS
- Windows only
- System-wide installation

◈ .NET Core

- Self-hosted with Kestrel
- Can run as console app
- Can run in Docker
- Can run in Linux
- Can run behind reverse proxy

Much more flexible.

◈ Cloud Concepts

---

10 What are instance types?

Instance types refer to:

- Different VM or compute configurations
- CPU, RAM, storage variations

Example:

- General purpose
- Memory optimized
- Compute optimized

Choose based on workload type.

---

11 What are spot instances?

Spot instances are:

- Discounted compute instances
- Can be terminated anytime
- Used for non-critical workloads

Good for:

- Batch jobs
- Testing
- Background processing

Not good for critical production apps.

---

12 What are shared instances?

Shared instances:

- Shared infrastructure
- Lower cost
- Limited resources
- Used in dev/test

Performance not guaranteed.

---

13 What are dedicated instances?

Dedicated instances:

- Reserved compute resources
- Isolated environment
- Predictable performance
- Higher cost

Used in production enterprise systems.

🔥 Strong Senior-Level Closing Statement

"I design systems to follow Open/Closed Principle by relying on abstraction and extensibility patterns, and I leverage .NET Core's cross-platform runtime, Kestrel

hosting, CLI tooling, and cloud-native capabilities to build scalable and maintainable enterprise-grade applications."

• How do you implement load balancing? • What is application load balancer? • What is classic load balancer? • How does round robin work? • How do you expose public IP? • How does load balancer route to internal servers? _____

Excellent — this is a **networking + cloud infrastructure maturity round**. I'll answer clearly and in architect-level format.

◈ 1 How do you implement load balancing?

Load balancing is implemented to distribute incoming traffic across multiple servers to:

- Improve availability
- Improve scalability
- Prevent overload
- Achieve high uptime

Implementation Options:

◈ In Cloud (Azure example):

- Azure Load Balancer (Layer 4)
- Azure Application Gateway (Layer 7)
- Azure Front Door (Global)
- AKS Ingress Controller

◈ On-Prem:

- NGINX
- HAProxy
- F5
- IIS ARR (Application Request Routing)

Basic flow:

Client → Public IP → Load Balancer → Multiple Backend Servers

---

◈ 2 What is Application Load Balancer?

Application Load Balancer (Layer 7 Load Balancer):

- Operates at HTTP/HTTPS level
- Can route based on:
  - URL path
  - Hostname
  - Headers
- Supports SSL termination
- Supports WAF (Web Application Firewall)

Example:

- `/orders` → Order Service
- `/payments` → Payment Service

In Azure:
Azure Application Gateway acts as Application Load Balancer.

◈ 3 What is Classic Load Balancer?

Classic Load Balancer (Layer 4 Load Balancer):

- Operates at TCP/UDP level
- Does not inspect HTTP content
- Routes traffic based on IP and port only

Example:
Port 443 → forward to backend pool

In Azure:
Azure Load Balancer (Basic/Standard)

Layer 4 = Transport level
Layer 7 = Application level

◈ 4 How does round robin work?

Round Robin distributes requests sequentially across servers.

Example:

Request 1 → Server A
Request 2 → Server B
Request 3 → Server C
Request 4 → Server A

It cycles through servers equally.

Advantages:

- Simple
- Fair distribution

Limitation:

- Does not consider server load

More advanced algorithms:

- Least connections
- Weighted round robin
- IP hash

◈ 5 How do you expose public IP?

Steps (Cloud example):

1. Create Public IP resource
2. Associate it with:
    - Load Balancer
    - Application Gateway
    - VM
3. Configure DNS mapping
4. Open required ports (80, 443)
5. Configure SSL certificate

Traffic flow:

Internet → Public IP → Load Balancer → Backend Pool

◈ 6 How does load balancer route to internal servers?

Load balancer uses:

◈ Backend Pool

List of internal server IPs or VM instances

◈ Health Probes

Regularly checks:

- Is server alive?
- Is port responding?

If server fails → removed from pool.

◈ Routing Rules

Defines:

- Frontend port
- Backend port
- Protocol
- Distribution algorithm

Example:
Public 443 → Backend port 80 → Round Robin

Flow:

Client
↓
Public IP
↓
Load Balancer
↓
Healthy Backend Server
↓
Response returned via same path

🔥 Strong Senior-Level Closing Statement

"Load balancing ensures high availability and scalability by distributing traffic across multiple backend instances using algorithms like round robin or least connections. Application load balancers operate at Layer 7 for intelligent routing, while classic load balancers operate at Layer 4 for transport-level distribution."

• What architectural patterns do you know? • What is Microservices architecture? • What is Monolithic architecture? • What is MVC? • When should you use Microservices? • When should you use Monolith? • What factors influence your decision? • What happens when a monolith becomes performance bottleneck?

Excellent — this is a **core architecture decision-making round**. I'll answer clearly and like a senior engineer who understands trade-offs.

◈ 1 What architectural patterns do you know?

I am familiar with:

- Monolithic Architecture
- Layered (N-tier) Architecture
- Microservices Architecture
- Service-Oriented Architecture (SOA)
- Event-Driven Architecture
- CQRS
- Clean Architecture
- Hexagonal Architecture (Ports & Adapters)
- MVC Pattern

I choose architecture based on business size, scalability, and team maturity — not trend.

◈ 2 What is Microservices architecture?

Microservices architecture is a design where:

- Application is divided into small, independent services

- Each service owns its business capability
- Each service has its own database
- Services communicate over network (REST or messaging)
- Services can be deployed independently

Example:

- Order Service
- Payment Service
- Inventory Service
- Notification Service

Advantages:

- Independent scaling
- Fault isolation
- Faster deployments
- Technology flexibility

But it introduces complexity.

◈ 3 What is Monolithic architecture?

Monolithic architecture means:

- Entire application is built as one unit
- Single codebase
- Single database
- Single deployment

All modules (Order, Payment, User) are part of one system.

Advantages:

- Simple to develop
- Easy to deploy
- Easier debugging
- Less operational overhead

Good for small to medium systems.

◈ 4 What is MVC?

MVC (Model-View-Controller) is a design pattern that separates:

- Model → Business logic
- View → UI
- Controller → Request handling

MVC improves:

- Separation of concerns
- Testability
- Maintainability

MVC is about application structure, not system architecture.

◈ 5 When should you use Microservices?

Use Microservices when:

- System is large and complex
- Different modules scale differently
- Multiple teams working independently
- Need independent deployment
- High availability required
- Business grows rapidly
- Need cloud-native scalability

Microservices are ideal for large enterprise SaaS platforms.

◈ 6 When should you use Monolith?

Use Monolith when:

- System is small to medium
- Team is small
- Business logic is tightly connected
- Early-stage startup
- Low operational complexity required

Starting with monolith is often practical.

Premature microservices create unnecessary complexity.

◈ 7 What factors influence your decision?

I evaluate:

◈ Business Complexity

Simple system → Monolith
Complex domain → Microservices

◈ Team Size

Small team → Monolith
Multiple teams → Microservices

◈ Scalability Requirements

Uniform scaling → Monolith
Independent scaling → Microservices

◈ Deployment Frequency

Independent releases needed → Microservices

◈ Operational Maturity

No DevOps maturity → Avoid microservices

Architecture decision is based on trade-offs.

◈ 8 What happens when a monolith becomes performance bottleneck?

Common issues:

- Entire application must scale together

- Heavy modules affect lightweight modules
- Database becomes bottleneck
- Long deployment cycles
- Hard to isolate failures

Solutions:

1. Optimize database
2. Add caching (Redis)
3. Scale horizontally
4. Introduce CQRS
5. Extract high-load modules into microservices (Strangler pattern)

We don't jump directly to full microservices — we evolve gradually.

🔥 Strong Senior-Level Closing Statement

"Architecture should be chosen based on business needs, team maturity, and scalability requirements. Monolith offers simplicity, while microservices offer scalability and independence — but only when operational complexity can be handled."

• How do you migrate .NET Framework application to .NET Core? • What steps should be followed? • How do you divide modules? • How do you handle database separation? • How do you handle inter-service communication? • How do you handle data migration? • What tools exist for migration? • What manual work is required? • How do you plan microservice decomposition? • How do you handle secrets during migration? • How do you use Azure Key Vault? • How do you manage configuration in multiple environments? • How do you use Azure AD? • What is ADFS? • How does Active Directory integrate with application?

_____

Excellent — this is a **serious modernization + cloud architecture + identity integration round**. I'll answer step-by-step like a senior engineer leading migration.

◈ 1⃣ How do you migrate .NET Framework application to .NET Core?

Migration is not just "retarget framework". It's a structured modernization.

High-level approach:

1. Assess compatibility
2. Upgrade dependencies
3. Separate concerns
4. Migrate project-by-project
5. Validate functionality
6. Modernize architecture gradually

Never rewrite everything at once.

◈ 2 What steps should be followed?

✅ Step 1: Assessment

- Identify .NET version
- Identify Web Forms / MVC / WCF usage
- Identify Enterprise Library usage
- Identify third-party packages
- Check NuGet compatibility

Use:

- .NET Upgrade Assistant
- Portability Analyzer

✅ Step 2: Upgrade Dependencies

- Replace unsupported libraries
- Remove Enterprise Library
- Replace WCF (if needed) with Web API
- Replace System.Web usage

✅ Step 3: Migrate to .NET Standard (Intermediate Step)

- Extract business logic into class libraries
- Convert those to .NET Standard first
- Then move to .NET Core

✅ Step 4: Create ASP.NET Core Project

- Move controllers
- Replace Startup logic
- Configure middleware
- Replace Global.asax with Program.cs

✅ Step 5: Replace Legacy Features

- Replace ViewState (if Web Forms)
- Replace HttpContext.Current
- Replace machine.config dependencies
- Replace Web.config with appsettings.json

✅ Step 6: Testing & Validation

- Unit testing
- Integration testing
- Smoke testing
- Performance validation

---

3 How do you divide modules?

Use Domain-Driven Design.

Steps:

1. Identify bounded contexts
2. Separate business capabilities
3. Extract independent modules
4. Avoid splitting by technical layers
5. Split by business ownership

Example:

- Order
- Billing
- Vendor

- Identity
- Reporting

◈ 4 How do you handle database separation?

Initial stage:

- Shared database

After decomposition:

- Database per service
- Remove cross-service joins
- Use events for data sync
- Introduce read models if needed

Avoid direct DB-to-DB access.

◈ 5 How do you handle inter-service communication?

Hybrid model:

◈ Synchronous

- REST APIs
- gRPC

◈ Asynchronous

- Azure Service Bus
- RabbitMQ

Use async for:

- Order processing
- Notifications
- Event-driven workflows

◈ 6 How do you handle data migration?

Options:

- Backup & restore
- ETL tools
- Data sync scripts
- Incremental migration
- Log shipping
- Azure Database Migration Service

Always:

- Validate row counts
- Validate checksums
- Test application connectivity

◈ 7 What tools exist for migration?

Microsoft tools:

- .NET Upgrade Assistant
- .NET Portability Analyzer
- Azure Migrate
- Azure Database Migration Service
- SQL Server Migration Assistant

◈ 8 What manual work is required?

Manual work includes:

- Replacing deprecated APIs
- Rewriting Web Forms UI
- Replacing WCF services
- Replacing Enterprise Library
- Adjusting configuration
- Refactoring tight coupling
- Performance retuning

Automation helps, but architecture changes require manual intervention.

◈ 9 How do you plan microservice decomposition?

Steps:

1. Identify business domains
2. Identify data ownership
3. Remove shared database dependencies
4. Introduce API boundaries
5. Extract high-change/high-load modules first
6. Use Strangler pattern
7. Deploy gradually

Don't break randomly — break by domain.

◈ 🔐 Secrets & Configuration

---

10 How do you handle secrets during migration?

Never store secrets in:

- Source code
- appsettings.json
- Web.config

Use:

- Azure Key Vault
- Managed Identity
- Environment variables

---

11 How do you use Azure Key Vault?

Steps:

1. Store secrets in Key Vault
2. Enable Managed Identity for App Service
3. Grant access policy to app
4. Configure app to fetch secrets at runtime

Example:

```
builder.Configuration.AddAzureKeyVault(...);
```

Secrets stay secure and encrypted.

---

12 How do you manage configuration in multiple environments?

Use:

- appsettings.json
- appsettings.Development.json
- appsettings.Production.json
- Environment variables
- Azure App Configuration

Avoid hardcoded values.

Use configuration providers.

◈ Identity & Authentication

---

13 How do you use Azure AD?

Azure AD is used for:

- Authentication
- OAuth2
- OpenID Connect
- Role-based access control

Flow:

User → Azure AD → JWT → API

API validates:

- Issuer
- Audience
- Signature

---

14 What is ADFS?

ADFS (Active Directory Federation Services):

- On-prem identity provider
- Allows single sign-on
- Federation with external apps
- Often used before Azure AD adoption

Legacy enterprise identity system.

---

15 How does Active Directory integrate with application?

Two ways:

◈ Windows Authentication (On-Prem)

- Application trusts domain
- User automatically authenticated

◈ Azure AD (Cloud)

- App registered in Azure AD
- Uses OAuth2 / OIDC
- JWT tokens issued

Integration allows:

- Role-based authorization
- Claims-based identity
- Single sign-on (SSO)

🔥 Strong Senior-Level Closing Statement

"Migrating from .NET Framework to .NET Core requires structured modernization — dependency assessment, architectural decomposition, database segregation, secure secret management via Key Vault, and identity integration with Azure AD — ensuring scalability, security, and cloud readiness."

• Given a numeric sequence, write logic to compute next values. • Explain incremental logic. • Can you implement logic? • Can you write code? • Can you email the solution?

Great — this is a **logic + coding clarity round**. I'll answer step by step like in an interview.

---

1 Given a numeric sequence, write logic to compute next values.

First, I would clarify:

- Is the sequence arithmetic?
- Is it geometric?
- Is it Fibonacci-like?
- Is it pattern-based?
- Is it dynamic (difference changes)?

Because without understanding the pattern, we cannot compute next value.

Example 1: Arithmetic Sequence

Sequence:

```
2, 4, 6, 8, ?
```

Logic:
Difference between numbers = +2
Next = 8 + 2 = 10

Example 2: Fibonacci Sequence

```
1, 1, 2, 3, 5, 8, ?
```

Logic:
Next value = sum of previous two numbers
Next = 5 + 8 = 13

Example 3: Geometric Sequence

```
3, 6, 12, 24, ?
```

Logic:
Multiply by 2 each time
Next = 24 × 2 = 48

---

2 Explain Incremental Logic

Incremental logic means:

Each new value depends on previous value(s).

For arithmetic sequence:

```
next = current + difference
```

For Fibonacci:

```
next = previous + current
```

General pattern:

- Detect rule
- Apply rule repeatedly

---

3 Can you implement logic?

Yes.

Let's implement arithmetic sequence detection:

Steps:

1. Compute difference between first two elements.
2. Check if difference remains constant.
3. Add difference to last element.

---

4 Can you write code?

◈ Example: Arithmetic Sequence (C#)

```
public static int GetNextArithmetic(List<int> sequence)
{
    if (sequence == null || sequence.Count < 2)
        throw new ArgumentException("Sequence must have at least 2
numbers");

    int difference = sequence[1] - sequence[0];

    return sequence[sequence.Count - 1] + difference;
}
```

◈ Example: Fibonacci Next Value

```
public static int GetNextFibonacci(List<int> sequence)
{
    if (sequence == null || sequence.Count < 2)
        throw new ArgumentException("Sequence must have at least 2
numbers");

    int last = sequence[sequence.Count - 1];
    int secondLast = sequence[sequence.Count - 2];

    return last + secondLast;
}
```

◈ Generic Incremental Generator

```
public static List<int> GenerateArithmeticSequence(int start, int
difference, int count)
{
    var result = new List<int>();

    for (int i = 0; i < count; i++)
    {
        result.Add(start + i * difference);
    }

    return result;
}
```

---

5 Can you email the solution?

In a real interview, I would respond:

“Yes, I can share the solution via email or repository link as requested.”

(And then follow their submission instructions.)

🔥 Strong Interview Closing Line

“To compute next values in a numeric sequence, I first identify the pattern — arithmetic, geometric, Fibonacci, or custom rule — then implement incremental logic based on detected difference or relation between previous elements.”

• Have you worked with multithreading? • Suppose you have a cached in-memory object shared by 100 threads — how do you manage refresh? • How do you prevent race conditions? • How do you use Mutex? • What is Mutex.WaitOne? • What is Mutex.ReleaseMutex? • When should you clear cache? • How do you prevent stale data? • What is locking mechanism? • What happens to main thread in async? • What happens when using Parallel.ForEach? • What happens when using Task Parallel Library? • What is difference between CPU-bound vs IO-bound? • When to use async/await? • What happens to main thread when calling async method? • What is state machine in async? _____

Excellent — this is a **deep multithreading + async maturity round**. I’ll answer clearly and at senior-engineer level.

◈ 1 Have you worked with multithreading?

Yes.

I have worked with:

- Task Parallel Library (TPL)
- async/await
- Parallel.ForEach
- Thread-safe caching
- Locking mechanisms (lock, Monitor, Mutex, Semaphore)
- Concurrent collections

I focus on avoiding race conditions and deadlocks.

◈ 2 Suppose you have a cached in-memory object shared by 100 threads — how do you manage refresh?

Problem:
100 threads accessing same cache.
One thread refreshes → others may read inconsistent data.

Solution approaches:

✅ 1. Double-checked locking

```
if (_cache == null)
{
    lock(_lock)
    {
        if (_cache == null)
            _cache = LoadData();
    }
}
```

✅ 2. Use Lazy<T>

Thread-safe initialization.

✅ 3. Use ReaderWriterLockSlim

- Many readers allowed
- Single writer allowed

✅ 4. Use ConcurrentDictionary

If caching per key.

◈ 3 How do you prevent race conditions?

Race condition happens when:

Multiple threads access shared resource without synchronization.

Prevention methods:

- lock keyword
- Monitor
- Mutex
- Semaphore
- Interlocked
- Thread-safe collections

Best practice:
Avoid shared mutable state when possible.

◈ 4 How do you use Mutex?

Mutex is used for cross-process synchronization.

Example:

```
Mutex mutex = new Mutex();
mutex.WaitOne();

// Critical section

mutex.ReleaseMutex();
```

◈ 5 What is Mutex.WaitOne?

`WaitOne()`:

- Blocks thread until mutex becomes available.
- Thread waits for lock.

If another thread holds mutex → current thread pauses.

◈ 6 What is Mutex.ReleaseMutex?

Releases ownership of mutex.

If not released → deadlock possible.

Always use try/finally.

◈ 7 When should you clear cache?

Clear cache when:

- Underlying data changes
- Configuration updated
- Scheduled refresh time reached
- Memory pressure
- Explicit invalidation event

Never rely only on TTL for critical data.

◈ 8 How do you prevent stale data?

Methods:

- Cache invalidation strategy
- Short TTL
- Event-driven invalidation
- Version-based cache keys
- Write-through caching
- Cache-aside pattern

Stale data prevention requires strategy, not guesswork.

◈ 9 What is locking mechanism?

Locking mechanism ensures:

Only one thread executes critical section at a time.

In C#:

```
lock(_lockObject)
{
    // critical code
}
```

Internally uses Monitor.

Prevents concurrent modification.

◈ 10 What happens to main thread in async?

In ASP.NET Core:

- There is no UI thread.
- Async does NOT block thread.
- Thread returns to thread pool while awaiting I/O.

In UI apps:

- UI thread is freed during await.
- Continues after task completion.

Async improves scalability.

◈ 11 What happens when using Parallel.ForEach?

Parallel.ForEach:

- Splits work across multiple threads.
- Uses ThreadPool threads.
- Suitable for CPU-bound work.
- Can cause race conditions if shared state used.

Should not be used for I/O-bound operations.

◈ 1️⃣2️⃣ What happens when using Task Parallel Library (TPL)?

TPL:

- Schedules tasks on thread pool.
- Manages concurrency automatically.
- Supports Task.WhenAll.
- Efficient for CPU-bound parallel workloads.

Example:

```
await Task.WhenAll(task1, task2);
```

◈ 1️⃣3️⃣ What is difference between CPU-bound vs IO-bound?

◈ CPU-bound

- Heavy calculations
- High CPU usage
- Use Parallel or TPL

Example:
Image processing

◈ IO-bound

- Waiting on database
- Waiting on API call
- Waiting on file read

Use async/await.

Async frees thread during wait.

◈ 1️⃣4️⃣ When to use async/await?

Use async/await for:

- Database calls
- HTTP calls
- File I/O
- Network operations

Not useful for CPU-bound operations.

Async improves scalability, not speed.

◈ 15 What happens to main thread when calling async method?

When async method hits await:

- Current thread is released
- Task continues in background
- After completion, continuation runs
- Thread from pool executes continuation

Thread is not blocked.

◈ 16 What is state machine in async?

Compiler converts async method into:

State machine class.

It:

- Tracks method progress
- Stores local variables
- Resumes execution after await
- Manages continuation

You don’t see it, but compiler generates it automatically.

Async is syntactic sugar over state machine + tasks.

🔥 Strong Senior-Level Closing Statement

"I use multithreading and async strategically — locking only when necessary, avoiding shared mutable state, differentiating CPU-bound from IO-bound workloads, and leveraging async/await and TPL to maximize scalability while preventing race conditions and stale cache issues."

• If you have two classes and want to reuse functionality, how will you do it? • Can you inherit from multiple classes? • How do you achieve multiple inheritance in C#? • What is multicast delegate? • What is lambda expression? • What is predicate? • What is delegate? • What is event? _____

Excellent — this is a **core OOP + C# language fundamentals round**. I'll answer clearly and in interview-ready format.

◈ 1 If you have two classes and want to reuse functionality, how will you do it?

There are multiple ways depending on scenario:

✅ Option 1: Inheritance (if "is-a" relationship)

If both classes share common behavior:

```
public class BaseLogger
{
    public void Log(string message) { }
}

public class OrderService : BaseLogger { }
public class PaymentService : BaseLogger { }
```

Use inheritance when:

- Strong relationship exists
- Shared base behavior

✅ Option 2: Composition (preferred in modern design)

Instead of inheriting:

```
public class OrderService
{
    private readonly ILogger _logger;

    public OrderService(ILogger logger)
    {
        _logger = logger;
    }
}
```

Composition is preferred because:

- More flexible
- Follows SOLID
- Avoids tight coupling

Modern architecture prefers composition over inheritance.

◈ 2 Can you inherit from multiple classes?

No.

C# does **not support multiple inheritance** of classes.

You can inherit from only one base class.

This avoids:

- Diamond problem
- Ambiguity issues

◈ 3 How do you achieve multiple inheritance in C#?

By using **interfaces**.

Example:

```
public interface ILogger
{
    void Log();
}
```

```
public interface IValidator
{
    void Validate();
}

public class OrderService : ILogger, IValidator
{
    public void Log() { }
    public void Validate() { }
}
```

C# supports:

✔ Multiple interface inheritance

✕ Multiple class inheritance

◈ 4 What is multicast delegate?

A multicast delegate holds references to **multiple methods**.

When invoked → all methods execute.

Example:

```
public delegate void Notify();

Notify notify = Method1;
notify += Method2;

notify();
```

Both Method1 and Method2 execute.

Used in:

- Events
- Notification systems

◈ 5 What is lambda expression?

Lambda expression is a concise way to write anonymous functions.

Example:

```
x => x * 2
```

Used heavily in:

- LINQ
- Delegates
- Functional programming

Example:

```
var result = numbers.Where(x => x > 5);
```

Lambda improves readability and brevity.

◈ 6 What is predicate?

Predicate is a delegate that:

- Returns bool
- Takes one parameter

Definition:

```
Predicate<int> isEven = x => x % 2 == 0;
```

Used for:

- Filtering
- Validation
- Conditions

Predicate<T> = delegate returning true/false.

◈ 7 What is delegate?

Delegate is a type-safe reference to a method.

It allows:

- Passing methods as parameters

- Callback functionality
- Event handling

Example:

```
public delegate int Calculator(int a, int b);
```

Delegates are foundation for:

- Events
- LINQ
- Async patterns

◈ 8 What is event?

Event is a wrapper around delegate.

Used to implement publish-subscribe pattern.

Example:

```
public event EventHandler OnOrderCreated;
```

Only class that defines event can raise it.
Other classes can subscribe.

Events are built on multicast delegates.

🔥 Senior-Level Summary

- Use composition over inheritance for reuse.
- C# supports single inheritance but multiple interfaces.
- Delegates are function pointers.
- Multicast delegates call multiple methods.
- Events are built on delegates.
- Lambda expressions provide concise inline functions.
- Predicates are boolean-returning delegates.

If you want, I can now explain **how delegates power async/await internally** — that's often asked in deeper interviews.

• Have you worked with Action Filters? • How do you handle global exception handling? • How do you implement output caching? • What are filter types in Web API? • What is difference between middleware and filters? _____ 6 Dependency Injection • What DI frameworks have you used? • What is Unity container? • What is Ninject? • What is built-in DI? • What is service lifetime (Transient / Scoped / Singleton)? • What is AppDomain? • Have you worked with multiple AppDomains?

_____

Excellent — this is a **pipeline + DI + runtime architecture round**. I'll answer clearly and in senior-level format.

◈ Action Filters & Request Pipeline

---

1 Have you worked with Action Filters?

Yes.

Action Filters are used in ASP.NET MVC / Web API to execute logic:

- Before action executes
- After action executes

Example use cases:

- Logging
- Validation
- Auditing
- Performance tracking
- Authorization checks

Example:

```
public class LoggingFilter : ActionFilterAttribute
{
    public override void OnActionExecuting(ActionExecutingContext
context)
    {
        // Before action logic
    }
```

```
    public override void OnActionExecuted(ActionExecutedContext
context)
    {
        // After action logic
    }
}
```

---

2 How do you handle global exception handling?

✅ In ASP.NET Core (Recommended)

Use middleware:

```
app.UseExceptionHandler("/error");
```

Or custom middleware:

```
public class GlobalExceptionMiddleware
{
    private readonly RequestDelegate _next;

    public async Task Invoke(HttpContext context)
    {
        try
        {
            await _next(context);
        }
        catch(Exception ex)
        {
            // Log error
            context.Response.StatusCode = 500;
        }
    }
}
```

Why middleware?

- Centralized
- Handles all requests
- Runs early in pipeline

---

3 How do you implement output caching?

In older ASP.NET:

```
[OutputCache(Duration = 60)]
```

In ASP.NET Core:

Use Response Caching Middleware:

```
services.AddResponseCaching();
app.UseResponseCaching();
```

Or use:

- Redis cache
- Distributed cache
- CDN caching

Output caching improves performance for read-heavy endpoints.

---

4 What are filter types in Web API?

In ASP.NET:

◈ Authorization Filter

Runs before authentication.

◈ Action Filter

Before and after action execution.

◈ Result Filter

Before and after result execution.

◈ Exception Filter

Handles unhandled exceptions.

Execution order:
Authorization → Action → Result → Exception

---

5 What is difference between middleware and filters?

Middleware	Filters
Runs at application level	Runs at MVC level
Works for all requests	Only works for MVC requests
Configured in pipeline	Applied via attributes
Executes before routing	Executes after routing

Middleware:

- Handles cross-cutting concerns globally

Filters:

- Specific to controllers/actions

Modern best practice:
Global exception handling → Middleware
Controller-specific logic → Filters

◈ Dependency Injection

---

6 What DI frameworks have you used?

I have used:

- Built-in ASP.NET Core DI
- Unity (older .NET Framework projects)
- Ninject
- Autofac (if applicable)
- Simple Injector (occasionally)

Modern projects → Prefer built-in DI container.

---

7 What is Unity container?

Unity is:

- A dependency injection container by Microsoft
- Used in older .NET Framework apps
- Supports constructor injection
- Supports lifetime management

Example:

```
container.RegisterType<IService, Service>();
```

---

8 What is Ninject?

Ninject is:

- Open-source DI container
- Uses convention-based binding

Example:

```
Bind<IService>().To<Service>();
```

Mostly used in legacy MVC projects.

---

9 What is built-in DI?

ASP.NET Core has built-in DI container.

Configured in:

```
builder.Services.AddScoped<IService, Service>();
```

Supports:

- Constructor injection
- Scoped lifetime
- Transient lifetime
- Singleton lifetime

It is lightweight and integrated into framework.

---

10 What is service lifetime?

◈ Transient

- New instance every time requested

◈ Scoped

- One instance per HTTP request

◈ Singleton

- One instance for entire application lifetime

Choosing correct lifetime prevents memory leaks and thread issues.

---

11 What is AppDomain?

AppDomain is:

- Isolation boundary inside .NET process
- Allows multiple applications inside same process
- Provides security isolation

In .NET Framework, heavily used.

In .NET Core:

- AppDomain concept mostly removed
- Now use processes or containers for isolation

---

12 Have you worked with multiple AppDomains?

In older .NET Framework projects:

- Yes, indirectly through IIS
- Each application pool runs in separate AppDomain
- AppDomain unload resets static variables

In .NET Core:

- No traditional AppDomain usage
- Isolation handled differently

🔥 Strong Senior-Level Closing Statement

"I understand the request pipeline deeply — using middleware for global concerns, filters for controller-level logic, and structured dependency injection with proper lifetimes to ensure clean, scalable, and maintainable architecture."

• What authentication mechanisms have you used? • What is OAuth2? • What is Authorization Code Flow? • What is OpenID Connect? • What is UserInfo endpoint? • How does Azure AD authentication work? • What is ADFS? • What is JWT? • How do you validate tokens? _____ 8 SQL Questions • How do you write recursive query? • How do you write recursive CTE? • How to avoid CTE? • How to use temp tables? • How do you troubleshoot slow query? • What is execution plan? • What is table scan? • What is index seek? • What is blocking session? • What is isolation level? • What is read committed? • How do you troubleshoot production slowness? • How do you handle transaction blocking? • How do you analyze RCA? _____

Excellent — this is a **Security + Identity + Advanced SQL troubleshooting round**. I'll answer clearly and at senior-level depth.

🔐 Authentication & Identity

---

1 What authentication mechanisms have you used?

I have worked with:

- JWT-based authentication
- OAuth2
- OpenID Connect (OIDC)
- Azure AD
- ADFS (legacy on-prem identity)
- Windows Authentication (on-prem)
- API Key authentication (for internal services)

Modern systems → JWT + OAuth2 + Azure AD.

---

2 What is OAuth2?

OAuth2 is an **authorization framework**.

It allows:

- Secure delegated access
- Applications to access APIs on behalf of users

Key concepts:

- Client
- Resource Owner
- Authorization Server
- Access Token

OAuth2 does NOT define authentication — it defines authorization.

---

3 What is Authorization Code Flow?

Authorization Code Flow is the most secure OAuth2 flow for web apps.

Flow:

1. User redirected to Identity Provider.
2. User logs in.
3. Authorization code returned.
4. Backend exchanges code for Access Token.
5. Access Token used to call APIs.

Why secure?

- Tokens are not exposed in browser.
- Exchange happens server-side.

---

4 What is OpenID Connect?

OpenID Connect (OIDC) is built on OAuth2.

OAuth2 → Authorization
OIDC → Authentication + Identity

OIDC provides:

- ID Token
- User claims
- Standard scopes (openid, profile, email)

OIDC confirms identity of user.

---

5 What is UserInfo endpoint?

UserInfo endpoint:

- Provided by OIDC
- Returns user profile information
- Called using Access Token

Example returns:

- Name
- Email
- Roles
- Claims

Used when additional user info needed beyond ID token.

---

6 How does Azure AD authentication work?

Flow:

1. Application registered in Azure AD.
2. Client redirects to Azure AD login.
3. User authenticates.
4. Azure AD issues JWT token.
5. API validates:
    - Signature
    - Issuer
    - Audience

Azure AD uses:

- OAuth2
- OpenID Connect
- Public key signature validation

APIs download public keys from Azure metadata endpoint.

---

7 What is ADFS?

ADFS (Active Directory Federation Services):

- On-prem identity provider
- Enables Single Sign-On (SSO)
- Federates on-prem AD with applications

Often used before Azure AD adoption.

ADFS is legacy enterprise identity solution.

---

8 What is JWT?

JWT (JSON Web Token):

- Compact, self-contained token
- Contains:
    - Header
    - Payload
    - Signature

Structure:

```
Header.Payload.Signature
```

JWT contains:

- User ID
- Roles
- Expiration
- Issuer
- Audience

Signed using symmetric or asymmetric keys.

---

9 How do you validate tokens?

Validation steps:

1. Validate signature
2. Validate expiration
3. Validate issuer
4. Validate audience
5. Validate scope/role

In ASP.NET Core:

```
services.AddAuthentication()
        .AddJwtBearer(options =>
        {
            options.Authority = "...";
            options.Audience = "...";
        });
```

Middleware handles validation automatically.

🗄 SQL & Performance

---

10 How do you write recursive query?

Recursive query is used for hierarchical data.

Example:
Employee → Manager hierarchy.

---

11 How do you write recursive CTE?

Example:

```
WITH EmployeeHierarchy AS
(
    SELECT Id, ManagerId
    FROM Employees
    WHERE ManagerId IS NULL
```

```
    UNION ALL

    SELECT e.Id, e.ManagerId
    FROM Employees e
    INNER JOIN EmployeeHierarchy eh
        ON e.ManagerId = eh.Id
)
SELECT * FROM EmployeeHierarchy;
```

Used for tree structures.

---

12 How to avoid CTE?

Alternatives:

- Temp tables
- Subqueries
- Derived tables
- Indexed views

If CTE causes performance issue:
Break into temp table.

---

13 How to use temp tables?

```
CREATE TABLE #TempData
(
   Id INT,
   Name VARCHAR(100)
);

INSERT INTO #TempData
SELECT Id, Name FROM Orders;

SELECT * FROM #TempData;
```

Use when:

- Large intermediate results
- Reused multiple times
- Complex joins

---

14 How do you troubleshoot slow query?

Steps:

1. Capture execution plan
2. Check:
    - Table scans
    - Missing indexes
    - Key lookups
    - High-cost operators
3. Check:
    - IO stats
    - CPU usage
4. Check:
    - Blocking sessions
    - Wait stats

Never guess — measure first.

---

15 What is execution plan?

Execution plan shows:

- How SQL executes query
- Index usage
- Join types
- Cost distribution

Used to detect performance bottlenecks.

---

16 What is table scan?

SQL reads entire table.

Occurs when:

- No index
- Poor filtering

Bad for large tables.

---

17 What is index seek?

SQL directly jumps to matching rows using index.

Most efficient data retrieval method.

Goal:
Replace table scan with index seek.

---

18 What is blocking session?

Blocking session occurs when:

One transaction holds lock
Another transaction waits

Causes application slowdown.

Use:

```
sp_who2
```

To detect blocking.

---

19 What is isolation level?

Isolation level controls:

- Visibility of uncommitted data
- Locking behavior

Balances:
Consistency vs concurrency.

---

20 What is read committed?

Default isolation level.

- Prevents dirty reads

- Still allows non-repeatable reads

Reads only committed data.

---

21 How do you troubleshoot production slowness?

Steps:

1. Check CPU, memory
2. Check SQL wait stats
3. Identify blocking sessions
4. Analyze slow queries
5. Check index fragmentation
6. Review recent deployments
7. Check connection pool exhaustion
8. Check application logs

Always isolate:
App issue or DB issue?

---

22 How do you handle transaction blocking?

Solutions:

- Identify blocking query
- Kill long-running transaction (if needed)
- Add proper indexes
- Reduce transaction scope
- Use lower isolation level (carefully)
- Break large transactions

Avoid long-running transactions.

---

23 How do you analyze RCA (Root Cause Analysis)?

Steps:

1. Identify symptom
2. Collect logs
3. Check metrics

4. Review deployment history
5. Reproduce issue in lower environment
6. Identify trigger
7. Fix permanently
8. Document learning

RCA is systematic — not assumption-based.

🔥 Senior-Level Closing Statement

"I approach authentication using OAuth2, OIDC, and JWT validation best practices, and I handle SQL performance issues by systematically analyzing execution plans, blocking sessions, isolation levels, and wait statistics to ensure reliable and scalable production systems."

• Why does C# not support multiple inheritance? • What problems arise with multiple inheritance? • What is the workflow of Garbage Collector? • What are generations? • What is managed vs unmanaged code? • What is Dispose? • What is Finalize? • What is difference between Dispose and Finalize? • What happens if Dispose is not called? _____ 3 SOLID Principles • Explain SOLID principles. • Explain Single Responsibility. • Explain Open/Closed. • Explain Interface Segregation. • Explain Liskov Substitution. • Explain Dependency Inversion. • Give real-world examples. _____ 4 Dependency Injection • How does DI work in .NET Core? • What are service lifetimes? • What is Scoped? • What is Transient? • What is Singleton? • What is IoC? • What DI frameworks have you used? • Have you used Unity? • Have you used Ninject? • Have you used built-in DI? _____ 5 LINQ • Difference between IQueryable and IEnumerable? • When to use each? • What happens in database query? • What happens in in-memory collection? _____ 6 Unit Testing • Have you used unit testing frameworks? • Have you used Moq? • Have you used xUnit? • Explain Test Driven Development. • Why use mocking? • What is purpose of Moq? _____ 7 MVC Request Lifecycle • Explain MVC request pipeline. • What happens when user hits a URL? • How routing works? • What is HttpContext? • What is ControllerFactory? • What are Action Filters? • What are Result Filters? • How Razor rendering works? _____ 8 REST vs SOAP • What is difference between REST and SOAP? • What formats SOAP uses? • What formats REST uses? • Why REST preferred? • What is statelessness? _____ 9 Web API Versioning • How to implement API versioning? • How to add version in route? • How to configure versioning middleware? _____ 10 Consuming External APIs • How do you

consume external API? • What is HttpClient? • Difference between HttpWebRequest and HttpClient? • How to send GET? • How to send POST? • How to pass headers? • How to use async with HttpClient? _____ 11 SQL Fundamentals • Why use indexes? • What are clustered indexes? • What are non-clustered indexes? • Difference between stored procedure and function? • Can SP return multiple result sets? • Can function do DML? • What is execution plan? • What is table scan? • What is index seek? • What is isolation level? • What is read committed? • What is blocking session? • How do you troubleshoot production slowness? • What are root cause analysis steps? _____ 12 Cloud & DevOps • What cloud services have you used? • What CI/CD tools have you used? • How to host MVC app in IIS? • How to configure App Pool? • How to configure HTTPS? • How to configure SQL connectivity? • How to open firewall port? • How to check database connectivity? • What is health check endpoint? • How to monitor app? • What tools for monitoring? • Have you used Kibana? • Have you used Elastic? • Have you used Azure Monitor? _____ 13 Performance & Background Processing • If a file upload process takes 5 minutes, how do you improve UX? • How to implement async background processing? • How to use cancellation token? • How to design cancel operation? • How to chunk large files? • How to use bulk insert? • How to handle long-running jobs? • How to send notification after processing? _____ 14 Agile & Estimation • What estimation strategy do you follow? • What is story point? • How do you split user stories? • What is sprint length? • How do you handle production hotfix? • How branching works? • What is hotfix branch? • What is retrospective? • How do you measure sprint success? • How do you handle blockers? _____

That's a full senior-level technical panel 😊
I'll answer this in **structured, concise, high-quality interview format** so you can revise effectively.

---

1 C# Core Concepts & GC

❓ Why does C# not support multiple inheritance?

C# avoids multiple class inheritance to prevent the **Diamond Problem**.

Diamond Problem:

If Class D inherits from B and C, and both inherit from A:
Which implementation of A should D use?

This creates ambiguity.

C# solves this by:

- Allowing **single class inheritance**
- Allowing **multiple interface inheritance**

❓ What problems arise with multiple inheritance?

- Method ambiguity
- State conflicts
- Complexity in maintenance
- Diamond problem
- Tight coupling

❓ Workflow of Garbage Collector?

1. Allocate objects on heap.
2. GC runs when memory pressure occurs.
3. Mark reachable objects.
4. Remove unreachable objects.
5. Compact memory.
6. Promote surviving objects to next generation.

❓ What are generations?

- Gen 0 → short-lived objects
- Gen 1 → medium-lived
- Gen 2 → long-lived

Most objects die young → GC optimized for this.

❓ Managed vs Unmanaged Code?

Managed	Unmanaged
Controlled by CLR	Not controlled by CLR

Managed	Unmanaged
Automatic memory management	Manual memory management
.NET code	Native code (C/C++)

What is Dispose?

Used to release **unmanaged resources** (file handles, DB connections).

Implements `IDisposable`.

What is Finalize?

Destructor called by GC before object is removed.

Non-deterministic.

Dispose vs Finalize?

Dispose	Finalize
Manual	Automatic
Deterministic	Non-deterministic
Releases unmanaged resources	Backup cleanup
Faster	Slower

What happens if Dispose is not called?

- Resource leaks
- File locks remain
- DB connections exhausted
- GC eventually calls finalizer (if exists)

---

2 SOLID Principles

S – Single Responsibility

One class → One reason to change.

❌ Bad: UserService handles DB + Email
✔ Good: Separate UserService + EmailService

O – Open/Closed

Open for extension, closed for modification.

Use interfaces instead of `if/else`.

L – Liskov Substitution

Child class must not break base behavior.

If Bird has Fly(), Penguin shouldn't inherit.

I – Interface Segregation

Don't force classes to implement unused methods.

Split large interfaces.

D – Dependency Inversion

Depend on abstraction, not concrete classes.

Controller → IOrderService
Not → SqlOrderService

---

3 Dependency Injection

How DI works in .NET Core?

- Services registered in IServiceCollection
- Container resolves constructor dependencies
- Uses reflection

Service Lifetimes

Lifetime	Meaning
Transient	New instance every time
Scoped	One per request
Singleton	One for entire app

IoC?

Inversion of Control — framework controls object creation.

DI Frameworks Used?

- Built-in DI
- Unity (legacy)
- Ninject (legacy)

Modern projects → Built-in DI.

---

4 LINQ

IQueryable vs IEnumerable

IEnumerable	IQueryable
In-memory	Database
Executes immediately	Translates to SQL
After ToList()	Before ToList()

Use IQueryable for DB queries.
Use IEnumerable for in-memory collections.

---

5 Unit Testing

Used:

- xUnit
- NUnit
- Moq

Why mocking?

- Isolate business logic
- Avoid real DB/API
- Control behavior

Moq creates fake implementations of interfaces.

TDD?

1. Write failing test
2. Write code
3. Refactor

---

6 MVC Request Lifecycle

---

1. User hits URL
2. Routing matches controller
3. ControllerFactory creates controller
4. Action executes
5. Filters execute
6. View rendered (Razor)
7. Response returned

HttpContext?

Contains:

- Request
- Response
- User
- Session

Razor Rendering?

.cshtml → Compiled into C# class → Generates HTML.

---

7 REST vs SOAP

REST	SOAP
Lightweight	Heavy
JSON	XML
Stateless	Stateful possible
Faster	More secure enterprise features

REST preferred due to simplicity and performance.

---

8 API Versioning

Route versioning:

```
[Route("api/v1/orders")]
```

Middleware:

```
services.AddApiVersioning();
```

---

9 Consuming External APIs

Use HttpClient:

```
await client.GetAsync(url);
await client.PostAsync(url, content);
```

Pass headers:

```
client.DefaultRequestHeaders.Add("Authorization", "Bearer token");
```

Always use async/await.

---

10 SQL Fundamentals

Why indexes?

Improve query performance.

Clustered vs Non-clustered

Clustered → Physical order
Non-clustered → Pointer-based

Stored Procedure vs Function

SP	Function
Can do DML	Cannot modify data
Multiple result sets	Single result
Can return int	Returns value

Execution Plan?

Shows query execution steps.

Blocking session?

Transaction holding lock prevents others.

---

11 Cloud & DevOps

Used:

- Azure App Service
- Azure SQL
- Azure Service Bus
- Azure Monitor
- Elastic + Kibana
- CI/CD with Azure DevOps

Health Check Endpoint

```
app.MapHealthChecks("/health");
```

---

12 Performance & Background Jobs

If upload takes 5 minutes:

- Use background job
- Return immediate response
- Notify when done

Tools:

- Hangfire
- Azure Functions
- Service Bus

Use CancellationToken for cancel support.

---

13 Agile

Story point = effort estimation.

Sprint length: 2 weeks (typical).

Hotfix branch:

- Created from production branch
- Merged back after fix

Retrospective:

- What went well
- What didn’t
- Improvements

Sprint success measured by:

- Delivered story points
- Quality
- No production bugs

🔥 Final Senior-Level Closing Line

"I combine strong fundamentals in C#, SOLID principles, and dependency injection with practical experience in SQL optimization, cloud deployment, microservices, identity integration, and production troubleshooting to build scalable and maintainable enterprise systems."

If you want, I can now convert this into a **rapid-fire revision cheat sheet for interviews.**